RWTH Aachen University
Software Engineering Group

Identification and evaluation of features for detecting generatable code

Master Thesis

presented by
Cruz Zapata, José Manuel

1st Examiner: Prof. Dr. B. Rumpe

2nd Examiner: Prof. Dr. Horst Lichter

Advisor: M. Sc. Klaus Müller

The present work was submitted to the Chair of Software Engineering

Aachen, 20th September 2016

The present translation is for your convenience only.
Only the German version is legally binding.

Statutory Declaration in Lieu of an Oath

Last Name, First Name _____ Matriculation No. (optional) _____

I hereby declare in lieu of an oath that I have completed the present Bachelor's thesis/Master's thesis* entitled

_____

_____

_____

independently and without illegitimate assistance from third parties. I have use no other than the specified sources and aids. In case that the thesis is additionally submitted in an electronic format, I declare that the written and electronic versions are fully identical. The thesis has not been submitted to any examination body in this, or similar, form.

Location/City, Date _____ The German version has to be signed. _____ Signature

*Please delete as appropriate

Official Notification:

Para. 156 StGB (German Criminal Code): False Statutory Declarations
Whosoever before a public authority competent to administer statutory declarations falsely makes such a declaration or falsely testifies while referring to such a declaration shall be liable to imprisonment not exceeding three years or a fine.

Para. 161 StGB (German Criminal Code): False Statutory Declarations Due to Negligence
(1) If a person commits one of the offences listed in sections 154 to 156 negligently the penalty shall be imprisonment not exceeding one year or a fine.

(2) The offender shall be exempt from liability if he or she corrects their false testimony in time. The provisions of section 158 (2) and (3) shall apply accordingly.

I have read and understood the above official notification:

City, Date _____ The German version has to be signed. _____ Signature

Abstract

Model Driven Engineering (MDE) has established as one of the main software development paradigms favoring automation. In a system developed using MDE, part of the code is generated from models. MDE techniques can be introduced in existing systems to automate the generation of code that is at present manually written and maintained. In order to perform such a MDE re-engineering process, an analysis of the code of the system is required with the goal of identifying those code artifacts which are well suited for an automatic generation. We call such code artifacts **generation candidates**.

In this thesis, a method is proposed to provide automated support for the identification of generation candidates in a system. In order to do that, different **generatability features** are defined. When applied to or measured on existing code, these features identify which parts of it are suitable to be generated. We present and describe the ideas underlying eight different features, and provide an implementation for four of them. The goal of the features is not to completely automate the identification of generation candidates, but instead serve as a guide to the user during the process. The features are not only meant to be applied individually. Their combination is allowed and can on occasion be particularly helpful. The reported results must be subject of a further manual analysis in order to identify the truly interesting generation candidates. A case study is described where the implemented features are applied to a real-world system. The results obtained from this case study demonstrate that the features are able to identify many interesting generation candidates and provide great support for the re-engineering process.

Scope of work

In order to introduce the advantages of MDE into an existing system, a MDE re-engineering process must be carried out. Sometimes, a generative architecture is already defined and we want to extend it. On other occasions, the system is completely handwritten and a new generative architecture must be developed. However, a software system can usually not be generated entirely. Instead, only some of the code is generated while other parts remain handwritten. Therefore, one of the main problems when developing or extending a code generator is to identify which parts of the system can be generated and which should remain handwritten. This is a very laborious process which is usually performed completely manually and requires a deep understanding of the system. We denote by **generation candidates** those artifacts in the system that can be generated, and the process that leads to their detection **identification of generation candidates**.

The goal of this thesis is to define **generatability features** which allow for a semi-automatic identification of generation candidates. When applied to or measured on existing code, these features should tell us which artifacts or parts of artifacts are suitable to be generated. Thus, this thesis aims at simplifying the process of developing a generative architecture. With the help of our features, developers do no longer need to analyze the complete code base in order to identify generation candidates. Instead, they must only analyze the artifacts proposed by the features. In this way, we provide support and reduce the manual effort required for this step of the re-engineering process.

As part of the thesis, the features identified must be evaluated through its application to a real system. Consequently, it is necessary to provide an implementation to at least a subset of the presented features. The results obtained by this case study should provide insights regarding the validity and real usefulness of the ideas described in the thesis.

Contents

Chapter 1

Introduction

In the last years model-driven software engineering (MDSE or just MDE) has established as one of the main software development paradigms favoring automation [Sel12, WHR14]. It proposes a systematic use of models in the construction of software. Code generators are created and used to obtain an executable software application, transforming abstract models into instances of a general-purpose language (GPL) [FR07]. The MDE paradigm has proven its usefulness not only for new developments but also for software modernization and re-engineering projects [CIM09, BBI+13, JJ94, RPP+14].

Figure 1.1 shows a simplified view of a system developed using MDE. The system is composed of generated and handwritten code. In order to achieve the interaction between them, different mechanisms are available [GHK+15]. The generated code is produced from one or multiple models by a generator. A complete generation of the code is possible, but only if every single aspect of the system is modelled. Such a detailed modelling process is not feasible for most systems. Moreover, it is not convenient, since too much complexity would be put on the models, eliminating the benefits of the MDE approach.

In a typical MDE re-engineering context, this system structure is sought by introducing a new generative architecture in a so far completely handwritten system. In this way, automation is incorporated into part of the code, thus benefitting from MDE techniques. An alternative re-engineering context occurs when, in a system that already adopts the MDE paradigm, we want to find additional parts of it that can also benefit from an automatic generation. The goal in this situation is to identify code from the manually written code base that can be integrated into the already existing generative architecture, increasing the amount of code that gets automatically generated.

In both re-engineering contexts, an important step arises in the development or extension of code generators: the separation of handwritten and generated code. When introducing MDE techniques into an existing system, an important distinction needs to be made between the code that will remain handwritten and the code that will be integrated into the generative architecture. It is worth noting that, in theory, every class could be generated. In an extreme case, and if template-based generators are used, a class can be fully copied into a template containing only static template code. This is, however, not desired, following the guideline that only as much code should be generated as necessary [SVC06, KT08, Fow10]. Consequently, an analysis of the current code of the system is required with the goal of extracting those code artifacts that can better benefit from an automatic generation. From these ideas, two fundamental concepts in our thesis are introduced below.

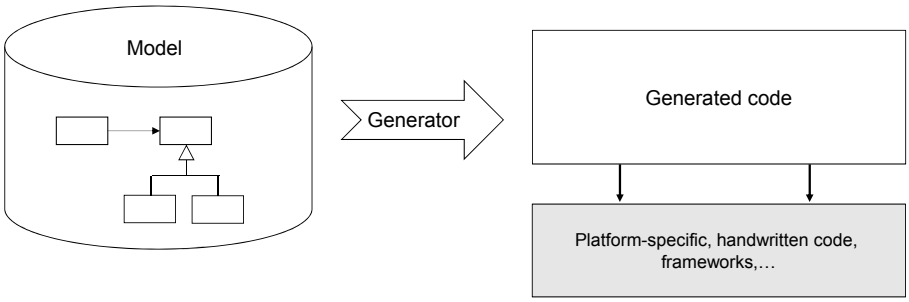


Figure 1.1: Structure of a MDE system where a generative architecture generates part of the code. Figure by Rumpe [Rum12].

Definition 1. ***Generation candidate****. Artifact that is suitable for an automatic generation.*

Definition 2. ***Identification of generation candidates****. The process that aims at identifying all the generation candidates in a software system.*

The identification of generation candidates is not a trivial process, since it requires a deep understanding of the software system in question as well as of the MDE field and its techniques. Therefore, expert engineers are required and the process entails considerable costs in time and resources. Problems arise particularly when dealing with a large code base. In such cases, it can be very hard even for experienced developers to know the whole system in detail, and the collaboration of additional domain experts may be needed. In conclusion, the manual identification of generation candidates is a laborious process, and the size and complexity of a software system can make the required effort excessive if no tool support is available.

This thesis proposes a method to partially automate the identification of generation candidates. The method is based on the definition of different **generatability features**, in the future simply referred to as features.

Definition 3. ***Generatability feature****. Measurable characteristic of a system or artifact which indicates its suitability for an automatic generation.*

The application of these features to a system helps us to identify generation candidates in it. The general process is depicted in Figure 1.2. In this process, at first the user applies one or more generatability features to the system. A set of reported generation candidates is obtained based on the results. The features can be applied individually, although the combination of certain features can also be of special interest. Then, the user needs to perform a detailed analysis of the generation candidates. Based on the artifacts suggested by the different features and the overall information shown in the results reports, the real generation candidates are finally extracted.

A post-analysis of the results is absolutely essential. The goal of the features defined in this work is not to completely automate the identification of generation candidates, but instead serve as a guide to the user during the process. Due to the complexity and subjectivity of the problem, we do not expect to correctly identify each and every interesting generation candidate. The data obtained from the application of the features is just intended to act as a starting point, providing supplementary information to assist the expert.

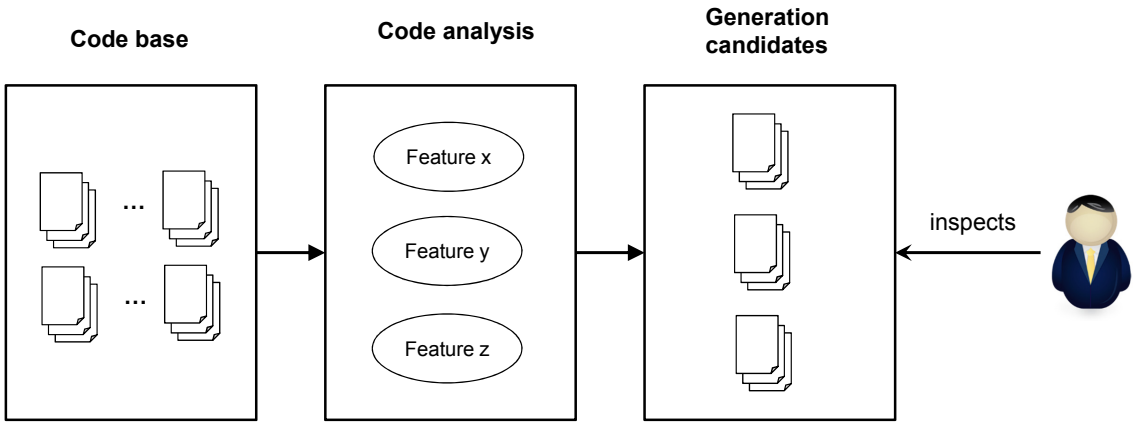


Figure 1.2: Identification of generation candidates process.

The rest of this thesis is structured as follows: Section 2 describes the state of the art regarding the identification of generation candidates, delving into the scarce literature that deals with this topic and the different and many fields related to it. In Section 3, we describe the ideas underlying the identified features, being this the main contribution of this work. A tool has been developed as a proof of concept of some of the features. Its overview is given in Section 4. Section 5 describes in more detail some of the interesting aspects about the implementation of the features that are included in the tool. A case study is presented in Section 6, where the tool is applied to a real-world system and the results are discussed. Finally, Section 7 ends with some conclusions and future lines of work.

Chapter 2

Related work

There is not much work to be found in the literature about the identification of generation candidates. The existing bibliography about MDE re-engineering does not usually focus on this particular step, which has traditionally been considered a manual part of the process. Consequently, automation efforts have been put into different areas, getting the identification of generation candidates little to no attention.

[NR15] is one of the few proposals that deal with our exact same problem. In this case, a semi-automatic categorization approach is described to classify all the classes in a software system, distinguishing between technology and domain-related classes. The idea is to identify and report the domain-related classes as generation candidates. This approach will be described in more detail in Section 3.1.4, in the context of one of our generatability features.

Some interesting work has been done in the general context of **MDE re-engineering**, although not focusing on the detection of generation candidates. [RGvD06] describes a re-engineering approach towards OMG's Model Driven Architecture (MDA). The legacy code is initially reverse engineered into a grammar, which is subsequently transformed into models at different abstraction levels. Further transformations are then performed to obtain UML models, which can be finally used to generate parts of the code of the new system. A similar re-engineering proposal is described in [QYCX03], where an intermediate language called Reengineering Wide Spectrum Language (RWSL) is used to build a MDA architecture from a legacy system.

Other works do not discuss a complete MDE re-engineering process, but deal with related fields such as **domain engineering**. [RB10] describes a proposal to automate the extraction of domain models from software systems. Information regarding the structure of a legacy system is obtained, which could be taken as the first step of a re-engineering process towards MDE. However, the system's code is not used as input. Instead, the method uses application models, matching, merging and generalizing the concepts in them into a domain model.

In the more general field of **software re-engineering**, plenty of work exists that resembles our proposal. Usually, the automation of a re-engineering process involves a phase where the code is reverse-engineered and analyzed. The result of this analysis guides the re-engineering process or tells which parts of the legacy system can benefit from it, in a similar way that our features help to identify generation candidates.

Component-based (CB) systems and **software product lines (SPL)** are two paradigms that gather a great amount of such re-engineering proposals. On the one hand, CB software engineering is a reuse-based approach to defining, implementing and composing loosely coupled independent components into systems [HC01]. [KSCC12] is a proposal for the automatic identification of components from existing software systems, where components are found and measured using a fitness function. A similar approach based on the existing dynamic dependencies between classes can be found in [ASSV10]. On the other hand, a SPL is defined as a set of software-intensive systems that share a common, managed set of features satisfying the specific needs of a particular market segment [Uni16]. Existing approaches to SPL re-engineering include proposals such as [MT12], where an automated process is described to look for reusable code by detecting clones, and [KKLK05], where a manual feature-oriented re-engineering process is presented. However, due to the natural differences between MDE and these paradigms, the methods proposed to support such re-engineering processes greatly differ from our feature-based approach.

Among the different factors that are analyzed and used in re-engineering processes, there is one that stands out, and that is **clone detection**. Clone detection is a very prolific research field on its own, with a wide scope of application. It can be very useful in re-engineering contexts to find pieces of similar code that may be later refactored, merged or, as it is our case, automatically generated. [JL06] presents a work related to our context, where clones are used in a generative programming context. Code clones are found and their generation is integrated in one or more templates, defined using a language called XVCL. However, the detection of the clones and their unification are performed through a manual analysis of the code. Clone detection is a very relevant field in the identification of generation candidates and it will be appropriately introduced and motivated in Section 3.3.

Apart from clone detection, other research fields are reused in this work for the conception of certain features. **Design pattern detection** [TCSH06, SO06, KGH06] and **software dependencies** [OG11] are two good examples, but the motivation behind their use along with some background on the fields will be provided respectively in Section 3.6 and Sections 3.7 and 3.8.

On a final note, our work, as well as some of the fields already mentioned, can be classified into the **software comprehension** field, insofar as it analyzes legacy code to help the user understand its structure and serve as a guide through the MDE re-engineering process. Software comprehension can be utilized for different goals, being re-engineering just an example. For instance, **software metrics**, taken as a software comprehension tool, can be used for maintenance [FMM94] or refactoring [TC09] purposes. [Sim15] uses metrics to measure the complexity of migrating a distributed system from one middleware to another, in a more re-engineering-oriented approach that resembles our proposal. A complete study of the state of the art in software comprehension can be found in [LELP02].

A wide range of related fields has been identified, and their understanding is important to provide a context for our proposal, while at the same time contributing to some of the ideas described in the identified generatability features. There is however a considerable lack of literature concerning the identification of generation candidates process, and hardly no proposals to support its automation.

Chapter 3

Generatability features

The main contribution of this work is the definition of generatability features for the identification of generation candidates in a software system. Different features are described, that analyze various aspects of the system's code. Every feature reports a set of generation candidates that the user can further analyze and filter. The identified features are described below, represented according to the following structure:

- **Motivation**. The idea behind the conception of the feature, that is, what its usefulness is and why it is interesting in order to detect generation candidates.
- **Description**. An explanation of how the feature is detected or measured and how the result can be used to obtain the candidates.
- **Application**. In some occasions, in order to apply a specific feature some preconditions need to be met. For example, a feature can be applied to the whole system or only to a subset of the software's code, e.g. artifacts of a specific type/language. It may also be interesting to combine it with other features. In this section, we address these circumstances and describe how and when the feature is applied.
- **Optimizations**. When appropriate, this section describes possible optimizations that can be done on the feature, extending the description given before.

Only the ideas underlying each feature are described. There is a strong presence of object-oriented terms in some of the features, and many of the examples may contain, for simplicity, code excerpts from a particular language. However, the main interest in this work is the conception of the ideas themselves and not their implementation. Consequently, the descriptions are kept as independent from a specific technology as possible. In fact, some of the identified features have not been eventually integrated into the prototype tool. For more details on this matter, we refer to Section 5.

Each feature can be taken and applied independently. However, there are some cases in which it is particularly interesting for two or more features to work together, so that the results of one of them can be used by the other. Some of these relations will be commented in the *Application* section of the features, although there is always freedom to interpret, implement and combine each feature in any way desired.

3.1 Identification of domain classes

3.1.1 Motivation

When developing a generative architecture, a model is defined that represents the main concepts of the system's domain and the relations between those concepts. From this domain model, different artifacts such as code artifacts or configuration files are generated. Therefore, when we want to define a generative architecture from an already implemented system, one of the first steps is to identify which are the concepts that are important for our domain. In order to do that, the code needs to be analyzed to find the classes that implement those concepts in the system. This is what we call **domain classes**.

In our context, domain classes can be directly presented as generation candidates. They are usually implemented as Plain Old Java Objects (POJOs), with private fields representing the attributes of the concept and simple `get` and `set` methods for each field. Therefore, in most occasions they can be easily generated and are in consequence good generation candidates. However, domain classes also play an important role in the identification of other generation candidates in the system. Since the domain concepts that they represent are eventually the origin of code generation, this information is important for the whole process and can be helpful in the application of other features. For this reason, the identification of domain classes is particularly useful as an auxiliary feature, intended to be combined with other features. The information it provides can be used to improve the efficacy or limit the applicability of other features. Some of these interactions will be commented in the corresponding *Application* sections.

The identification of domain classes can be classified as a **software categorization** technique. Software categorization deals with the classification of code artifacts according to different criteria. Proposals have been made for the automatic categorization of software at different levels, ranging from projects [TRP09, KGMI06] to components [SSS07]. In our context, a categorization approach is required which allows us to differentiate between the domain classes and the rest of artifacts in the system.

3.1.2 Description

Different approaches could be taken to identify the domain classes in a system. The simplest one, however, is to just request this information from the user. His knowledge about the system domain should be sufficient to effectively detect and indicate the classes in the system that represent these concepts, better than any automated approach.

3.1.3 Application

The domain classes are identified among all the classes in the system, with no particular restrictions. Additional artifacts in the system which are not classes can be disregarded.

3.1.4 Optimizations

We have proposed a completely manual identification of the domain classes. This approach is remarkable for its simplicity and the accuracy of its results. However, when dealing with

complex software systems, or systems that are not well structured, domain classes may not be that distinguishable. A manual identification can then be very laborious, even when performed by developers familiar with the code. As an alternative, different approaches exist to partly or completely automate the process.

[NR15] deals with this problem, and proposes a partial automation based on software categories. The approach arises in a context where a generative architecture is to be introduced. In order to do that, a distinction is made between the artifacts that will be generated and those which will stay handwritten. In this sense, the addressed problem is the exact same as the one we deal with in this thesis. By categorizing the software artifacts, this method aims at distinguishing between domain-related and non-related artifacts, so that only the former are considered for generation purposes.

Five different categories are defined:

- 0 classes. Global software that is well-tested and independent of the domain.
- Domain Global (DG) classes. Software that is global for the whole domain.
- Domain (D) classes. Software that is related to but not global for the whole domain.
- Technical (T) classes. Containing only technical software.
- Domain and Technical (DT) classes. Combination of domain and technical classes, that is, classes that refine both a domain and a technical category.

Figure 3.1a shows the resulting category after combining any two given categories, whereas Figure 3.1b illustrates the allowed dependencies between them. A free interpretation of the term dependency is allowed, although it will usually consider relations such as inheritance, instantiation or usage. By analyzing the dependencies between the classes, and with the information available in both tables, some categories can be automatically identified. For example, if a class depends on both a D-class and a T-class, then it is categorized as a DT-class. Since only other DT-classes can depend on a DT-class, then every other class that depends on it is categorized as a DT-class.

+	DT	D	T	DG	0’
DT	DT	DT	DT	DT	DT
D		D	DT	D	D
T			T	T	T
DG				DG	DG
0’					0’

(a) Addition of software categories

→	DT	D	T	DG	0’
DT	✔	✔	✔	✔	✔
D	✘	✔	✘	✔	✔
T	✘	✘	✔	✔	✔
DG	✘	✘	✘	✔	✔
0’	✘	✘	✘	✘	✔

(b) Allowed dependencies between categories

Figure 3.1: Relations between categories. Figure by Rumpe et al. [NR15].

The method proceeds in the same manner with the rest of the classes. Therefore, all we need is to initially categorize some of the classes manually. On the basis of this initial categorization, the procedure will try to automatically assign the rest of categories by analyzing the dependencies of the classes. This process is shown in Figure 3.2. Different iterations can be applied after further manual categorizations of some of the remaining classes, until all the classes are finally categorized.

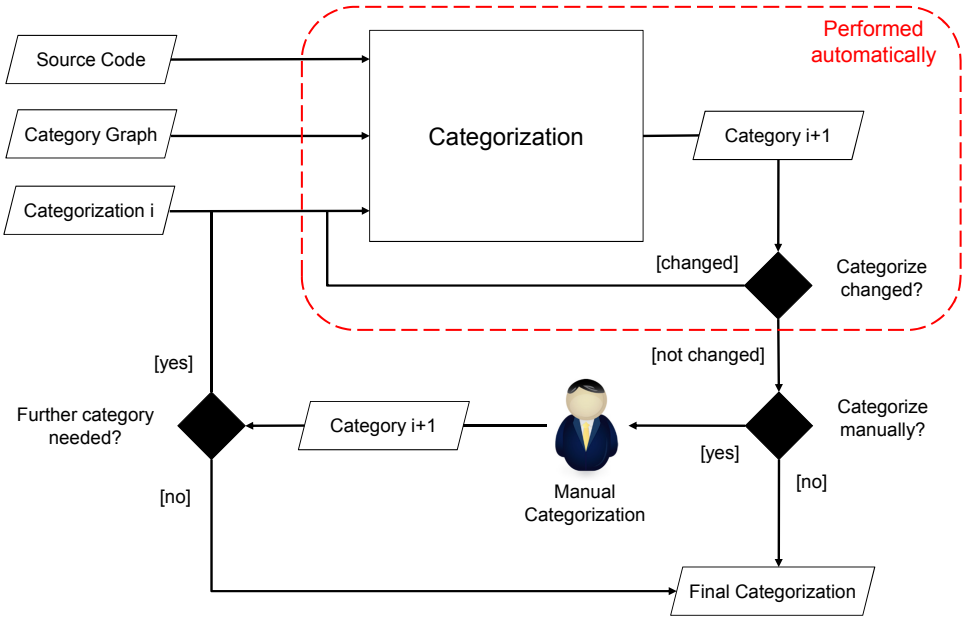


Figure 3.2: Overview of the categorization approach. Figure by Rumpe et al. [NR15].

This semi-automated process is of great interest for our purposes. Not only the domain-related classes are identified, but a complete categorization of all the classes in the system is available. In consequence, whereas the main goal highlighted by the paper is to propose D and DT classes as generation candidates, we can use the overall categorization in any other way desired. A feature could for example only be applied to D and DT classes, the ones related with the domain. Additionally, the generation candidates proposed by a feature can receive a special consideration if found to belong to a category T (not interesting for our purposes) or DG (specially interesting for our purposes). In conclusion, it is an optimization that would not only partially automate the identification process, but also provide some additional information that can be useful in other parts of the architecture.

A simpler alternative is possible when a domain model is already available. This is usually the case when a generative architecture has already been defined, or when models are employed for documentation purposes. In such cases, the concepts in the model can be extracted and directly reported as the domain classes of the system.

3.2 Name similarity

3.2.1 Motivation

With a generative architecture, code is generated from models, commonly using a template language [Acc16, Xpa16]. From one or several classes in the model, different artifacts can

be created, containing code with the implementation of different functions related to that class or classes. The artifacts generated from a model class will usually have a name related in some manner to the concept represented by that model class. If we assume a similar behavior from implementors when manually writing the code of a system, then we can take advantage of this situation to identify generation candidates. Just by analyzing the names of the files, we can identify and report groups of related artifacts and the concepts they are derived from.

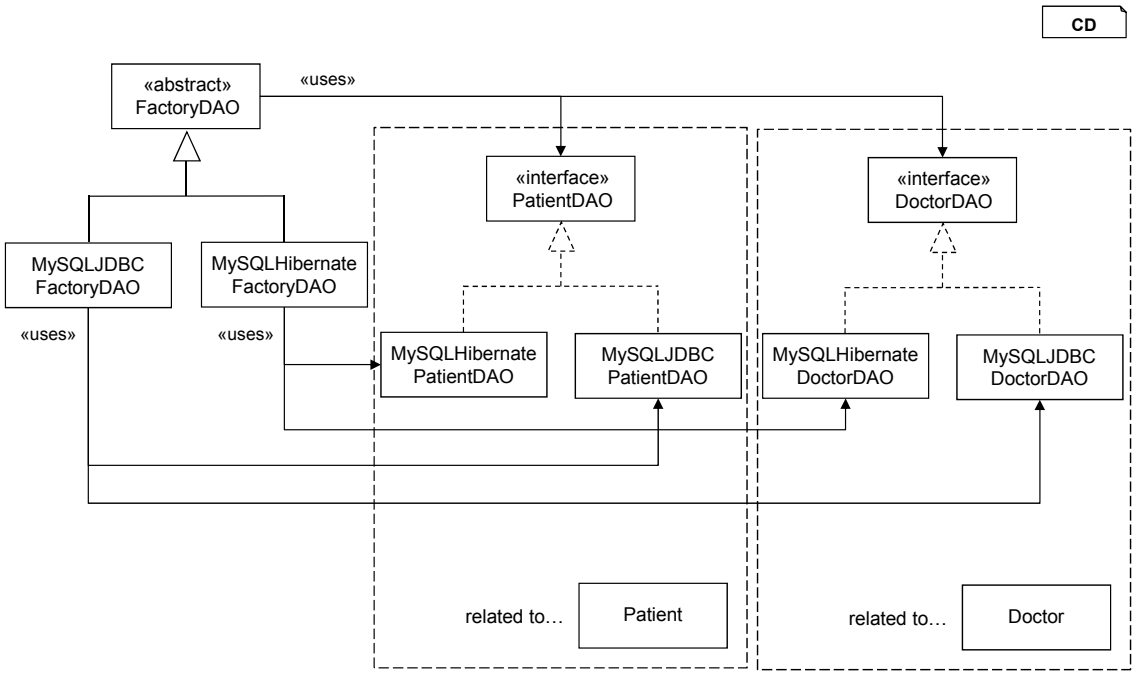


Figure 3.3: Group of related classes that implement the Factory and DAO patterns.

Figure 3.3 depicts a group of related artifacts in a health context system that serves as an example to illustrate this idea. The artifacts are an implementation of two design patterns: the Abstract Factory pattern [GHJV95] and the Data Access Object (DAO) pattern [AMC+03]. Two interfaces, `PatientDAO` and `HospitalDAO`, are defined to handle the database-related operations for the `Patient` and `Hospital` entities. Concrete implementations of these interfaces will work with different types of databases or database technologies. On the other hand, a `FactoryDAO` abstract class is defined to handle all the classes that implement the database operations for each entity. `FactoryDAO` makes use of `PatientDAO` and `HospitalDAO`, whereas the concrete implementations of `FactoryDAO` only know the DAO classes specific to each database type.

The resulting structure contains a number of artifacts related to each other in different ways. Interestingly enough, the relations between the artifacts show up by just inspecting their names. Even if we do not have the information about class hierarchies or interface implementations that we see in the figure, we can still correctly extract the relations between the artifacts just from their names:

- `FactoryDAO`, `PatientDAO` and `HospitalDAO` are all types of *DAO*, since they share that as a suffix.

- `MySQLJDBCFactoryDAO` and `MySQLHibernateFactoryDAO` are types of *FactoryDAO*, since they share that as a suffix.
- `MySQLJDBCPatientDAO` and `MySQLHibernatePatientDAO` are types of *PatientDAO*, in the same way that `MySQLJDBCHospitalDAO` and `MySQLHibernateHospitalDAO` are types of *HospitalDAO*.
- `MySQLJDBCFactoryDAO` is related to both `MySQLJDBCPatientDAO` and `MySQLJDBCHospitalDAO`, since they share the prefix *MySQLJDBC*, in the same way that `MySQLHibernateFactoryDAO` is related to `MySQLHibernate-PatientDAO` and `MySQLHibernateHospitalDAO`.
- `PatientDAO` and `HospitalDAO` are related to the `Patient` and `Hospital` entities, respectively, since the latter names are contained in the former.

This is a consequence of the logical and systematic way in which the artifacts are named. Although it is not strange to find occasional variations, in practice programmers tend to use these logical conventions when naming the artifacts in a software system. Overall, the analysis of the names is a very simple yet effective way of identifying groups of related artifacts. If artifacts are derived from others by their names, their inner code may also be related. Therefore, report them as generation candidates.

The name similarity feature is also particularly useful when combined with other features. Sometimes, the system is too big and we can not apply a feature to the whole code. Instead, we can use the name similarity feature as a first step, and only apply other features to the groups of related artifacts identified by it. In this way, additional measures are obtained and we can better understand the relations between the artifacts, obtaining from them the truly relevant generation candidates. However, it is worth mentioning that if we limit the application of other features like this, their usefulness will be restricted by the candidates identified by the name similarity feature, thus possibly under-using their real potential.

3.2.2 Description

In order to identify groups of related artifacts in the context of name similarity, we define a simple method where the names of the artifacts are processed and replacements are performed. Our method to assess name similarity consists of the following steps:

1. **Sort the names of the artifacts by decreasing length.**

   This is done so that, in subsequent replacements, the longest matching name will always be replaced first.

2. **Search the name of each artifact in the names of the others, in order, and replace it by a unique identifier.**

   All the artifacts named from different entities using the same conventions will be made equal after this step. The information about the original string being replaced must be kept.

3. **Look for artifacts that have the same name, and report them along with the artifact whose name was used in the replacement/s.**

   A group of related artifacts is formed by one or several *main* artifacts and those other artifacts whose names were derived from them.

To illustrate the process, we use once again the classes depicted in Figure 3.3 on page 11. This example contains different artifacts related to each other in distinct ways, and the names have been assigned in a way that favors the automatic identification of such relations. When applied on this example, the method works in the following way:

1. Sort the names of the artifacts by decreasing length.

The resulting list of names is the following:

- `MySQLHibernateHospitalDAO`
- `MySQLHibernatePatientDAO`
- `MySQLHibernateFactoryDAO`
- `MySQLJDBCHospitalDAO`
- `MySQLJDBCPatientDAO`
- `MySQLJDBCFactoryDAO`
- `HospitalDAO`
- `PatientDAO`
- `FactoryDAO`
- `Hospital`
- `Patient`

2. Search the name of each artifact in the names of the others, in order, and replace it by a unique identifier.

Replacing each name by the unique identifier X, the resulting list of names is the following:

- `MySQLHibernateX` (`HospitalDAO` was replaced)
- `MySQLHibernateX` (`PatientDAO` was replaced)
- `MySQLHibernateX` (`FactoryDAO` was replaced)
- `MySQLJDBCX` (`HospitalDAO` was replaced)
- `MySQLJDBCX` (`PatientDAO` was replaced)
- `MySQLJDBCX` (`FactoryDAO` was replaced)
- `XDAO` (`Hospital` was replaced)
- `XDAO` (`Patient` was replaced)
- `FactoryDAO`
- `Hospital`
- `Patient`

3. Look for artifacts that have the same name, and report them along with the artifacts whose names were used in the replacement/s.

The following groups of related artifacts are detected in our example:

- **Name of the group:** `MySQLHibernateX`

  Contained artifacts:

  - `MySQLHibernateHospitalDAO`, derived from `HospitalDAO`.
  - `MySQLHibernatePatientDAO`, derived from `PatientDAO`.
  - `MySQLHibernateFactoryDAO`, derived from `FactoryDAO`.

- **Name of the group:** `MySQLJDBCX`

  Contained artifacts:

  - `MySQLJDBCHospitalDAO`, derived from `HospitalDAO`.
  - `MySQLJDBCPatientDAO`, derived from `PatientDAO`.
  - `MySQLJDBCFactoryDAO`, derived from `FactoryDAO`.

- **Name of the group:** `XDAO`

  Contained artifacts:

  - `HospitalDAO`, derived from `Hospital`.
  - `PatientDAO`, derived from `Patient`.

Each artifact has a name that was derived from the name of another artifact. This derivation has the same form for every artifact in each group. If we suppose that this similarity also applies to the inner code of the artifacts in every group, then the generation of these artifacts could be automated. However, we should not jump to such conclusions from just the names of the artifacts. Instead, further measures should be taken on the artifacts of each group by applying additional features. For instance, we can calculate the similarity of the inner code of the artifacts by applying the custom clone detection feature. This possibility is described in Section 3.4.3.

It is worth noting that the proposed method assumes that the artifacts are named in a systematic and consistent way, and are derived from other names following fixed rules. This is, however, not always the case when dealing with real systems, since it is not strange to find little variations or exceptions in the naming policies being adopted. In order to solve these problems, heuristics can be used. We show some examples of such heuristics in Section 3.2.4, as extensions of the process that we have described here.

3.2.3 Application

There are no restrictions to apply this feature, since all that a implementation of this method needs are the names of the files. Consequently, not only code classes but any file in the system with any extension can be used. This is helpful to not only detect classes related to other classes, but also, for example, configuration files or scripts that can be generated.

Alternatively, instead of trying to find every file name in every other file name, the set of names that are searched and replaced can be defined separately. This allows for some interesting modifications of the process. For example, the names of the files to be used in the replacements can be provided by the user directly as strings, instead of files. This way, if the user knows additional model concepts that should be checked, it is not necessary for an artifact to be defined for that concept in the system. This can be specially useful when a generative architecture already exists and a model is available. Not all the model classes may have a corresponding implementation class, but their names can still be submitted to take part in the process and identify artifacts in the system related to them.

Additionally, the domain classes of the system, as defined in Section 3.1, can be used if they have been identified. Since they are considered the starting point in the generation process, it can be argued that we should only look for artifacts that are derived from the domain classes. Consequently, an alternative is to only use the names of the domain classes in the replacements, so that only the groups of artifacts related to the domain classes are reported as generation candidates.

3.2.4 Optimizations

Heuristics

In order to add some flexibility to the process and be able to support cases where the naming rules are not completely strict but still interpretable, different heuristics are defined in the following. They deal with common situations where the derivation of the names can be slightly different from what is expected. Therefore, they can help to detect additional candidates if they are added to the implementation of the basic method.

Plural forms

Example: `AppointmentsRecord` and `Appointment`.

Although singular forms are often the ones used to name all the concepts in a system, sometimes the plural form is taken instead. Depending on the case, the relation may still be identified, because the singular form is often contained in the plural form. But if that is not the case, or if the search is made in the opposite way, the artifacts are not identified by our method. To implement this heuristic, a procedure can be defined to identify the different words that form each file name. This could be easily done if we suppose that the Java UpperCamelCase naming convention is used. Once we have the words, all of them would be converted to their corresponding singular form, and only after that the replacement would be attempted.

Getting the singular form of a word can be done by either a *stemming* or a *lemmatization* process [Sta16a], concepts related to the field of natural language processing (NLP) [ID10]. Different tools and libraries can be used for this purpose, such as the Stanford CoreNLP tool suite [Sta16b].

Synonyms

Example: `PhysicianLicense` and `Doctor`.

Another problematic situation, although less common, is the use of synonyms. It can happen that along a software system, artifacts related to the same concept are named

differently because of the use of synonyms. Syntactically, such words can greatly differ, and therefore the relation will not be identified by our method. Again, we need to separate the different words forming the file name. Once we have identified these words, a unique form is obtained for each of them, so that every synonym is converted into the same word. Lemmatization arises again as the required process, and tools like WordNet [Pri16] can be used to perform this kind of unification.

Derived words

Example: `SurgeryDepartment` and `Surgeon`.

An additional, analogous case is that of derived words. Different derivations of one domain concept can form the names of new artifacts in a system. As it was the case with plural forms, sometimes the derived word can contain the word it is derived from, if just a prefix or suffix is added to it. However, this is not always the case, and so our basic method will not necessarily identify such cases completely. Once again, a lemmatization process needs to be performed on the individual words that form every file name, which can be made with tools such as the already mentioned WordNet.

Abbreviations

Example: `OCDepartment` and `OutpatientClinic`.

The concatenation of names is usual when new concepts are defined from others. Sometimes, the resulting name is too long and abbreviations are used. This approach may not be optimal, since it hinders the understanding of the concept, but it is definitely a common practice in software systems. To identify related artifacts when abbreviations are used, a simple heuristic can be defined, where the initials of the words forming each artifact's name are also searched during the replacements.

Inexact matches

Example: `SurgeryDept` and `Department`.

This heuristic differs from the others, since its intention is not to make similar words equal, but to just tolerate a certain degree of inequality instead. A threshold value is set and the replacement is only done when the word being searched is found in a similar enough form in the file name, according to such threshold. In this way, words that only differ in one or a few letters could be considered equal. This heuristic is more general, and depending on the threshold the results can be more or less accurate. However, it can help to detect additional cases or just be used as a simple alternative to the heuristics described before.

3.3 Clone detection

3.3.1 Motivation

One of the key advantages of adopting a MDE approach is the decrease in coding effort, since part of the code is usually generated from the developed models. To accomplish this, template languages such as Xpand [Xpa16] or Acceleo [Acc16] are often used. These template languages work by specifying static and dynamic portions of text, so that some code is always directly output and some other is dynamically generated according to the characteristics of the model. While the dynamic parts introduce differences between the

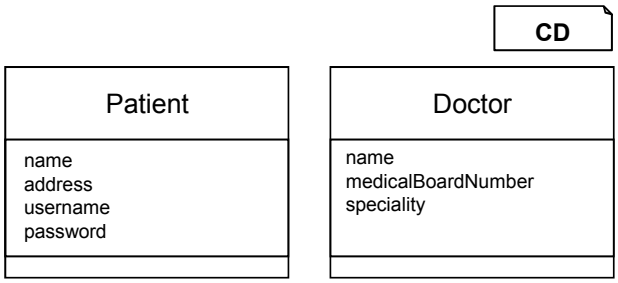


Figure 3.4: `Patient` and `Doctor` classes used in the generation example.

artifacts, the statically generated code is the same for every artifact generated using the template.

As an example, Listing 3.1 and Listing 3.2 show database-related Java classes generated from the two model classes `Patient` and `Doctor`, depicted in Figure 3.4. The template used for this generation is not listed for reasons of brevity, but the dynamic parts have been highlighted in the listings. Most of the code is statically generated and it is the same for both classes, which only differ in a few details. Of course, this is a very simple example. In real systems, templates tend to be more complex and the differences bigger. However, it should help to illustrate the idea that similar code is, to a greater or lesser extent, always present when adopting a generative approach.

```
1  public class PatientDAO {
2   private final SessionFactory sessionFactory;
3
4   public PatientDAO(SessionFactory sf) {
5    this.sessionFactory = sf;
6   }
7
8   public Patient addPatient(String name, String address,
9     String username, String password) {
10   Session session = sessionFactory.openSession();
11   Transaction tx = null;
12   Patient patient;
13
14   try {
15    tx = session.beginTransaction();
16    patient = new Patient(name, address, username, password);
17    session.save(patient);
18    tx.commit();
19   } catch (Exception e) {
20    if (tx != null) tx.rollback();
21   } finally {
22    session.close();
23   }
24   return patient;
25  }
26 }
```

Listing 3.1: Database access class generated from the `Patient` model class.

```
1  public class DoctorDAO {
2   private final SessionFactory sessionFactory;
3
4   public DoctorDAO(SessionFactory sf) {
5    this.sessionFactory = sf;
6   }
7
8   public Doctor addDoctor(String name, long medicalBoardNumber,
9     MedicalSpeciality speciality, String username,
10    String password) {
11   Session session = sessionFactory.openSession();
12   Transaction tx = null;
13   Doctor doctor;
14
15   try {
16    tx = session.beginTransaction();
17    doctor = new Doctor(name, medicalBoardNumber, speciality, username,
18      password);
19    session.save(doctor);
20    tx.commit();
21   } catch (Exception e) {
22    if (tx != null)
23     tx.rollback();
24   } finally {
25    session.close();
26   }
27   return doctor;
28  }
29 }
```

Listing 3.2: Database access class generated from the `Doctor` model class.

In a reverse-engineering context, this situation can be turned over. We assume that, because generative techniques tend to produce similar code, handwritten code with a high similarity can be easier to generate. Similar code is therefore seen as an opportunity to introduce a generative architecture that puts in common such similarities and automates the generation of the code.

In this sense, clone detection arises as a tightly related research field. A code clone is defined as a code portion in source files that is identical or similar to another [KKI02]. Code clones are considered bad smells of a software system [FB99]. They can reduce the maintainability [Bak95, Joh93] and introduce subtle errors [LLMZ06, CYC+01], among many other associated problems [RC07]. Clone detection deals with this situation by proposing tools to help the user to identify code clones. Once identified, the clones are analyzed and treated accordingly. Usually, refactoring actions need to be carried out in order to remove the clones, while in other cases it may be better to keep them in the system by applying maintenance techniques [KN05]. In our context, we can reuse these tools and report the detected code clones as generation candidates.

3.3.2 Description

Depending on the clone detection approach adopted, different types of clones can be detected. The most widespread classification [RCK09] considers four types of code clones:

- **Type-1**. Identical code fragments except for variations in whitespace, layout and comments.

  While it is true that equal code can be generated together statically from a template, generating code that is exactly equal is not the goal of a generative architecture. However, the existence of Type-1 clones can have some interesting implications. For example, multiple Type-1 clones can be detected throughout the code of two different artifacts. Depending on the case, a template can be created which defines the clones as static sections and integrates the code that has not been detected as clones into dynamic sections, thus reaching a common generation of the artifacts. Therefore, Type-1 clones are not particularly interesting in our context, but nevertheless can, on occasion, lead to the identification of good generation candidates.

- **Type-2**. Syntactically identical fragments except for variations in identifiers, literals, types, whitespace, layout and comments.

  Type-2 clones allow a small degree of variability in the clones. Such variations can be often easily integrated into a generative architecture, due to their simplicity. For example, we can identify two artifacts that define a similar functionality where only the types and names of the variables are changed. A common generation of these artifacts can be reached by defining a template such that a) each variable's name and type is defined in a dynamic section as dependent on the model and b) the rest of the code is defined in a static section. Although the interest of implementing such a generation depends on the case, in general Type-2 clones can be easily generated together and therefore are good generation candidates.

- **Type-3**. Copied fragments with further modifications such as changed, added or removed statements, in addition to variations in identifiers, literals, types, whitespace, layout and comments.

  Type-3 clones allow for an even greater degree of syntactic variability. Due to the added flexibility, the amount of reported clones is greater than if only Type-1 or Type-2 are supported. Furthermore, the changes in the clones are still easy to integrate into a generative architecture. Modified, added or removed statements can be produced by dynamic sections in templates. Therefore, Type-3 clones can also be good generation candidates, and supporting their detection is recommended in our context.

- **Type-4**, also known as **semantic clones**. Two or more code fragments that perform the same computation but are implemented by different syntactic variants.

  Type-4 clones disregard the syntax of the code and only focus on the semantics. This means that two artifacts can have any degree of syntactic similarity, even be completely different, and still be detected as Type-4 clones if they are found to perform the same computation. On the contrary, code generation is fundamentally a syntactic approach. Templates allow us to generate code that shares similar parts or an overall common structure, but always from a syntactic point of view. Two artifacts that are completely different can therefore not be generated together, regardless of the fact that they are semantically equivalent. In consequence, and even though some clones may have similarities that make them good generation candidates, in general Type-4 clones are not interesting for our purposes.

In order to automatically detect the clones in a system, tools provide an implementation of the general process depicted in Figure 3.5. The process is composed of the following steps:

1. **Preprocessing**. Uninteresting code is removed, e.g. embedded code such as SQL embedded in Java or Assembler in C.

2. **Transformation**. The preprocessed code is extracted and normalized to obtain an intermediate representation of the code. **Extraction** refers to the process where the source code is turned into a form that is suitable as input to the comparison algorithm. A typical extraction technique is, for example, **tokenization**, where each expression is divided into tokens [ALSU06] according to the lexical rules of the programming language. Before or after the extraction process, **normalization** actions can be applied to eliminate superficial differences. Examples of these actions are the removal of whitespace and comments or the normalization of the identifiers by replacing all the names by the same unique identifier.

3. **Match detection**. The transformed code is fed into a **comparison algorithm**. The output of this algorithm is a list of matches in the transformed code which is represented or aggregated to form a set of candidate clone pairs.

4. **Formatting**. Clone pair/class locations of the transformed code are mapped to the original code base by line numbers and file location.

5. **Post-processing: Filtering**. In this post-processing phase, clones are extracted from the source, visualized with tools and analyzed to filter out false positives.

6. **Aggregation**. In order to reduce the amount of data or for ease of analysis, clone pairs (if not already clone classes) are aggregated to form clone classes or families.

Section 5.2 deals with the implementation of the feature, for which an already available tool is selected. The results of the clone detection process are presented directly to the user as generation candidates.

3.3.3 Application

Clone detection tools are usually applied to the whole software system. However, they can also be applied to any other subset of artifacts inside the system. Some tools may only support a specific programming language, while others accept any file. However, this depends on the specific tool selected. The general clone detection process as described in this section defines no restrictions on the input code.

3.4 Custom clone detection

3.4.1 Motivation

The usefulness of code similarity as an indicator for generation candidates was previously introduced in Section 3.3.1. Code generators produce code from other artifacts, and when

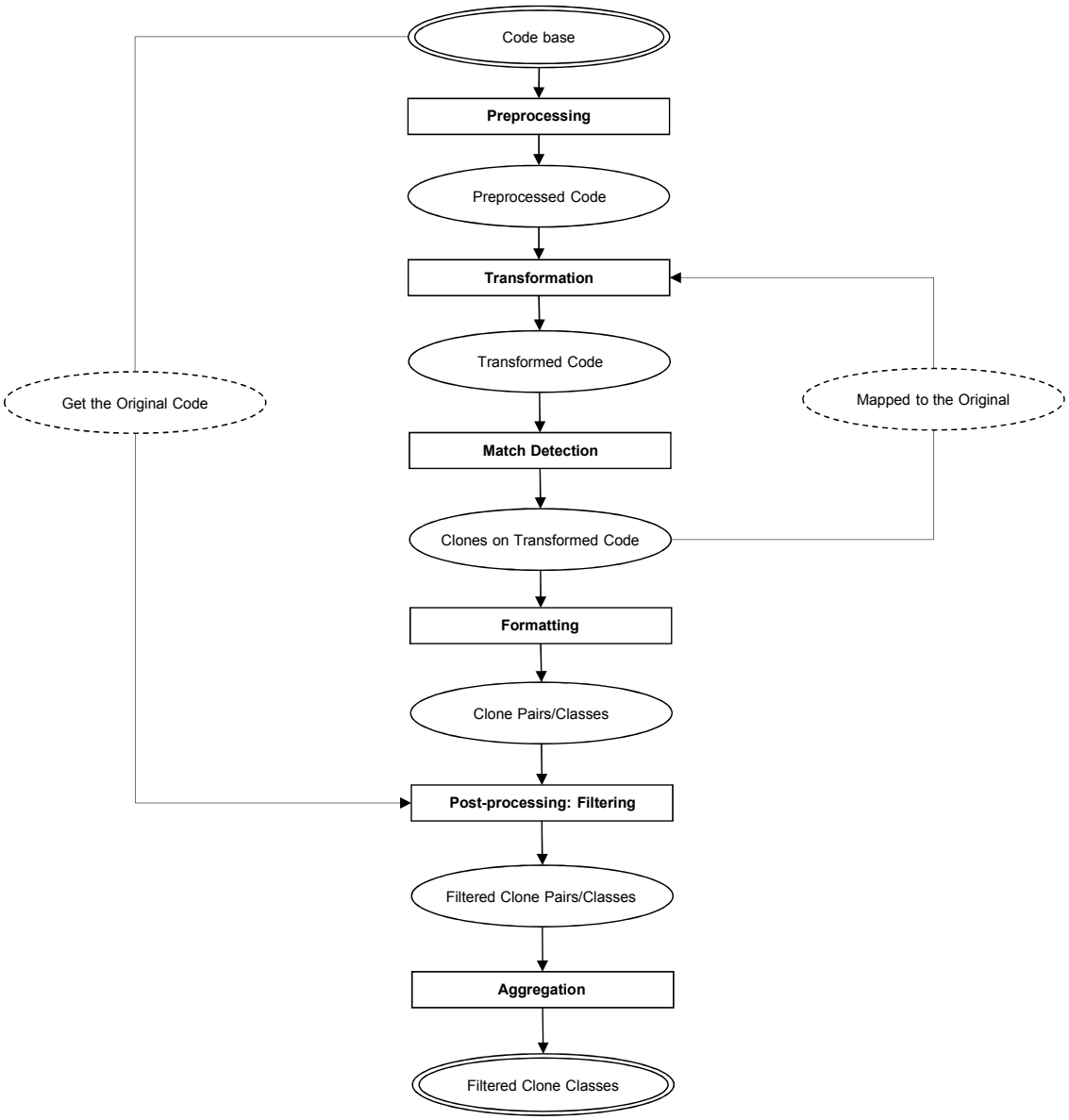


Figure 3.5: A generic clone detection process. Figure by Roy et al. [RCK09].

two pieces of code are generated using the same logic, there is usually a significant portion of them that comes out as equal. Existing similar code in a system can therefore represent a set of potential generation candidates. Clone detection was introduced as a promising field to take advantage of this circumstance. Using a clone detection algorithm, we are able to identify code clones, that is, pieces of code that are identical or similar. The detected clones are reported to the user for a further, manual analysis that eventually identifies the code clones that are truly interesting for generation purposes. However, we have detected that, in our context, standard clone detection tools have certain shortcomings.

We described how different clone types are defined, which allow for a different level of variability between the clones. As the variability increases, clone detection tools need to implement more aggressive normalization actions in order to eliminate additional differences between the clones. However, the adoption of such actions entails a certain risk, and the results may be less accurate as the supported variability increases. In the end, every clone detection tool needs to find a compromise between the variability supported in the clones and the accuracy of the results.

In order to solve this problem, we propose a custom clone detection method where multiple **pre-processing steps** are defined, each one implementing more aggressive normalization actions. A similarity measure is calculated after each step. In this way, we provide a greater flexibility in the process. The user can configure the levels of pre-processing to be applied to the code. A combined result is presented, but the result of each applied step is also available, thus providing a more complete report about the similarity of the code.

Additionally, we customize the method to better adapt to our context by defining generation-related pre-processing steps. These steps implement actions tailored for the identification of generation candidates. Furthermore, some other general optimizations are done, which most clone detection tools do not implement. One example is the removal of simple `get` and `set` methods. These methods can greatly affect the results, despite being in most occasions not relevant at all.

Finally, we have also detected some limitations in the way most standard clone detection tools present their results. Some tools just report a list with the detected code clones, while others also provide a similarity measure for each pair of clones detected. In the latter case, a threshold value is usually defined so that only clones with a high enough similarity get reported as such. However, such value is not always customizable. This poses a problem since, in our context, generation candidates do not always need to have a very high similarity to be interesting. We just need them to be related enough so that a common generation is possible and profitable. Therefore, there can be considerable differences in the two pieces of code and still they can be good generation candidates, since the creation of those differences could be integrated into the generator logic. Even if we can only generate together a small portion of those pieces of code, it could still be interesting depending on the situation. As a consequence, a greater flexibility is desired in our context when presenting the results. Instead of implementing a fixed filtering process which could deprive the user of some interesting results, we propose to directly report all the similarity results. This means that the user has the complete information about the existing similarity in the code. This fosters a further, more detailed analysis, where the user himself can sort and filter the results according to his own criteria. While this is a tool-dependent issue, it is one that differentiates our method from most available standard clone detection tools.

In short, our custom clone detection method is an attempt to adapt the general clone detection process to the identification of generation candidates, focusing on the results of a more complete similarity calculation process and using the knowledge specific to our context to the greatest advantage. The method is described in more detail below. An example is included, which helps us to illustrate the ideas presented and to discuss the advantages and disadvantages of the method.

3.4.2 Description

We have defined a method to customize the standard clone detection process and reach a solution more suitable for our context. The input of our method is a pair of **code units**, defined as a syntactically valid block of code. A code unit can be a whole file, a class or even just a method. The output is a similarity measure specified in some quantitative way, e.g. a percentage. Ideally, an implementation of this method will also provide a list with the actual parts of the code that have been found similar, but this is not strictly necessary for our purposes.

Our custom clone detection method is based on a four-step pre-processing structure, depicted in Figure 3.6. Both code units are pre-processed following four different, consecutive steps, such that the output of one step is the input of the following. After each step, the corresponding transformed code is obtained and both code units are fed to a matching algorithm in order to obtain a similarity measure. In this way, four measures are obtained, representing the similarity between the code units after each pre-processing step. This several-step pre-processing structure allows us to implement different actions separately and obtain different individual results to gain a deeper understanding of the similarity between the code units, allowing for more insightful interpretations than the one we could obtain from a single, final result. The approach will be further motivated during this section and illustrated with some examples.

The four-step schema depicted in Figure 3.6 can be extended, incorporating additional pre-processing steps if necessary. Our proposal consists, however, of four steps, which will be presented below. Steps 1 and 4 include some of the usual actions performed by most clone detection algorithms, while steps 2 and 3 are generation-specific pre-processing steps where some actions related to our context are applied to the code units. Additionally, an extra measure could be obtained in the first place by comparing the two code units in their original form, which can be used as an optional basic similarity measure.

In order to perform generation-specific pre-processing on the code, it is required for each input code unit to be related to another class that we call domain class, as introduced in Section 3.1. The domain class is used to try to reduce the domain-related differences between the two code units during the generation-specific pre-processing steps. If not specified, the process can still be applied, but those specific steps will be useless and therefore the results much more limited.

The method is an adaptation of the general clone detection process, as depicted in Figure 3.5 on page 21. The code units first go through a basic pre-processing and are transformed afterwards. Only then the four steps are applied to the code, and the similarity calculation measures are obtained during the match detection phase for each step. Final formatting and post-processing phases are also required to present each individual measure to the user, as well as to calculate some final result that summarizes all the individual similarity results.

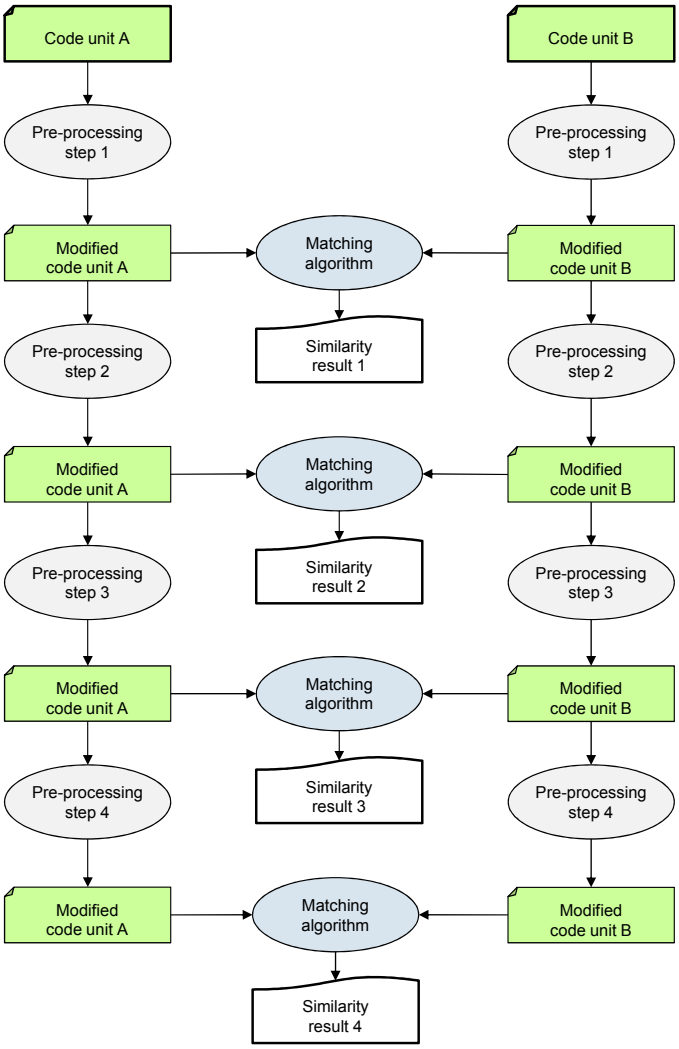


Figure 3.6: Four-step pre-processing scheme to calculate the similarity between two code units.

However, while the method can be integrated in such a manner into the general clone detection process, our intention is to solely present its main contribution, that is, the four-step pre-processing structure and the actions defined in each step. It is, in consequence, not relevant for us at this point how the rest of the process is implemented. Therefore, the technology and even the technique employed to process the code are not of our concern here. Neither are the way the different actions defined in each step are performed nor the way the results are post-processed and presented to the user. In the same manner, the actual matching algorithm to be used in the comparison is not part of the method that we have defined, although it is undoubtedly a detail that has a great influence on the final results. For us, these are implementation-related decisions and we address them in Section 5.

Examples introduction

In order to illustrate how the method works, we have defined a couple of example cases that show the result of applying each pre-processing step and help to understand the usefulness of the actions in them. We use three different Java classes to define two clone detection examples:

- `HospitalDAO` - `AppointmentDAO`. These two classes share a decent amount of their code, but are not completely suitable to be generated together.
- `HospitalDAO` - `PatientDAO`. These two classes share a slightly bigger portion of their code, and are clearly suitable to be generated together.

The set of classes has been extracted from a real, small system, and slightly adapted so that it can better show the benefits of most pre-processing actions. It consists of three classes that implement the Data Access Object (DAO) design pattern [AMC+03], and include only a few lines of code with a method accessing the database in each of the classes. We decided to use Java classes because we think that a real, definite example is better than a generic one for illustrative purposes. In consequence, some of the actions are Java-specific. However, most of the process stays the same regardless of the programming language used in the code units.

For each pre-processing step, we show the resulting code of each pair of classes after the transformation actions, highlighting the identical code so that the similarity can be easily perceived.

`HospitalDAO` and `AppointmentDAO`

Figure 3.7 shows the original code of this first pair of classes. `HospitalDAO` defines three attributes, a couple of simple `get` and `set` methods for one of the attributes and a `registerHospital` method that is the main part of the class. This method creates a new `Hospital` object from the parameters' values, performs an additional operation regarding a registry and finally accesses the database to store the new entity. On the other hand, `AppointmentDAO` defines only one attribute with its corresponding simple `get` and `set` methods, and a `getAppointments` method to access the database. In this case, a `Patient` object is obtained from the `ID` parameter and their appointments are obtained and returned.

The two classes have database-related functions and in consequence share a certain amount of code, related to the database initialization and handling. A common attribute is also

```java
import java.util.List;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Query;
import org.hibernate.Transaction;

public class HospitalDAO {
  private SessionFactory sessionFactory;

  @Override
  public Hospital registerHospital(
      String name, String address,
      String phoneNumber)
      throws DAOException {
    Session session = sessionFactory.openSession();
    Transaction tx = null;
    Hospital hospital;
    try {
      tx = session.beginTransaction();

      hospital = new Hospital(name);
      hospital.setAddress(address);
      hospital.setPhoneNumber(phoneNumber);

      if (Validator.isValidHospital(hospital)) {
        Registry.addPlace(hospital.getName());
      }

      session.save(hospital);

      tx.commit();
    } catch (Exception e) {
      if (tx!=null) tx.rollback();
      throw new DAOException(e.getMessage());
    } finally {
      session.close();
    }
    return hospital;
  }

  public SessionFactory getSessionFactory() {
    return sessionFactory;
  }

  public void setSessionFactory(
      SessionFactory sessionFactory) {
    this.sessionFactory = sessionFactory;
  }
}
```

```java
import java.util.List;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Query;
import org.hibernate.Transaction;

public class AppointmentDAO {
  private SessionFactory sessionFactory;

  @Override
  public List<Appointment> getAppointments(
      Long idPatient)
      throws DAOException {
    Session session = sessionFactory.openSession();
    Transaction tx = null;
    List<Appointment> appointments;
    try {
      tx = session.beginTransaction();

      Patient patient = (Patient)
        session.get(Patient.class, idPatient);
      appointments = patient.getAppointments();

      Logger.log("Appointments of patient "
    + patient.getName() + " have been recovered
        ");

      tx.commit();
    } catch (Exception e) {
      if (tx != null) tx.rollback();
      throw new DAOException(e.getMessage());
    } finally {
      session.close();
    }
    return appointments;
  }

  public SessionFactory getSessionFactory() {
    return sessionFactory;
  }

  public void setSessionFactory(
      SessionFactory sessionFactory) {
    this.sessionFactory = sessionFactory;
  }
}
```

Figure 3.7: `HospitalDAO` and `AppointmentDAO` comparison: original classes. The identical code has been highlighted.

declared to work with the database, and so the `get` and `set` methods are equal in both classes. The important methods, however, have different functions, since one of them stores content and the other queries it. The lines of code relevant to the method's behavior are therefore different. In conclusion, a common generation is possible but not entirely suitable for these classes.

`HospitalDAO` and `PatientDAO`

The classes involved in the second comparison are depicted in Figure 3.8. This time, the class `HospitalDAO` is compared with another DAO class: `PatientDAO`. `PatientDAO` defines the same attributes and `get` and `set` methods as `HospitalDAO`, and a method called `registerPatient` to access the database. Within this method, a new `Patient` object is created from the data received in the parameters, and stored in the database after some registry-related actions.

```java
import java.util.List;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Query;
import org.hibernate.Transaction;

public class HospitalDAO {
  private SessionFactory sessionFactory;

  @Override
  public Hospital registerHospital(
      String name, String address,
      String phoneNumber)
      throws DAOException {
    Session session = sessionFactory.openSession();
    Transaction tx = null;
    Hospital hospital;
    try {
      tx = session.beginTransaction();

      hospital = new Hospital(name);
      hospital.setAddress(address);
      hospital.setPhoneNumber(phoneNumber);

      if (Validator.isValidHospital(hospital)) {
        Registry.addPlace(hospital.getName());
      }

      session.save(hospital);

      tx.commit();
    } catch (Exception e) {
      if (tx!=null) tx.rollback();
      throw new DAOException(e.getMessage());
    } finally {
      session.close();
    }
    return hospital;
  }

  public SessionFactory getSessionFactory() {
    return sessionFactory;
  }

  public void setSessionFactory(
      SessionFactory sessionFactory) {
    this.sessionFactory = sessionFactory;
  }
}
```

```java
import java.util.List;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Query;
import org.hibernate.Transaction;

public class PatientDAO {
  private SessionFactory sessionFactory;

  @Override
  public Patient registerPatient(
      String name, String user,
      String pass, String phone,
      String address, String email,
      long idHospital)
      throws DAOException {
    Session session = sessionFactory.openSession();
    Transaction tx = null;
    Patient patient;
    try {
      tx = session.beginTransaction();

      Hospital hospital = (Hospital) session
          .get(Hospital.class, idHospital);
      patient = new Patient(name, user, pass);
      patient.setAddress(address);
      patient.setPhoneNumber(phone);
      patient.setEmail(email);
      patient.setHospital(hospital);

      if (Validator.isValidPatient(patient)) {
        Registry.addPerson(patient.getName());
      }

      session.save(patient);

      tx.commit();
    } catch (Exception e) {
      if (tx != null) tx.rollback();
      throw new DAOException(e.getMessage());
    } finally {
      session.close();
    }
    return patient;
  }

  public SessionFactory getSessionFactory() {
    return sessionFactory;
  }

  public void setSessionFactory(
      SessionFactory sessionFactory) {
    this.sessionFactory = sessionFactory;
  }
}
```

Figure 3.8: `HospitalDAO` and `PatientDAO` comparison: original classes. The identical code has been highlighted.

The creation and initialization of the object to be stored is in this case a little more complex, but we can see that both classes perform a really similar function. Only the

type of the object being handled changes, and some of the lines of code vary according to this difference, but the purpose of the methods is essentially the same and the classes are therefore clearly suitable for a common generation.

Along with the explanation of each step, we will apply the described actions to these examples and analyze the transformations made, the resulting similarity and the conclusions that can be derived from all of it.

Step 1. Language-specific basic pre-processing

This first step is meant to be the basic one, where risky assumptions are kept minimal and the code is just simplified and normalized as much as possible, trying not to lose any of its important semantics in the process. Five different pre-processing actions are performed. Some of them are specific to class declarations, others differ depending on the programming language, but most of them are applicable to any code unit:

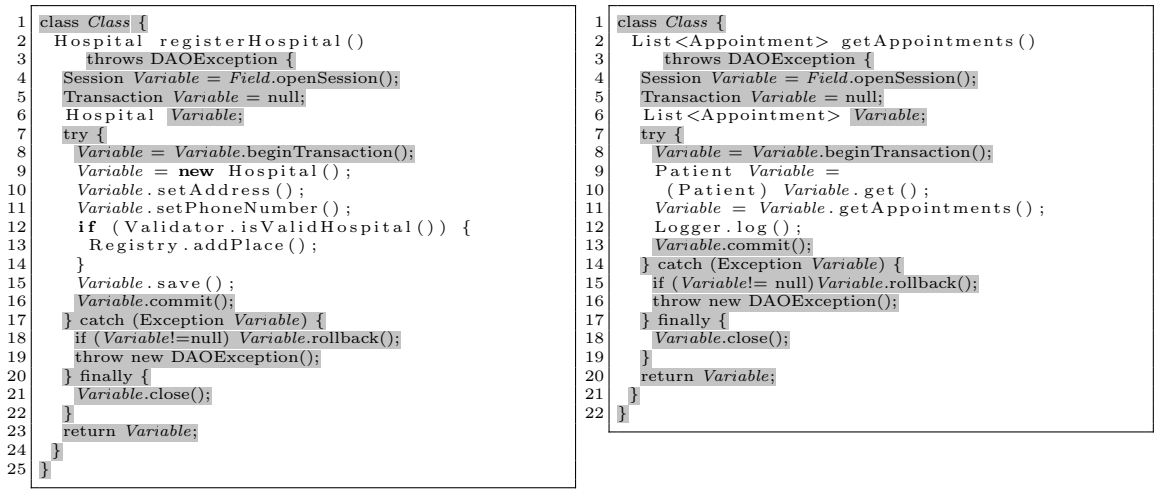
- Remove comments and layout related elements such as blank lines and whitespace.
- Remove unnecessary keywords and modifiers (i.e. import statements or visibility declarations).
- Normalize variables, type declarations, fields and enumerates, replacing their names with an identifier which must be unique for each type. Keep the names of all the method declarations and invocations.
- Remove field declarations and simple `get` and `set` methods.

  These methods are not usually relevant in the process but, if left, are often included in the results.
- Remove the parameters from method declarations, and the arguments from method and constructor invocations.

  We consider that in most situations the relevant information is the method that is being invoked or declared and not the parameters themselves. Therefore, eliminating this from the process helps to simplify it without compromising the reliability of the results.

Figure 3.9 shows the resulting code if we apply the pre-processing actions defined in this step to the classes `HospitalDAO` and `AppointmentDAO`, whose original code was presented in Figure 3.7 on page 26. The layout-related pre-processing steps are not appreciable, since we have decided to keep a basic format for illustrative purposes. The consequences of the other actions are, on the contrary, fairly perceptible. The removal of the `get` and `set` methods and the field declarations has reduced the total size of the code units considerably. In the same manner, some keywords have been eliminated, like the `Override` tag, the visibility-related declarations and the `import` statements. This action is the only one that is technology-dependent and, while similar modifications can be done when dealing with other programming languages, these are specific to Java code.

The names of all the classes, variables, fields and arguments have been normalized, replacing them with an identifier. This identifier is unique for every type of element, so that no differences are made, for example, between all the variables or between all the fields after this transformation. This normalization is not particularly useful in this particular example, though, since the equivalent variables that exist in the two code units



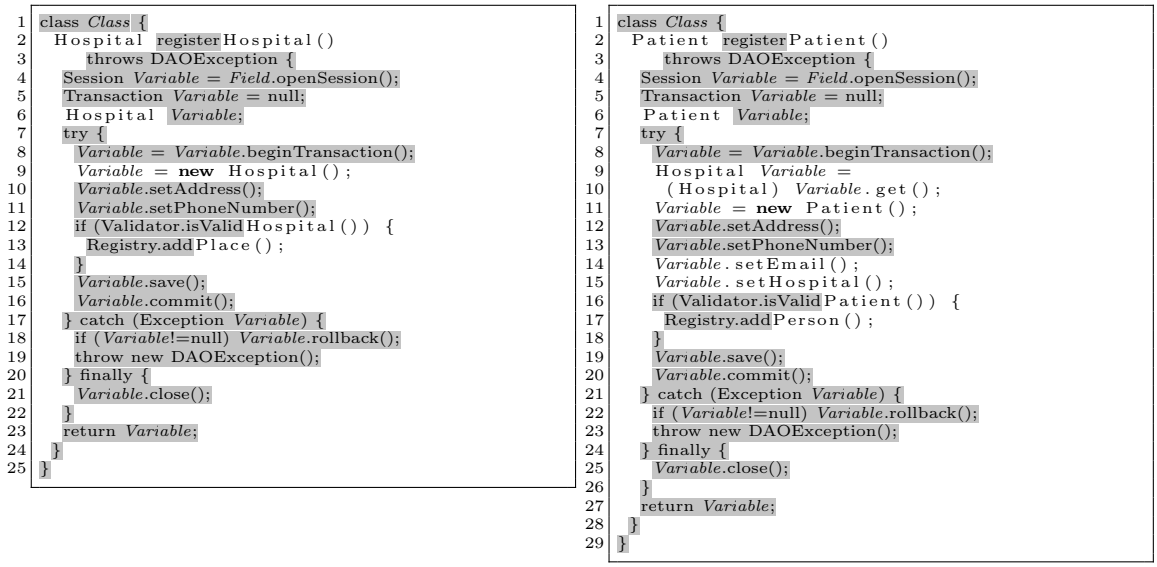
```
class Class {
  Hospital registerHospital()
      throws DAOException {
    Session Variable = Field.openSession();
    Transaction Variable = null;
    Hospital Variable;
    try {
      Variable = Variable.beginTransaction();
      Variable = new Hospital();
      Variable.setAddress();
      Variable.setPhoneNumber();
      if (Validator.isValidHospital()) {
        Registry.addPlace();
      }
      Variable.save();
      Variable.commit();
    } catch (Exception Variable) {
      if (Variable!=null) Variable.rollback();
      throw new DAOException();
    } finally {
      Variable.close();
    }
    return Variable;
  }
}
```

```
class Class {
  List<Appointment> getAppointments()
      throws DAOException {
    Session Variable = Field.openSession();
    Transaction Variable = null;
    List<Appointment> Variable;
    try {
      Variable = Variable.beginTransaction();
      Patient Variable =
        (Patient) Variable.get();
      Variable = Variable.getAppointments();
      Logger.log();
      Variable.commit();
    } catch (Exception Variable) {
      if (Variable!= null) Variable.rollback();
      throw new DAOException();
    } finally {
      Variable.close();
    }
    return Variable;
  }
}
```

Figure 3.9: `HospitalDAO` and `AppointmentDAO` comparison: classes after step 1. The identical code has been highlighted.

already had originally the same names. Finally, the parameters and arguments have been removed from all the method declarations and invocations. The result of this step is a significant simplification of the code. Eventually, the total percentage of identical code has not changed much, but the result is now much more significant since only the important part of the code is being compared.

The resulting code after applying the same actions to our second example is depicted in Figure 3.10. Evidently, the transformation yields the same result for the `HospitalDAO` class. With regard to the `PatientDAO` class, the result is very similar to what we have already seen. The code gets greatly simplified and only the main method remains, with the names of all the variables, classes, fields and arguments normalized.



```
class Class {
  Hospital registerHospital()
      throws DAOException {
    Session Variable = Field.openSession();
    Transaction Variable = null;
    Hospital Variable;
    try {
      Variable = Variable.beginTransaction();
      Variable = new Hospital();
      Variable.setAddress();
      Variable.setPhoneNumber();
      if (Validator.isValidHospital()) {
        Registry.addPlace();
      }
      Variable.save();
      Variable.commit();
    } catch (Exception Variable) {
      if (Variable!=null) Variable.rollback();
      throw new DAOException();
    } finally {
      Variable.close();
    }
    return Variable;
  }
}
```

```
class Class {
  Patient registerPatient()
      throws DAOException {
    Session Variable = Field.openSession();
    Transaction Variable = null;
    Patient Variable;
    try {
      Variable = Variable.beginTransaction();
      Hospital Variable =
        (Hospital) Variable.get();
      Variable = new Patient();
      Variable.setAddress();
      Variable.setPhoneNumber();
      Variable.setEmail();
      Variable.setHospital();
      if (Validator.isValidPatient()) {
        Registry.addPerson();
      }
      Variable.save();
      Variable.commit();
    } catch (Exception Variable) {
      if (Variable!=null) Variable.rollback();
      throw new DAOException();
    } finally {
      Variable.close();
    }
    return Variable;
  }
}
```

Figure 3.10: `HospitalDAO` and `PatientDAO` comparison: classes after step 1. The identical code has been highlighted.

In this case, the removal of unimportant code and the normalization of the names do not have a great influence on the final result either, for the same reasons than in the previous case. Only a couple of additional identical lines are found now, once the name of the variable that is being created is not relevant any more. The `Hospital` and `Patient` classes share some attributes with identical name, and the use of their `set` methods is now identified as equal in both code units. This slightly increases the similarity, and it is the first sign that indicates that these two blocks of code might be interesting generation candidates. However, the results are not very clarifying. The similarity measure is now more significant, but its value does not allow us yet to comprehend the real nature of the existing differences, and therefore find out their actual interest as generation candidates.

Step 2. Generation-oriented basic preprocessing

This step represents the first approach to try to customize the clone detection process to better adapt to our specific goal: the detection of generation candidates. For this step, we need for the code units being compared to be related to a domain class. The defined actions make use of the domain class that is related to each of the two code units that are being compared. The goal is to eliminate the differences in both pieces of code that are related to the domain classes, that is, that originate from the variations present in both domain classes. It is a very simple step that mainly aims at normalizing class types and methods that are named in a systematic way using the domain class' name (or any of its attributes' names). Although not specially sophisticated, these actions can provide decent results increasing the similarity measure between both code units. As part of this step, the following actions are performed on both input code units:

- Replace every occurrence of the domain class' name in the code with a unique identifier.
- Replace every occurrence of the domain class attributes' names in the code with a unique identifier.

For our first example, the classes `HospitalDAO` and `AppointmentDAO` are associated with two domain classes: `Hospital` and `Appointment`. The transformation actions are applied and the resulting code after this step is shown in Figure 3.11.

The domain class' name is replaced, in both cases, in the name of the method and its return type, but this replacement has not made the corresponding lines equal in the code units. The similarity has only very slightly increased, since the domain class is present but in different forms: the return type in `AppointmentDAO` is a collection and not a single object and the names of the methods are not formed in the same way.

As for our second example, the result of these transformations is depicted in Figure 3.12. The same replacements are made in `HospitalDAO`. Apart from the ones previously mentioned, additional substitutions were made during the creation and initialization of the domain class variable that is stored in the database. These replacements become important in this particular comparison, since similar ones are performed on the `PatientDAO` class, where a domain class variable is also created and initialized. This results in a few lines of code becoming equal after this step. In addition to that, the method's name and return type of both code units also match after this step, contrary to what happened in the previous example, and so does the name of the method being invoked on the `Validator` class.

```
class Class {
  ModelClass register ModelClass()
      throws DAOException {
    Session Variable = Field.openSession();
    Transaction Variable = null;
    ModelClass Variable;
    try {
      Variable = Variable.beginTransaction();
      Variable = new ModelClass();
      Variable.setModelAttribute();
      Variable.setModelAttribute();
      if (Validator.isValidModelClass()) {
        Registry.addPlace();
      }
      Variable.save();
      Variable.commit();
    } catch (Exception Variable) {
      if (Variable!=null) Variable.rollback();
      throw new DAOException();
    } finally {
      Variable.close();
    }
    return Variable;
  }
}
```

```
class Class {
  List<ModelClass> getModelClasss()
      throws DAOException {
    Session Variable = Field.openSession();
    Transaction Variable = null;
    List<ModelClass> Variable;
    try {
      Variable = Variable.beginTransaction();
      Patient Variable =
        (Patient) Variable.get();
      Variable = Variable.getModelClasss();
      Logger.log();
      Variable.commit();
    } catch (Exception Variable) {
      if (Variable!=null) Variable.rollback();
      throw new DAOException();
    } finally {
      Variable.close();
    }
    return Variable;
  }
}
```

Figure 3.11: `HospitalDAO` and `AppointmentDAO` comparison: classes after step 2. The identical code has been highlighted.

```
class Class {
  ModelClass registerModelClass()
      throws DAOException {
    Session Variable = Field.openSession();
    Transaction Variable = null;
    ModelClass Variable;
    try {
      Variable = Variable.beginTransaction();
      Variable = new ModelClass();
      Variable.setModelAttribute();
      Variable.setModelAttribute();
      if (Validator.isValidModelClass()) {
        Registry.addPlace();
      }
      Variable.save();
      Variable.commit();
    } catch (Exception Variable) {
      if (Variable!=null) Variable.rollback();
      throw new DAOException();
    } finally {
      Variable.close();
    }
    return Variable;
  }
}
```

```
class Class {
  ModelClass registerModelClass()
      throws DAOException {
    Session Variable = Field.openSession();
    Transaction Variable = null;
    ModelClass Variable;
    try {
      Variable = Variable.beginTransaction();
      Hospital Variable =
        (Hospital) Variable.get();
      Variable = new ModelClass();
      Variable.setModelAttribute();
      Variable.setModelAttribute();
      Variable.setModelAttribute();
      Variable.setModelAttribute();
      if (Validator.isValidModelClass()) {
        Registry.addPerson();
      }
      Variable.save();
      Variable.commit();
    } catch (Exception Variable) {
      if (Variable!=null) Variable.rollback();
      throw new DAOException();
    } finally {
      Variable.close();
    }
    return Variable;
  }
}
```

Figure 3.12: `HospitalDAO` and `PatientDAO` comparison: classes after step 2. The identical code has been highlighted.

In the end, we find that the similarity between these two code units has significantly increased after the application of the actions defined in this second, generation-specific step. Only a few lines remain different, and the `HospitalDAO` class is almost completely replicated in `PatientDAO`. Moreover, the meaning of the results obtained from this step can go beyond the actual measure eventually calculated. The fact that this is a step specifically oriented to the identification of generation candidates can contribute to better interpret the results obtained. If the similarity measure between the two code units has increased after this step, compared to the measure obtained in the first one, then it means that the units had some differences that originated merely from the associated domain classes. This situation is particularly interesting for generation candidates, since usually those are the kind of differences that can be integrated into the generator logic. If, on the contrary, the similarity measure remains the same after these transformations, then it is likely that the existing differences between the code units have nothing to do with the domain classes associated to them, and they may be more difficult to integrate into the logic of a common generation.

In consequence, it is usually a good sign when there is a significant increase in the similarity measure after this step. In this sense, the results obtained in both examples point out a difference in the variations that remain between the code units after the first pre-processing step. The transformation actions do not have a big influence on the similarity between the code units in the first example, but increase this measure in the second one. This indicates that the differences in the latter case can be more easily handled and integrated into a generation process. Therefore, `HospitalDAO` and `PatientDAO` appear to be better generation candidates.

Step 3. Generation-oriented advanced pre-processing

This step adds some further actions specifically related to the identification of generation candidates. Only this time, the assumptions made are more speculative than in the previous step. The results can improve, but there is also the possibility that the changes made are too invasive and result in the equality of two portions of code that are not really equivalent. This is the reason why such actions are grouped together in a new step, so that the similarity measure corresponding to this step can be individually considered. For this step, we need once again the domain classes associated to both code units being compared. Using this domain class, the following actions are performed on each input code unit:

- Replace all method calls on objects of the domain class with a unique identifier.

  In this way, all the uses of the domain class are normalized in both code units, and some differences related to the domain classes are removed.

- Remove consecutive duplicated lines of code.

  In case there are different consecutive uses of the domain class in the code, all of them are replaced with a single identifier. It usually happens, for example, that a certain action has to be done for some of the fields of the domain class. Depending on how many fields the domain class has, there is a longer or shorter series of method calls. These differences are not really significant, and the procedure in this step aims at solving this problem by eliminating them.

  Duplication is checked at a statement level. Ideally, only the statements that are duplicated as a consequence of the previous action should be removed. Nevertheless, as a more general approach every duplicated statement found in the code can be

removed during this step. However, this may have additional and possibly undesired consequences on the results.

Figure 3.13 shows the resulting code after applying these steps to the code units in the first example. As we can see, in `HospitalDAO` the invocations of the domain class' `set` methods have all been replaced with a unique identifier after applying the two transformation actions. During the first one, they are substituted by a unique identifier. In this case, this is not relevant since they were all already equal after the previous steps. Being all consecutive identical lines, they are all subsequently replaced again by a single identifier. This is however the only replacement done in both code units, since in `AppointmentDAO` the domain class variable is only initialized and returned, but not modified or used in any other way.

```
class Class {
  ModelClass register ModelClass()
      throws DAOException {
    Session Variable = Field.openSession();
    Transaction Variable = null;
    ModelClass Variable;
    try {
      Variable = Variable.beginTransaction();
      Variable = new ModelClass();
      ModelClassUse;
      if (Validator.isValid ModelClass()) {
        Registry.addPlace();
      }
      Variable.save();
      Variable.commit();
    } catch (Exception Variable) {
      if (Variable!=null) Variable.rollback();
      throw new DAOException();
    } finally {
      Variable.close();
    }
    return Variable;
  }
}
```

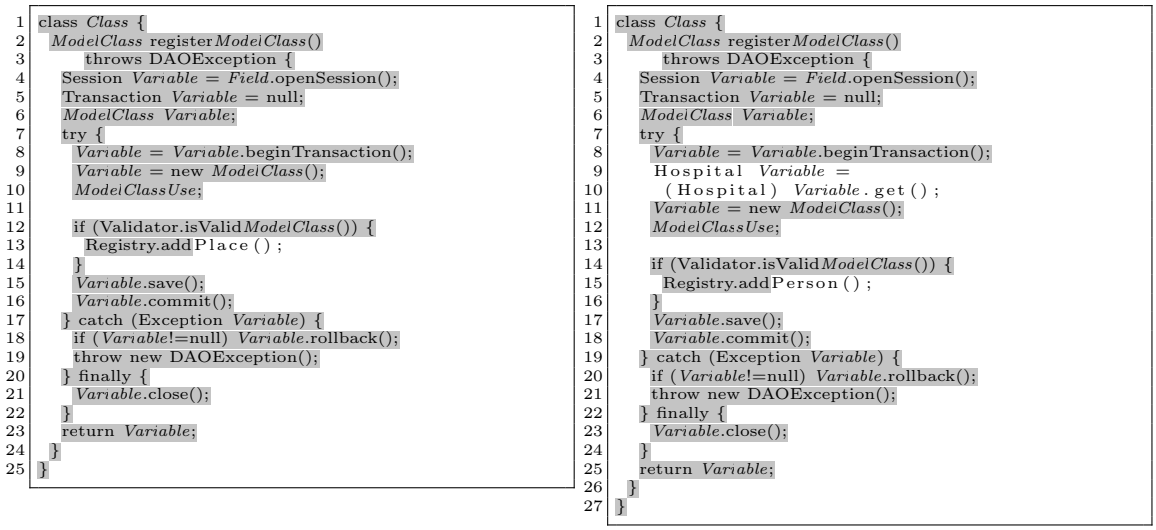
```
class Class {
  List<ModelClass> get ModelClasss()
      throws DAOException {
    Session Variable = Field.openSession();
    Transaction Variable = null;
    List<ModelClass> Variable;
    try {
      Variable = Variable.beginTransaction();
      Patient Variable =
        (Patient) Variable.get();
      Variable = Variable.get ModelClasss();
      Logger.log();
      Variable.commit();
    } catch (Exception Variable) {
      if (Variable!=null) Variable.rollback();
      throw new DAOException();
    } finally {
      Variable.close();
    }
    return Variable;
  }
}
```

Figure 3.13: `HospitalDAO` and `AppointmentDAO` comparison: classes after step 3. The identical code has been highlighted.

The similarity measure remains accordingly with a similar value. The number of identical lines is, indeed, the same, although the total size of `HospitalDAO` has been reduced, and so a bigger percentage of its code is now identical to `AppointmentDAO`. However, the similarity increase is negligible.

Figure 3.14 shows the resulting code after applying the transformation actions to the code units of our second example. This time, similar substitutions are made in the `PatientDAO` class, where all the `set` methods that were invoked on the domain class variable are replaced by a single identifier. Both blocks of methods are therefore compressed and made equal. `HospitalDAO` is still almost completely replicated in `PatientDAO`, and the same differences remain in the code. However, `PatientDAO` is now almost completely replicated in `HospitalDAO` as well, and so it can be stated that the similarity measure between both classes has increased.

This increase is not as significant as the one reached in the previous step, but can help to support the conclusions that we derived from it. Once again, the application of a generation-specific pre-processing step has had almost no consequences on the first example, and slightly increased the similarity measure in the second. Taken together, the results of these two steps show that the classes in the second example appear to be better generation candidates than the ones in the first example.



```
class Class {
  ModelClass registerModelClass()
      throws DAOException {
    Session Variable = Field.openSession();
    Transaction Variable = null;
    ModelClass Variable;
    try {
      Variable = Variable.beginTransaction();
      Variable = new ModelClass();
      ModelClassUse;

      if (Validator.isValidModelClass()) {
        Registry.addPlace();
      }
      Variable.save();
      Variable.commit();
    } catch (Exception Variable) {
      if (Variable!=null) Variable.rollback();
      throw new DAOException();
    } finally {
      Variable.close();
    }
    return Variable;
  }
}
```

```
class Class {
  ModelClass registerModelClass()
      throws DAOException {
    Session Variable = Field.openSession();
    Transaction Variable = null;
    ModelClass Variable;
    try {
      Variable = Variable.beginTransaction();
      Hospital Variable =
        (Hospital) Variable.get();
      Variable = new ModelClass();
      ModelClassUse;

      if (Validator.isValidModelClass()) {
        Registry.addPerson();
      }
      Variable.save();
      Variable.commit();
    } catch (Exception Variable) {
      if (Variable!=null) Variable.rollback();
      throw new DAOException();
    } finally {
      Variable.close();
    }
    return Variable;
  }
}
```

Figure 3.14: `HospitalDAO` and `PatientDAO` comparison: classes after step 3. The identical code has been highlighted.

Step 4. Language-specific advanced pre-processing

This step contains more aggressive normalization actions, which might end up matching pieces of code with a rather different behavior. However, sometimes the actions performed before are not enough to discard all the irrelevant differences in the code. Some may remain that, although not being really distinctive, prevent the code blocks from being reported as similar. That is why in some occasions it can be convenient to perform some higher level substitutions that may help to identify additional similar lines, increasing the overall similarity measure between the two code units. As it was the case in the previous step, the assumptions made here are too broad and can lead to inaccurate results. Therefore, we strongly recommend a careful interpretation of the measures reported by this step. The following actions are performed on both input code units as part of the step:

- Normalize the variable types, replacing them with a unique identifier.

  It can be discussed how important variable types really are when comparing two pieces of code. In our opinion, they are important enough to keep them intact until this last step, since they are the only way a variable can really keep its identity once the variable names have been normalized. By just knowing a variable is being used without keeping its name or type, important information relevant to the way the code works is lost. Some replacements involving the domain class in previous steps would also lose part of their potential.

- Replace variable declarations and initializations with a unique identifier.

- Replace method invocations with a unique identifier.

  The normalization of method invocations greatly simplifies the code, but with an evident risk. It is clear that two method invocations can do completely different things and not be related whatsoever. Nevertheless, if after these replacements we find large blocks of code where methods are invoked in a similar manner, it could be a good indicator to detect code that, despite considerable syntactic differences, may be still similar enough to be generated together.

Figure 3.15 illustrates the first example. Since many replacements have already been made at this point, there is not a great amount of code that is affected. Still, some variable types and method invocations are identified, as well as a couple of initializations. However, the changes made in this first example do not have a great influence on the final result. Only the method's return type and one of the method invocations become equal after the replacements. In particular, the `save` method used to store the variable in the database in `HospitalDAO` and the `log` method invoked on the `Logger` in `AppointmentDAO` both match after being substituted. It is evident how these methods are not related at all, and although this action has slightly increased the similarity, it was only because a wrong assumption was made. The consequences are not very important in this case, but this should be taken as an example of how the actions performed here have to be considered carefully, and their results manually analyzed.

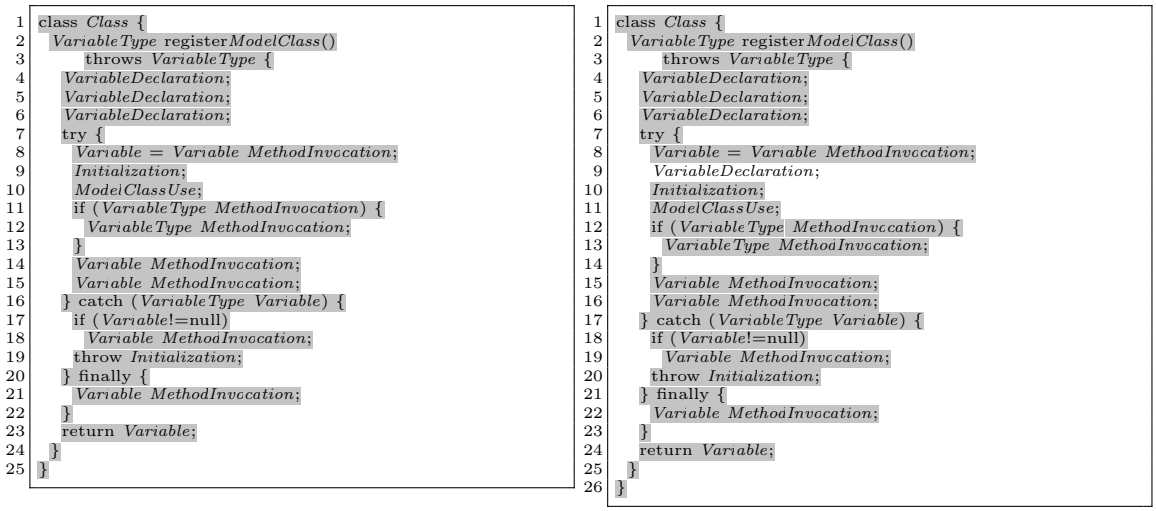
```
1  class Class {
2    VariableType register ModelClass()
3        throws VariableType {
4      VariableDeclaration;
5      VariableDeclaration;
6      VariableDeclaration;
7      try {
8        Variable = Variable MethodInvocation;
9        Initialization;
10       ModelClassUse;
11       if (VariableType MethodInvocation)
12       {
13         VariableType MethodInvocation;
14       }
15       Variable MethodInvccation;
16       Variable MethodInvccation;
17     } catch (VariableType Variable) {
18       if (Variable!=null)
19         Variable MethodInvccation;
20       throw Initialization;
21     } finally {
22       Variable MethodInvccation;
23     }
24     return Variable;
25   }
26 }
```

```
1  class Class {
2    VariableType getModelClasss()
3        throws VariableType {
4      VariableDeclaration;
5      VariableDeclaration;
6      VariableDeclaration;
7      try {
8        Variable = Variable MethodInvccation;
9        VariableDeclaration;
10       Variable = Variable MethodInvocation;
11       VariableType MethodInvocation;
12       Variable MethodInvccation;
13     } catch (VariableType Variable) {
14       if (Variable!=null)
15         Variable MethodInvocation;
16       throw Initialization;
17     } finally {
18       Variable MethodInvccation;
19     }
20     return Variable;
21   }
22 }
```

Figure 3.15: `HospitalDAO` and `AppointmentDAO` comparison: classes after step 4. The identical code has been highlighted.

Figure 3.16 shows the code for the second example after applying the transformation actions defined in this final step. By just taking a quick look at it, it stands out the fact that almost the whole code units are now identical, and in particular `HospitalDAO` is completely included in `PatientDAO`, where only a single line remains different. The consequence of the transformations performed in this step is that the methods invoked on the `Registry` class have now been made equal. In this particular case, there is a real relation between the methods `addPlace` and `addPerson`, which have a similar function but deal with different types of entities. Therefore, the substitution has been useful this time, identifying additional lines of code that could be generated together. The similarity measure, however, does not change significantly, since it was already very high.

When dealing with bigger, more complex code units, the actions introduced in this step can produce greater variations in the results. The similarity measure calculated here can be useful as an additional factor, but it is only significant once the results have been carefully analyzed to see if the new matching lines are indeed similar lines, like in the second example, or completely unrelated lines, like in the first.



```
class Class {
  VariableType registerModelClass()
      throws VariableType {
    VariableDeclaration;
    VariableDeclaration;
    VariableDeclaration;
    try {
      Variable = Variable MethodInvocation;
      Initialization;
      ModelClassUse;
      if (VariableType MethodInvocation) {
        VariableType MethodInvocation;
      }
      Variable MethodInvocation;
      Variable MethodInvocation;
    } catch (VariableType Variable) {
      if (Variable!=null)
        Variable MethodInvocation;
      throw Initialization;
    } finally {
      Variable MethodInvocation;
    }
    return Variable;
  }
}
```

```
class Class {
  VariableType registerModelClass()
      throws VariableType {
    VariableDeclaration;
    VariableDeclaration;
    VariableDeclaration;
    try {
      Variable = Variable MethodInvocation;
      VariableDeclaration;
      Initialization;
      ModelClassUse;
      if (VariableType MethodInvocation) {
        VariableType MethodInvocation;
      }
      Variable MethodInvocation;
      Variable MethodInvocation;
    } catch (VariableType Variable) {
      if (Variable!=null)
        Variable MethodInvocation;
      throw Initialization;
    } finally {
      Variable MethodInvocation;
    }
    return Variable;
  }
}
```

Figure 3.16: `HospitalDAO` and `PatientDAO` comparison: classes after step 4. The identical code has been highlighted.

Conclusions

The four pre-processing steps defined represent different ways of trying to reduce the unimportant differences between two code units being compared, with an increasing level of risk being taken in each set of actions. For each step, the code units are analyzed to obtain a similarity measure. Separating the comparison process in different steps allows us to implement more aggressive strategies without compromising the results obtained by a more basic, reliable previous step. In this way, the user obtains a final measure of the similarity between the two code units and, if necessary, can analyze the results in more detail to see where the final measure originates from.

Since the steps are easily distinguishable by their compounding actions, additional interpretations can be derived from the individual measures obtained in each step. A significant increase in the similarity after the second step, for example, is usually a good sign, since that means that some of the differences that exist between the code units are related to the domain classes and therefore their integration into a generative architecture may be easier. A great increase in the similarity obtained in the fourth step, on the contrary, can be promising but should be taken much more carefully, since some of the assumptions made can be wrong. In the end, having a multiple-step process gives us more flexibility to implement different actions, as we are not restrained by the idea of defining a single algorithm that should perform good in every situation. Additionally, we consider that the user greatly profits from this scheme too. The information he obtains is much more complete and dissected in a way that favors further, more detailed analysis. The presented approach defines four pre-processing steps, but the scheme can be extended to a n-step pre-processing structure, if further actions are defined which require the definition of a separate step.

We have seen how the four pre-processing steps have been applied to two different examples. For the first one, the two code units had similar code but their functions were not the same. After the first step, a high similarity was obtained, but the result hardly changed after the second and third steps, and only increased a little bit after the fourth step. For the second example, two code units with similar code and functions were compared.

The similarity measure was initially high, and increased specially after the second step, but also after the third and fourth step. The overall similarity between the code units of both examples is pretty high, and they could in consequence be both good generation candidates. However, the four-step approach helped us to better understand the situation and correctly identify the second pair of code units as a much more interesting generation candidate.

3.4.3 Application

Our custom clone detection method works with two given code units, which we have defined simply as a syntactically valid block of code. According to that, the method can compare from whole files to single lines of code, so the question of what to apply the method to arises. Here, we analyze different situations where the use of our method can be useful.

Between classes

The most straight-forward application of our method is to calculate the similarity between two classes. This is the case shown in the example that we used in order to explain the pre-processing steps forming the method, where three different classes were compared pair-wise. The domain classes related to each compared class need to be provided. A similarity result is obtained after applying the general and generation-specific pre-processing steps to the code of the classes.

A pair-wise application of the method can be useful in some situations, where the user has already selected or identified some candidates and wants to check the similarity between two specific classes. However, it is often the case that the user knows no particular groups of classes worth comparing, but instead he intends for the tool to report such interesting classes to him. In this case, the method could be applied to not just a pair of classes but to multiple classes, such as all the classes in the system.

When applying the method to more than two classes, a pairing process needs to be performed, so that all the classes involved get compared to each other. A set of similarity results is reported, with the similarity measures between each possible pair of classes in the group. These results can be sorted and filtered, so that only the pairs of classes whose similarity exceeds some threshold value are reported to the user.

However, when dealing with big systems, the number of comparisons can grow exponentially. In this case, the overall clone detection process can be too costly in terms of time and/or resources. In some occasions, this burden is acceptable, since it is not a process intended to be executed on a daily basis. Nevertheless, when comparing every possible pair of classes in a system, the great majority of those comparisons are useless and not necessary since the classes involved have no relation whatsoever. In order to make the process more efficient and get rid of many unnecessary comparisons, it can be convenient to first select a subset of the classes of the system. The clone detection process is then performed only on the selected classes.

One interesting approach is to make use of the results of the name similarity feature (see Section 3.2). The analysis of the names of the files in the system helped us to obtain groups of related artifacts, where one or more of them are derived from other artifacts

in a similar manner. We can perform the custom clone detection process only on these groups. With this approach, the derived classes are compared to each other pair-wise. The classes that they are derived from (in terms of name similarity) act like their domain classes during the clone detection process.

Within a class

Although the comparison of classes is the most direct and promising application of the custom clone detection method, there can be additional ways of applying it. One of them is to calculate the similarity within a class, that is, between the methods that the class defines. The idea is to detect classes in which a major part of the code can be generated directly from the corresponding class in the domain model. The simplest example of this are the `get` and `set` methods. When we have a standard implementation of these methods, they can be fully generated by just knowing the name and type of the corresponding attributes, and so all the code can be produced from the class in the domain model. Of course, this is a very simple example, and in fact, as we have seen, our clone detection method does not even take into account simple `get` and `set` methods. However, there can be methods in a class that, although not being standard `get` and `set` methods, can nevertheless be easily generated, at least for the most part, from the class in the domain model.

```
1  public class Hospital {
2   private String name;
3   private List<Patient> patients = new LinkedList<Patient>();
4   private List<Doctor> doctors = new LinkedList<Doctor>;
5
6   public String getName() {
7    return name;
8   }
9
10  public void setName(String name) {
11   this.name = name;
12  }
13
14  public List<Patient> getPatients() {
15   return patients;
16  }
17
18  public List<Doctor> getDoctors() {
19   return doctors;
20  }
21
22  public void addPatient(Patient patient) {
23   patients.add(patient);
24   patient.setHospital(this);
25  }
26
27  public void removePatient(Patient patient) {
28   if (patients.contains(patient)) {
29    patients.remove(patient);
30    patient.setHospital(null);
31   }
32  }
```

```
33
34  public void addDoctor(Doctor doctor) {
35    doctors.add(doctor);
36    doctor.setHospital(this);
37  }
38
39  public void removeDoctor(Doctor doctor) {
40    if (doctors.contains(doctor)) {
41      doctors.remove(doctor);
42      doctor.setHospital(null);
43    }
44  }
45 }
```

Listing 3.3: Definition of the generatable class `Hospital`.

Listing 3.3 shows a simple class example that gives an idea of the kind of methods that can be identified. The `Hospital` class contains three fields, two of which are collections. For each field, a standard `get` method is defined. Additionally, a standard `set` method is defined for the `name` field, along with two methods for each collection to handle the addition and removal of objects. In practice, all the methods can be automatically generated. The generator would produce simple `get` methods for every field, simple `set` methods for non-collection fields and the custom `add` and `remove` methods that we see in the listing for collection fields.

While the simple `get` and `set` methods are not really relevant, the identification of the collection-specific methods can be useful for the user. In the example, the `add` and `remove` methods we see may or may not be specific to the `Hospital` class. Other classes in the system may treat collections in a different way, or they may also include similar `add` and `remove` methods. If this is the case, the generation of these methods could be applied globally in the system, easily integrating it in a generative architecture. Even if it is not the case, it is always interesting anyway to indicate this situation to the user.

For this reason we propose the application of the custom clone detection method within a class, to detect further possibilities of automatic generation in the system. In order to do this, the methods in the class need to be extracted and paired, so that a similarity measure is obtained for each possible pair of methods in the class. In this case, the domain class is the class containing the methods.

The four defined pre-processing steps are applied to the code of the methods. Figure 3.17 shows the resulting code after each step for the comparison of the two `remove` methods. The code of both methods is almost identical after the first step, and only their names remain different from that point on. Even the names could be made equal if the custom clone detection method included some heuristics when checking the names for the replacement actions, since the words *patient* and *doctor* come from the entities handled by the fields named *patients* and *doctors*, and only the distinction between the singular and plural forms of the words prevent these strings from being replaced. The high similarity shows the user that these two methods can be generated together. The `add` methods defined for the collections would be reported in the same way.

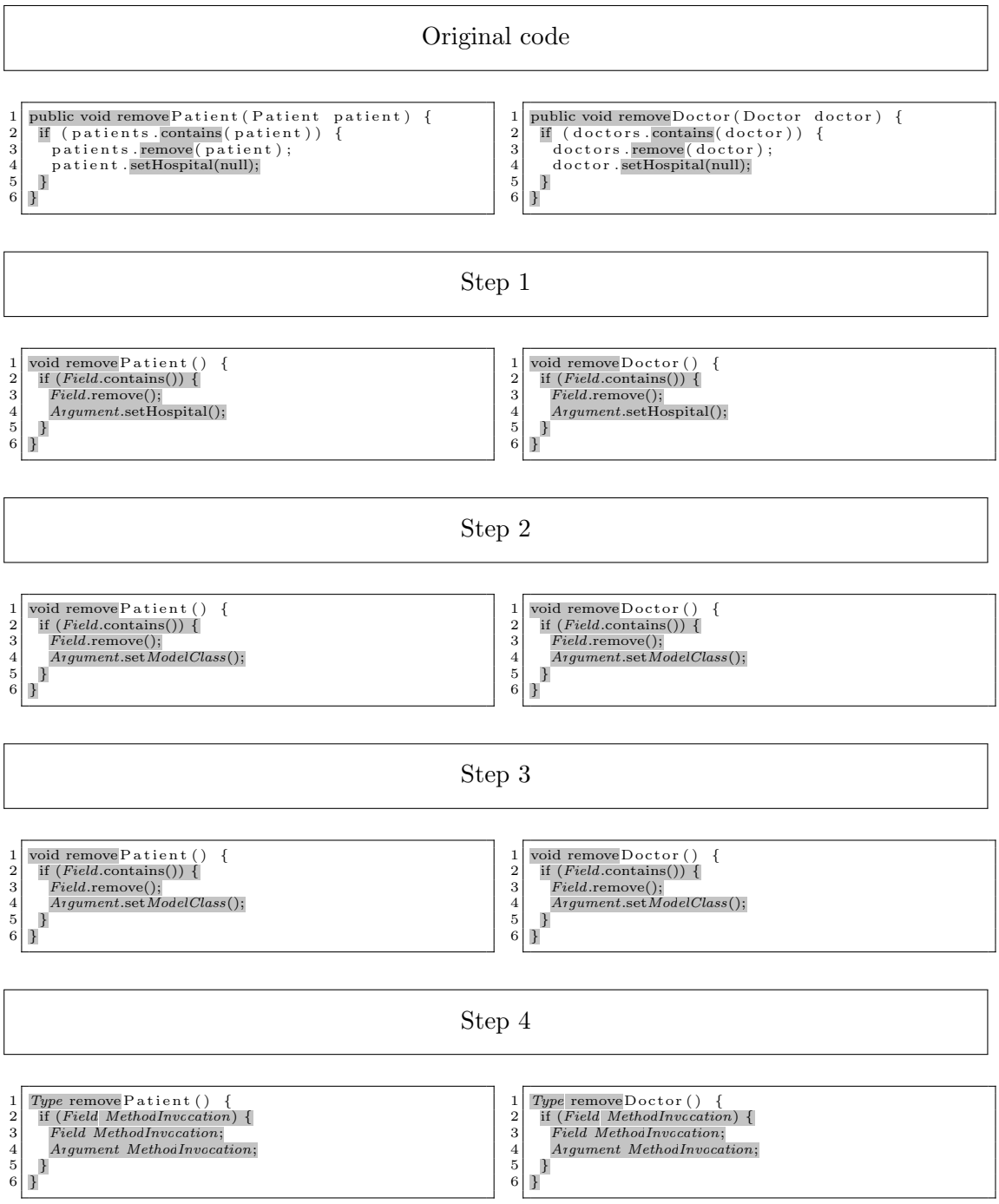


Figure 3.17: Application of the four pre-processing steps to two methods of the `Hospital` class.

Between files

Although some of the actions performed in certain pre-processing steps are related or even specific to programming concepts such as methods or classes, it was stated from the beginning that the method's description would remain technology-independent. As such, our clone detection method can be applied not only to one specific programming language (e.g. Java), nor even to a particular programming paradigm (object-oriented languages) but to any code unit. In consequence, virtually every file in a software system can be compared using the method. This can be useful to also detect similarities between configuration files, database scripts, etc.

As with the similarity between classes, the application of the method can be restricted to a certain subset of files in the system. Since the name similarity feature (see Section 3.2) can also be applied to any file, the structures of related files reported by such a feature can be also used as the input of our clone detection method. Because some of the actions in the pre-processing steps rely on certain programming concepts, it is possible that not all the steps are fully executed. The method, however, remains useful and the results, although not so significant, can still be of great interest to detect generation candidates in different parts of the system.

3.4.4 Optimizations

The method has been conceived so that its extension or adaptation is not only possible, but also easy and even encouraged. Actions can be added to any of the four defined steps, as well as new steps. In this way, an implementation of the method could just take some ideas and replace, remove or add others, modifying the steps and actions defined by us for the basic method but essentially adopting the same structure and thus benefiting from the approach. We comment here some of the modifications that we have identified which could be done to improve or enlarge the current four-step process.

A variation of this process could take code comments into account in one of the steps. Comments are now eliminated in the first pre-processing step and therefore ignored during the whole clone detection process. However, it can be interesting to include them in the comparison, since comments also contain information about what a block of code does. It can be of special interest to analyze Javadoc-like comments, which define with certain systematicity the purpose of each method or class along with the function performed by each parameter or field. Since this information comes in natural language, a different approach would be needed to compare the comments. If that is the case, a new, separate step can be defined to perform this comparison and obtain a similarity measure between the descriptions contained in each comment for both code units.

The already existing steps could also be modified. The first step has been defined as a basic step, where big changes in the code and aggressive actions are kept minimal. However, if results show that such a shallow pre-process yields irrelevant or inaccurate results, new actions could be added to the first step to include further substitutions and obtain a more abstract representation of the original code which may obtain better results. In this regard, step one could become more similar to step four. Additionally, the fourth step could also be extended with new actions that perform more aggressive tasks on the code. A wider range of syntactic elements could be considered than the current one, including new replacements for concepts like loops, conditions, exception handling, etc., depending

on the specific language and the available syntax. It can also be of great interest to develop new actions for the generation-specific pre-processing steps, since the measures obtained by such steps become specially relevant when analyzing the overall results. Unfortunately, no new ideas were found in the course of this thesis, but this is undoubtedly one of the most promising lines of future work.

One additional action that could be integrated, most suitably in the fourth step, is a more precise method substitution process. Instead of replacing every method invocation by a unique identifier, different custom method types can be defined, so that different identifiers are used in the substitutions. The resulting code would end up being less abstract, and therefore the results more accurate. Method types can typically include categories like database-related operations or GUI methods, but they could also be tailored to one specific system. An identification process would be necessary to correctly tie each method to its category. In order to do that, classes can be grouped together according to their functionality. Each method in a class would belong to a specific category depending on the way that class has been classified. A default category could be included so that not every class needs to be defined as part of a specific group.

On a more general note, other optimizations could be done on the described process. For example, heuristics can be included when performing the string replacements, such as the ones performed by the actions in step two. This way, the names would not need to be found exactly. Instead, minor differences could be tolerated or alternative forms of the words considered (e.g. plural or derived forms). Different heuristics were already introduced with the same purpose in Section 3.2.4 in the context of the name similarity feature.

Finally, there is currently the restriction that an artifact can only have one related domain class. In most cases, an artifact is indeed generated from one model class, and that is the reason why the process is conceived that way. However, this is not necessarily always the case, and two or more domain classes can exist related to an artifact. The name similarity process, for example, is able to identify groups of related files such that each file can be derived from multiple other files. Including the support to more than one domain class is therefore an optimization that would not be particularly hard to implement and that would help to adapt the custom clone detection process to such cases.

3.5 Clone evolution

3.5.1 Motivation

The convenience of code similarity in our context has already been introduced in previous sections. The use of clone detection tools out of the box was proposed as one of our features in Section 3.3, whereas a custom clone detection method was presented in Section 3.4. Clone evolution is defined as the field that studies clones across versions of the source code [PTK11]. For each clone group, a **clone genealogy** is constructed, which traces the lineage and change history of the clone group across different versions. The study of the evolution of clones in a system is used to better comprehend their origin and implications, and helps to understand how the clones should be best managed.

In our context, the additional information about clones provided by this feature is useful to better understand their generation possibilities. In the next section, we analyze different

ways in which clones' history can be interpreted from a generative point of view. A high similarity in past versions of the artifacts is possibly the most interesting scenario. Regardless of the specific degree of overall similarity, generation candidates usually share a structure that makes them suitable for a common generation. It may happen that such basic similarities are more evident in the initial versions of the code artifacts. As the implementation evolves, the code starts deviating more and more as developers introduce changes in the artifacts. New functionalities may be added and parts of the code are refactored. However, such changes may not be relevant for generative purposes. If the code artifacts still share a common structure, they may remain as good generation candidates even though the overall similarity has decreased. By analyzing clone evolution we can easily identify these kinds of situations.

This feature can be used as an extra factor in the analysis and filtering of the results reported by traditional clone detection tools. In this way, for example, clones that were also similar in the past can be reported as better generation candidates than those which were not. Additionally, clone evolution can also detect generation candidates that are not reported by the clone detection feature. This is the case when high levels of similarity are detected in old versions of artifacts which are currently not very similar. Depending on the nature of the introduced changes, such artifacts can still be interesting generation candidates, but they are only detected if we analyze their history.

3.5.2 Description

A systematic review of the state of the art in clone evolution is provided in [PTK11]. This research field arises from the initial work by Kim et al. [KN05, KSNM05], and different approaches have been proposed since then for the extraction of clone genealogies and their management [SRS11, NNP+08, GK10]. These tools can be reused in our context, as long as we redefine the way the results are interpreted.

A distinction is commonly made [KSNM05, SAZ+10] between **dead** and **alive** clone genealogies. This classification has important implications in our context:

- Dead genealogy.

  A genealogy is defined as dead if it does not contain a clone group in the final release. This means that code clones that were detected in previous versions stopped being clones at some point of the version history and are currently not considered clones any more.

  In our context, dead genealogies are particularly interesting. Changes were introduced in the artifacts, relevant enough so that they are not considered code clones any more. However, some similarities may still remain that make them suitable for a common generation. Therefore, a more detailed analysis of dead genealogies is recommended, and they are reported as generation candidates.

- Alive genealogy.

  A genealogy is defined as alive if it contains a clone group in the final release. This means that code clones have been detected in the final version. We can trace back its evolution across different versions up to the point where the clones were first detected.

In our context, the clones of an alive genealogy are also detected as generation candidates by other features, since they exist in the final release. However, the extracted genealogy provides additional information that can help in the analysis of the candidates. If a similarity measure is obtained for the clones for each version, then there are four possible ways in which the similarity may have evolved. Here, we a give our own interpretation of the implications that each possibility may have in our context. However, in the future a more detailed analysis should be done to empirically test these hypothesis.

- Same similarity. The clones may have been consistently changed or not changed at all. In any case, there is no useful information that we can extract from the analysis of the old versions.
- Decreasing similarity. Similarly to dead genealogies, when we detect a decreasing similarity it is interesting to analyze the initial versions of the code artifacts, and check whether the introduced changes really hinder a common generation or not.
- Increasing similarity. The situation where two code artifacts converge to be equal is less common [SL16] and can be interpreted in different manners. Anyway, whenever a pattern like this is clearly identified, the study of the genealogy is recommended in order to understand the real nature of the current similarity between the artifacts.
- Fluctuating similarity. When no particular pattern is detected in the variability of the similarity it is difficult to reach significant conclusions from the study of older versions. However, it can still be interesting to analyze the genealogy if anomalous situations are detected, such as too big or too frequent fluctuations.

[PTK11] identifies four basic approaches to construct clone genealogies. In the most common approach, clones are detected in all source code versions of interest, then corresponding clones in consecutive versions are retroactively linked using an heuristic. This is the most convenient technique for our purposes, since clone detection tools can be integrated out of the box to be applied to every version of the code. This allows us to not only reuse the tool selected in Section 5.2 for the clone detection feature, but also to integrate our custom clone detection method in the clone evolution process. Different clone evolution tools are available which can be used for such purposes. [SRS11] and [KN05], for example, both claim to be adaptable to any clone detection tool, having the first been tested with tools like CCFinder [KKI02], NiCad [RC08] or iClones [HG11], and the second with CCFinder.

3.5.3 Application

We have identified two situations where the analysis of clone evolution helps in the identification of generation candidates. The process can be applied to a whole software system. In this case, every genealogy is obtained and the clones that are part of a dead genealogy are reported as generation candidates. Another possibility is to apply the feature to two specific artifacts. For example, artifacts that were reported by another feature. In this case, the corresponding genealogy is obtained and reported to the user.

3.6 Design patterns

3.6.1 Motivation

In software engineering, a design pattern is a description of communicating objects that are customized to solve a general design problem in a particular context [GHJV95]. Design patterns make it easier to reuse successful designs and architectures. A design pattern is not defined in a way that allows a direct transformation into source code, but instead a general description is given for how to solve the problem, so that it can be used in many different situations. Typically, in the definition of a design pattern a set of artifacts is identified, along with the roles that they play and their interactions. This structure needs to be instantiated for every specific implementation of the design pattern.

Since the structure defined by a design pattern is always the same and it is known beforehand, in some occasions its generation can be automated by integrating such structure into a generative architecture. The amount of code that can be generated depends on the type of structure defined by the design pattern, but a certain degree of automation is possible for most design patterns. Consequently, the detection of design patterns arises as an interesting feature for the identification of generation candidates. The idea is that, once design patterns are identified in an existing system, the artifacts implementing them are proposed to the user as generation candidates. Many approaches exist that propose the automation of the detection process [TCSH06, SO06, KGH06]. These tools can be reused for the implementation of our feature, and will be considered in later sections.

The information about the identified patterns is used to detect structures of artifacts that can be generated. These structures can be further analyzed and, if found actually generatable, manually integrated into the generative architecture on a one-to-one basis. However, it is worth noting that, if we want the generation candidates reported by this feature to be truly useful, the generative architecture must be adapted to support the modelling and generation of design patterns. We need to be able to specify in the model where in the system a certain pattern is used, and which artifacts play the roles defined by the pattern. In the same manner, we need to provide the generation logic corresponding to each pattern. Only if this information is available, design patterns can be automatically generated in a systematic way and therefore the candidates reported by this feature become more relevant. As an additional advantage, this approach would foster the proper documentation of the design patterns used in a system.

3.6.2 Description

Classification

Design patterns gained popularity in computer science after [GHJV95] was published in 1994 by the so-called "Gang of Four" (Gamma et al.), in the following abbreviated as "GoF". In this book, a total of 23 design patterns are introduced, divided into three categories: creational, structural and behavioral patterns. Many other patterns have been proposed during the years [GHJV95, BMR+96, SSRB00, Fow02], addressing additional problems or focusing on specific domains. This diversity is an important factor that needs to be taken into account. Even if every design pattern can be generatable, the relevance of this generation and the amount of code involved depends on each specific pattern, and

the differences can be significant. For this reason, a classification process is necessary in order to analyze and measure the generation capabilities of each design pattern. Only in this way we can provide a more meaningful significance to the results.

[Mad13] is an interesting work along these lines. The design patterns defined by the GoF are analyzed from a generative perspective. In this work, the following information is provided for each pattern:

- A brief description of the design pattern.
- A figure depicting the structure of classes and their roles. A distinction between the classes that have been identified as generatable and those which are not is shown and explained.
- A use case where the design pattern is instantiated to show an example of how its generation works. A representation of the corresponding necessary input model is also included.

Although the analysis performed is not particularly thorough, the results obtained in this work are valuable and can be reused in our context. Table 3.1 shows a summary of the design patterns analyzed by [Mad13] and gives a measure on the amount of code that can be generated from each pattern, a characteristic that we have called **generatability**.

The generatability of a design pattern is a value that we calculate using the analysis performed in [Mad13] as a reference. A number of classes, interfaces and methods can be identified as generatable for every given design pattern. We assign a different value to each of these possibilities, and add up the values to obtain a final generatability measure for each pattern. The following values are assigned, according to what can be generated:

- Abstract class or interface, a value of 10.

  A complete abstract class or interface can be generated. We assign a fixed value regardless of the number of method declarations, which we do not consider important enough to be measured.

- A non-abstract class, a value of 8.

  A complete, non-abstract class can be generated. This class is specific to the design pattern and can be generated completely. The class contains in turn methods that are generated, and that have their own generatibility value. This is the reason why the value assigned in this case is lower than the one defined for interfaces and abstract classes, where the method declarations are considered as part of the code generated for the interface and not separately.

- Multiple non-abstract classes, a value of 24.

  A variable number of non-abstract classes can be generated. In some occasions, the structure of classes defined by the design pattern has a variable size. If classes can be generated from other classes in the model but the number of classes in the model is not known, then there is no way to measure in advance the amount of code that can be generated for the design pattern. In order to give a rough measure of the generatability of the design pattern in such situations, we assign a value that is three times the value corresponding to a single non-abstract class.

- A method, a value of 5.

  A method's declaration and implementation can be generated. Ideally, the number of lines in the method should be taken into account. In practice, however, the methods that can be generated from a design pattern often have a very simple implementation with no more than five lines of code. Consequently, a fixed value is assigned. As we consider that this situation is less interesting from a generative perspective, we use a smaller value here.

- Multiple methods, a value of 15.

  A variable number of methods can be generated. Once again, the number of methods that can be generated from a design pattern can depend on the size of the input model. We assume that three methods are generated in order to obtain a rough measure of the generative potential of the design pattern.

This classification system gives an idea of the generative capabilities of each design pattern, and allows us to differentiate between the benefits of generating each of them. The values used are, however, quite subjective, and the final measure obtained can not be completely accurate in some occasions. When the amount of code that can be generated depends on the size of the input model, the real generatability of the design pattern varies with each specific implementation. Therefore, the generatability value that we obtain here should be taken as a general measure of the average potential of a pattern, but additional factors such as the number of classes involved in the implementation must be considered.

The generation-oriented analysis of two design patterns, Abstract Factory and State, is included as an example. The evaluation performed in [Mad13] for both patterns is reused and expanded, including the calculation of the generatability value.

Abstract factory

The intent of the abstract factory design pattern is to provide an interface for creating families of related or dependent objects without specifying their concrete classes [GHJV95]. Its structure is depicted in Figure 3.18 on page 49. The products are organized in families (`ProductA` and `ProductB` in the figure) and types (`1` and `2` in the figure), where a product of each type is specified in each family. The pattern defines a hierarchy of factories so that there is a concrete factory to create all the products of each specific type. A distinction is visually made in the diagram between the classes that can be generated and those that must be manually written.

The hierarchies of products need to be manually implemented, since that is completely dependent on the domain. No relevant information in them can be extracted from the definition of the design pattern, and therefore they can not be generated at all. The same applies to the `Client`. This is the class that uses the factory to create products, but there is no way that this behavior can be predicted, and the class can not be generated either.

The `AbstractFactory` interface is completely generatable. A method is defined for creating each family of products. The name and return type of each method depend on the associated family, and therefore the whole method declaration can be generated. An abstract name such as `AbstractFactory` can be set for the interface. Alternatively, the name of the product families can be used to form it. In this way, a name such as `ProductAProductBFactory` would be obtained.

Name	Generatability
Creational patterns	
Abstract Factory	79
Builder	28
Factory Method	39
Prototype	10
Singleton	5
Structural patterns	
Adapter	0
Bridge	25
Decorator	20
Facade	0
Flyweight	33
Composite	33
Proxy	10
Behavioral patterns	
Observer	0
Visitor	35
Interpreter	10
Iterator	20
Command	20
Memento	26
Template Method	0
Strategy	10
Mediator	20
State	30
Chain of responsibility	15

Table 3.1: Classification of the design patterns and their generatability

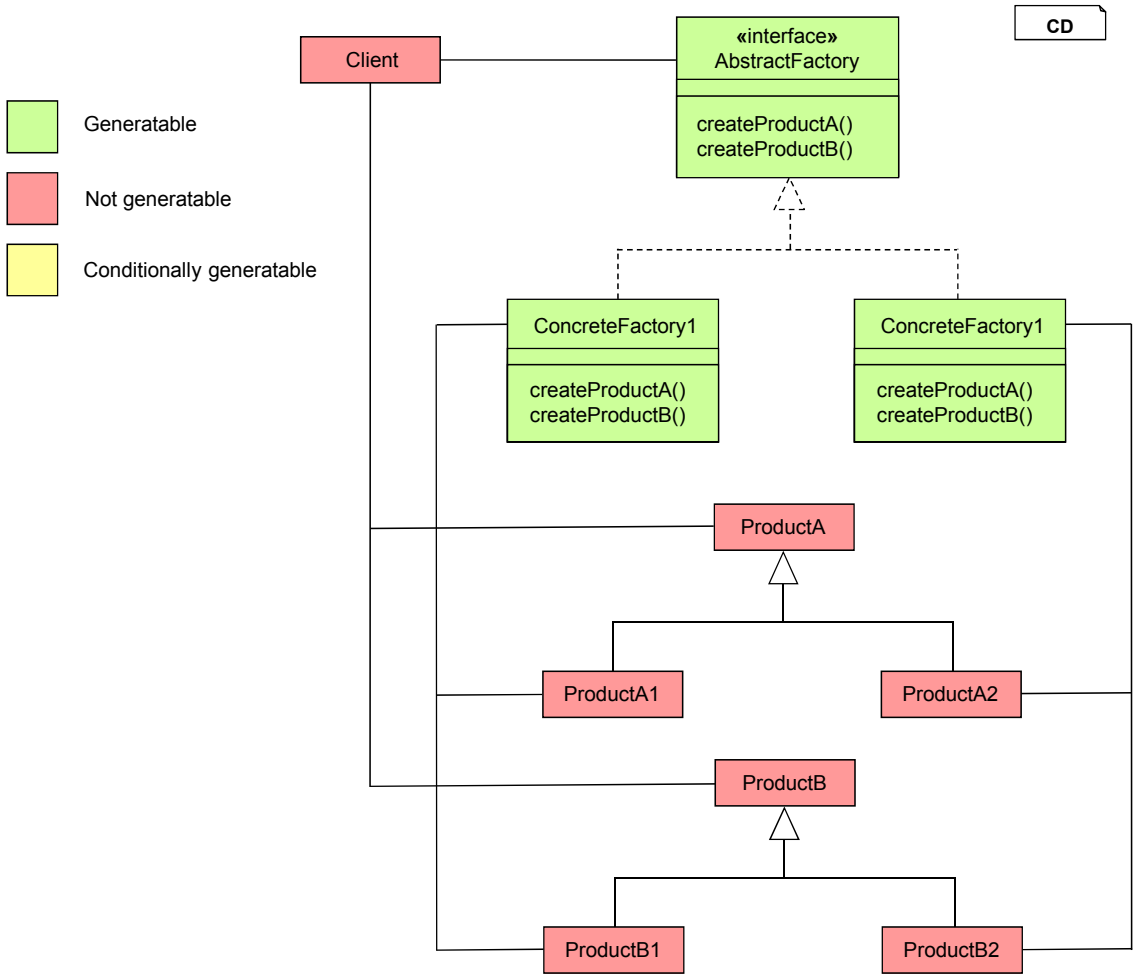


Figure 3.18: Generative-oriented classification of the structure defined by the abstract factory design pattern. Figure by Mades [Mad13].

The concrete factories can also be completely generated. A concrete factory is defined for every type of product, and can be easily named after it. In each factory, the methods declared in the interface are implemented by returning a product of the corresponding type. Listing 3.4 shows the method for the family `ProductA` in the type 1 concrete factory. The implementation code depends completely on the type associated to the factory, and can be fully generated.

```
1 public ProductA createProductA( ) {
2     return new ProductA1();
3 }
```

Listing 3.4: Example of a create method in a concrete factory.

Figure 3.19 shows an example of an implementation of the abstract factory design pattern. In this example, different GUI widgets are available with implementations for different systems. The required model is shown on the left-hand side of the figure, with the design pattern roles specified in the corresponding classes.

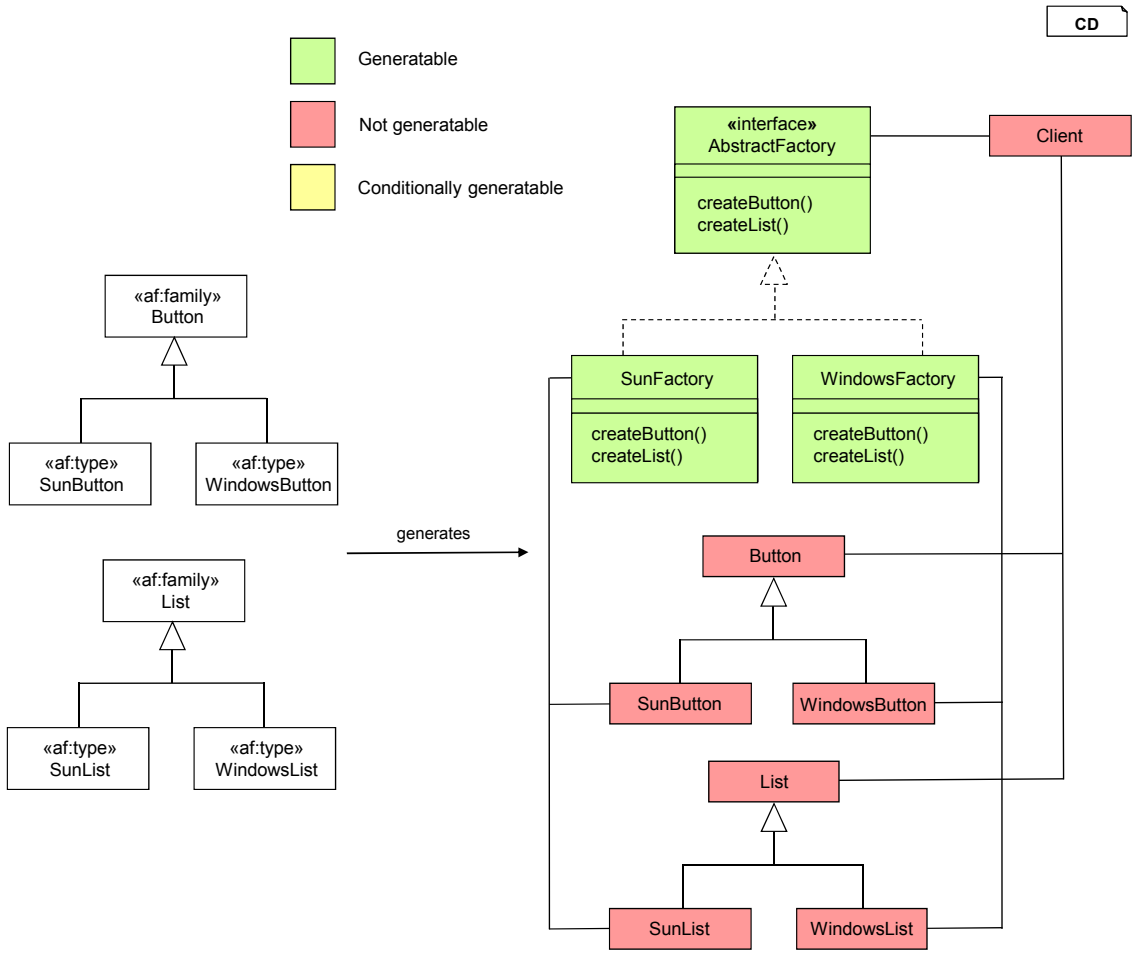


Figure 3.19: Generation example for the abstract factory design pattern.

`Button` and `List` are provided as product families, and the `Sun` and `Windows` types are specified in each family. Once this information is identified from the model, an `AbstractFactory` interface can be automatically generated with `create` methods for each product family. Two concrete factories are also generated for each product type:

WindowsFactory and SunFactory. Listing 3.5 shows the generated code for the Button's create method in the Windows concrete factory. The line for creating and returning a WindowsButton can be completely generated, and the same applies to the rest of methods in all the concrete factories. The client and the product families need to be manually written, but the whole hierarchy of factories is completely generatable.

```
1 public Button createButton( ) {
2     return new WindowsButton();
3 }
```

Listing 3.5: create method for a Button in WindowsFactory.

To calculate the generatability value of the design pattern, we identify the following generation possibilities:

- An interface. One AbstractFactory interface is always generated.
- Multiple non-abstract classes. Every concrete factory is a non-abstract class that can be fully generated. The number of concrete factories depends on the number of product types.
- Multiple methods. Every concrete factory defines a create method for every product family, whose implementation can be completely generated. The number of methods in each factory depends on the number of product types and it is not known in advance.

Taking the corresponding values, the following calculation obtains the generatability value of the abstract factory pattern:

$$10 + 24 + 3 * 15 = \mathbf{79}$$

A number of three product families and types is assumed, following the conventions defined by our classification system. Therefore, we suppose that three concrete factories are generated as well as three methods for each one of them, which added to the AbstractFactory interface results in a total generatability value of 79. This is the highest value among all the considered design patterns. The abstract factory pattern is, indeed, quite suitable for an automatic generation. Depending on the size of the product hierarchies, a fairly big number of classes can be completely generated from it.

State

The intent of the state design pattern is to allow an object to alter its behavior when its internal state changes, such that the object appears to change its class [GHJV95]. Its structure is depicted in Figure 3.20. A Context class is defined, with an associated State and some operations that will depend on this state. A hierarchy of states is defined where these operations are implemented in different ways, thus allowing the context to change its behavior depending on the specific state selected. A distinction is visually made in the diagram between the classes that can be generated and those that must be manually written.

If the Context's methods that depend on the state are specified, then the State interface can be completely generated with the declarations of such methods. As for the concrete

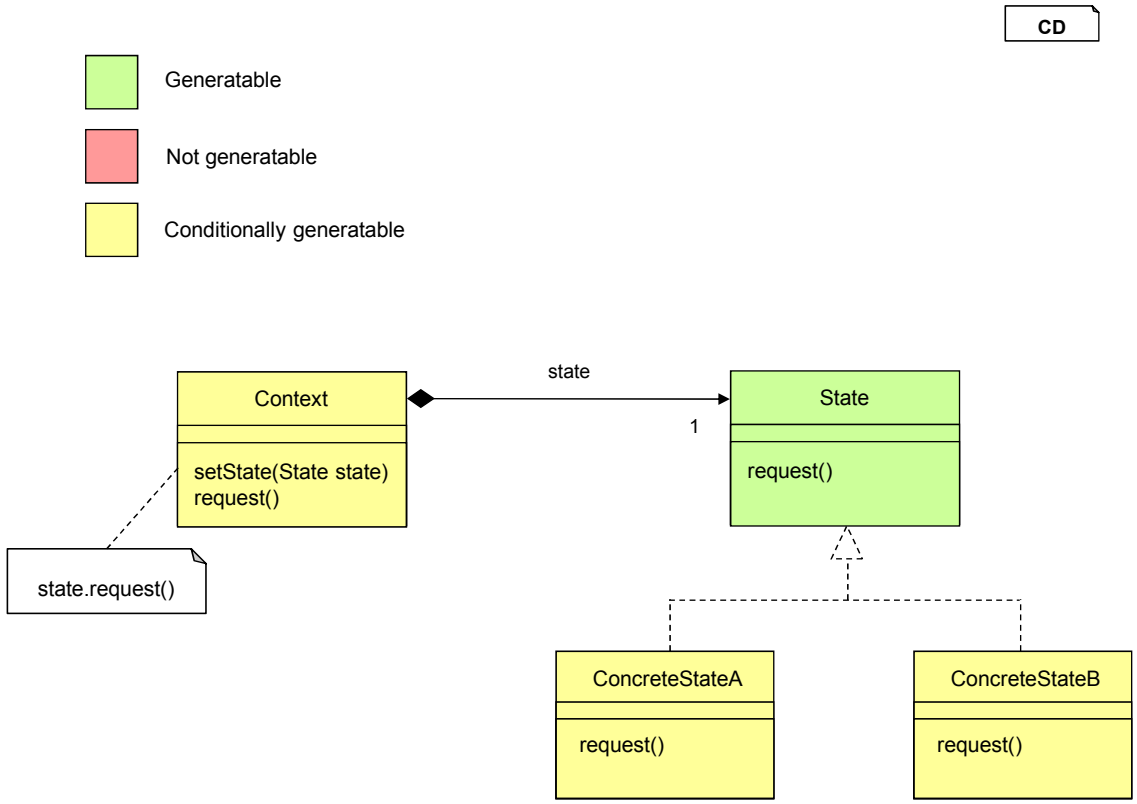


Figure 3.20: Generative-oriented classification of the structure defined by the state design pattern. Based on the figure by Mades [Mad13].

states, their methods usually need to be manually implemented. However, part of this code can also be generated if we have additional information in the model. A state machine diagram, for example, can be provided with information about all the possible states and the transitions between them. In some occasions, this information is useful to generate part of the code in the methods, and therefore these classes are marked as *conditionally generatable*.

The `Context` can be partially or completely generated too. Listing 3.6 shows a generic implementation of one of the methods that depends on the state. The action is delegated to the state, and the line can be easily generated. A `setState` method is also always generated to set the state. Only when all the methods in the `Context` depend on the state, the class can be generated completely.

```
1 public void request() {
2     state.request();
3 }
```

Listing 3.6: Implementation of a method in the `Context` class that depends on the state.

Figure 3.21 shows an example of an implementation of the state design pattern. In this example, a TCP connection with different states is defined. The required model is shown in the top part of the figure, with the design pattern roles specified in the corresponding classes.

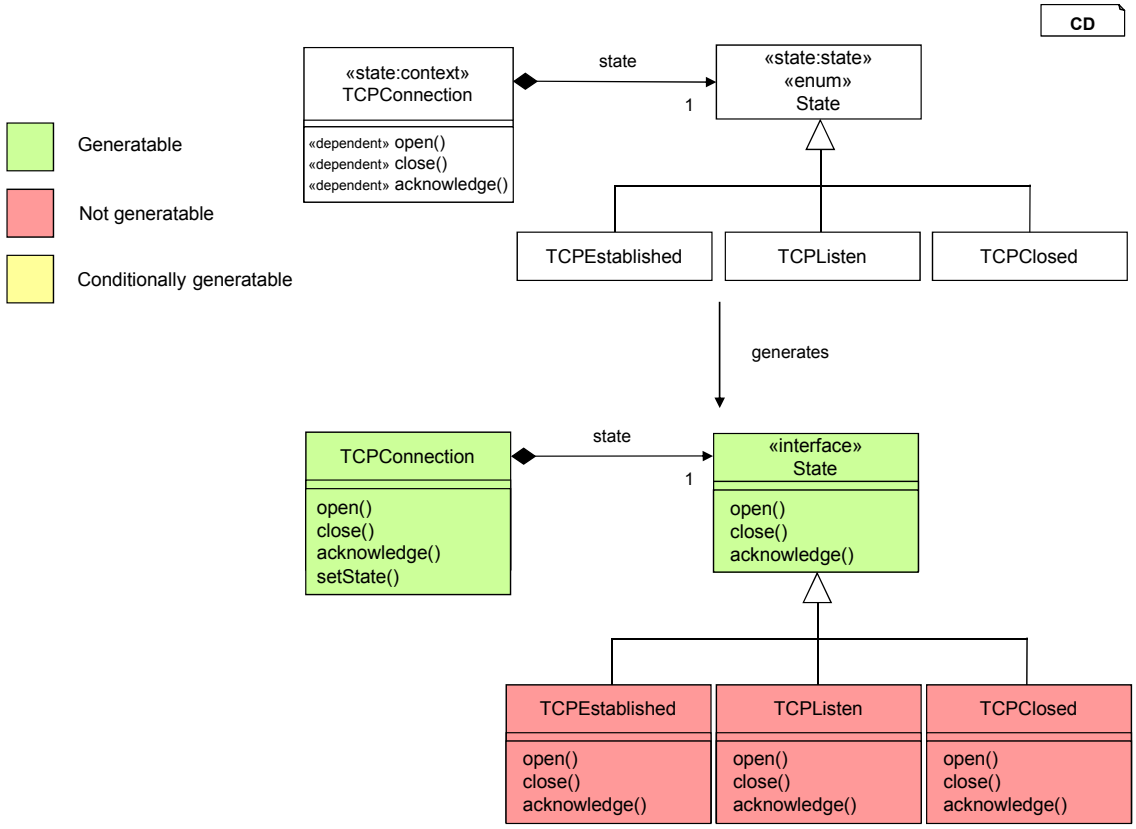


Figure 3.21: Generation example for the state design pattern.

All the three methods in `TCPConnection` are marked as dependent on the state. Con-

sequently, the whole class can be automatically generated. The implementation of these methods delegates to the state, as seen in Listing 3.7 for the `open` method. Finally, the generator also adds a `setState` method. A hierarchy of states is defined from the types specified in the model. The `State` interface can be completely generated. The implementation of the methods in each specific state must be manually written, since no additional information, such as a state machine diagram, is provided in the example's model.

```
1 public void open() {
2     state.open();
3 }
```

Listing 3.7: Implementation of the state-dependent method `open` in TCPConnection.

To calculate the generatability value of the design pattern, we identify the following generative possibilities:

- An interface. One `State` interface is always generated.
- A method. The `setState` method is always generated in the `Context` class.
- Multiple methods. The implementation of the methods that depend on the state is completely generated in the `Context` class.

We assume that the `Context` class has additional methods that do not depend on the state, and therefore can not be completely generated. In the same manner, we assume that no additional information is provided in the model that might help in the generation of the concrete states. Taking the corresponding values, the following calculation obtains the generatability value of the state pattern:

$$10 + 5 + 15 = \mathbf{30}$$

We suppose that there are three methods that depend on the state, following the conventions of our classification system. The final generatability measure is 30. While this value is not as high as the one obtained for the abstract factory pattern, the state pattern still remains as one of the most suitable patterns for automatic generation.

Process

Many different techniques have been proposed in order to automatically identify design patterns in a software system. The existing proposals range from similarity scoring algorithms [TCSH06] to fingerprint methods [YGSZ04] to bit-vector algorithms [KGH06], among other techniques. This great variety makes it difficult to talk about a general design pattern identification method. Instead, and since we do not want to tie our feature to a specific tool, we just assume that one approach is selected and describe the rest of the process.

Our method works with two values: the generatability of a design pattern and a number that we call **fit value**. The generatability of a design pattern gives a measure on the amount of code that can be generated from it. The concept was introduced in Section 3.6.2, where the design patterns of the GoF were classified according to it. The fit

value is a number ranging from 0 to 1 that measures to which extent the identified classes match the structure defined by the design pattern. In some occasions, a pattern has been implemented in a way that differs to a certain degree from the structure defined as the typical solution. These differences may hinder the automatic generation of code from the design pattern. The fit value allows us to take this factor into account and distinguish between different implementations of the same pattern.

A final measure is obtained for every instance of a design pattern, calculated from the generatability of the pattern and the fit value of its implementation as:

```
final_measure = generatability * fit_value
```

This final measure gives an idea of the amount of code that can be potentially generated. As a result of the method, the artifacts identified for each design pattern are proposed as generation candidates and ranked according to this measure.

The use of the fit value provides extra flexibility to the process. However, existing design pattern detection tools do not usually provide this kind of measure. This means that a manual analysis of the results is required in order to measure the adequacy of each implementation, and assign the fit values accordingly. As an alternative, the value can be disregarded. If, for example, the identification tool does not support any kind of flexibility in its process, then whenever a design pattern is detected, we can assume that the implementation matches perfectly the typical structure defined as a solution. In such cases, a fit value of 1 is always assigned, and only the generatability of each design pattern is used to differentiate between the proposed generation candidates.

The fit value can also be particularly useful when we are using multiple tools, which can be convenient to support the detection of a wider range of design patterns. In this scenario, there can be differences in the way each tool performs. The fit value can help us here as a way of distinguishing between the quality of the different results, by assigning higher fit values to more precise tools. Alternatively, we could also set different fit values according to the number of tools that have reported each instance. In this way, an instance that has only been reported by one tool is assigned a lower fit value that one reported by multiple tools.

There is a final factor that needs to be considered when evaluating the results obtained. In some occasions, the number of artifacts identified as implementing a specific design pattern also determines the amount of code that can be generated. This is the case for the following patterns:

- Abstract Factory
- Factory Method
- Bridge
- Flyweight
- Visitor
- State

For these design patterns, some code is generated from parts of the model which have a variable size. The amount of generated code therefore depends on the number of classes or methods in the model. When calculating the generatability of these patterns, a fixed size was defined and assumed for such parts of the model. This helped us to obtain a general, immutable generatability measure for the design pattern. However, once specific implementations have been found, it is convenient to also consider the number of identified artifacts when manually analyzing and comparing the proposed candidates.

3.6.3 Application

There are no restrictions to the design pattern identification method, since patterns can be detected in any kind of software system. However, when implementing the method, the tools selected to automate the identification process may impose their own constraints.

3.6.4 Optimizations

A more sophisticated classification method can be developed, that obtains a more accurate measure of the amount of code that can be generated for a design pattern. In particular, additional factors can be considered, that the current approach overlooks for the sake of simplicity. For example, when there is the possibility of generating one or more methods in a class, the number of lines of those methods could be obtained and used to calculate the generatability value. Currently, no distinction is made between generatable methods because, in practice, we have found no significant differences between them in terms of lines of code. However, incorporating this factor in the calculation increases its accuracy. It can also be useful in case we add the support of new design patterns where greater differences between the generatable methods exist.

In the same manner, the number of identified artifacts can be integrated into the classification process. In our current approach, a fixed number is used when calculating generatability values that depend on the size of the input model. The number of identified artifacts is only considered as an external factor during the manual interpretation of the results. As an alternative, a more sophisticated method can be defined to include this variability in the classification process. However, taking this factor into account is not trivial, since the input model can grow in different ways but only some specific parts of it influence the amount of generated code.

3.7 Structural dependencies

3.7.1 Motivation

Structural dependencies, also known as syntactic dependencies, occur whenever a compilation unit depends on another at compilation or linkage time [Lak96]. In an object-oriented context, a compilation unit refers to an abstract class, concrete class or an interface. Inheritance, interface implementation or external operation calls are examples of structural dependencies, according to the given definition. Structural dependencies have been investigated in the contexts of impact analysis [BWDP00, LM93], error-proneness [SB91], change propagation [Raj97, YCM93] or regression testing [WJRR08], among others. In

our context, structural dependencies can be used for the identification of generation candidates. We have analyzed the structural dependencies that exist in groups of generatable classes. As a result of this analysis, we identified two structural dependency patterns that help us to detect generation candidates in a system.

Pattern 1. *Artifacts that are structurally dependent on a domain class can on occasion be generated from it.*

Domain classes were introduced in Section 3.1. An artifact is structurally dependent on a domain class if it uses the class, its attributes or its methods. Such references can be easily integrated into a generative architecture as part of dynamic sections in a template. Regardless of the generatability of these references, in general a strong structural dependency means that the artifact is quite dependent on the domain class. When such a dependency exists, it is necessary to perform a more detailed analysis of the class to find out if additional parts of the artifacts' code also depend on the domain class and can be generated from it.

Pattern 2. *Artifacts that share a similar scheme of structural dependencies can on occasion be generated together.*

Two artifacts have the same scheme of structural dependencies if they inherit from the same class, implement the same interface and/or use the same set of artifacts. Since equality is too strict of a requirement in this case, some differences can be tolerated between the schemes of structural dependencies. In general, a similar scheme of structural dependencies between two artifacts can indicate that a relation exists between them. For example, if two artifacts use the same set of classes, they may implement a functionality that is related to the same domain. However, a more detailed analysis is necessary to find out if this relation allows for a common generation of the artifacts.

Both patterns have been defined as hypothesis that result from a manual inspection of generatable structures of classes, and have not been formally or empirically tested. Additional patterns could also be identified, which we have not considered.

The identification of these two patterns can help to detect generation candidates in a system. However, using only structural dependencies for this purpose has important shortcomings. Two artifacts can be structurally dependent for many reasons, and even though our patterns identify situations which are particularly interesting in our context, multiple uninteresting generation candidates can be reported with this approach. Two artifacts can accidentally have similar schemes of structural dependencies but not be generatable, nor even related. In the same way, we may be able to generate only some of the artifacts that structurally depend on a domain class. For these reasons, the patterns are more useful when applied to structures of generation candidates that have already been identified. The presence or absence of the patterns in such structures can be used as an additional factor to measure their potential as generation candidates. Consequently, it is recommended to apply the structural dependencies feature in combination with other features.

As an example, Figure 3.22 shows a structure of classes in a health-related context, similar to other examples previously used throughout this document. `Patient` and `Doctor` are domain classes, representing important concepts in the system. The DAO interfaces `PatientDAO` and `DoctorDAO` are defined to handle them in a database, and an Hibernate implementation of these interfaces is provided for each type. Each implementation

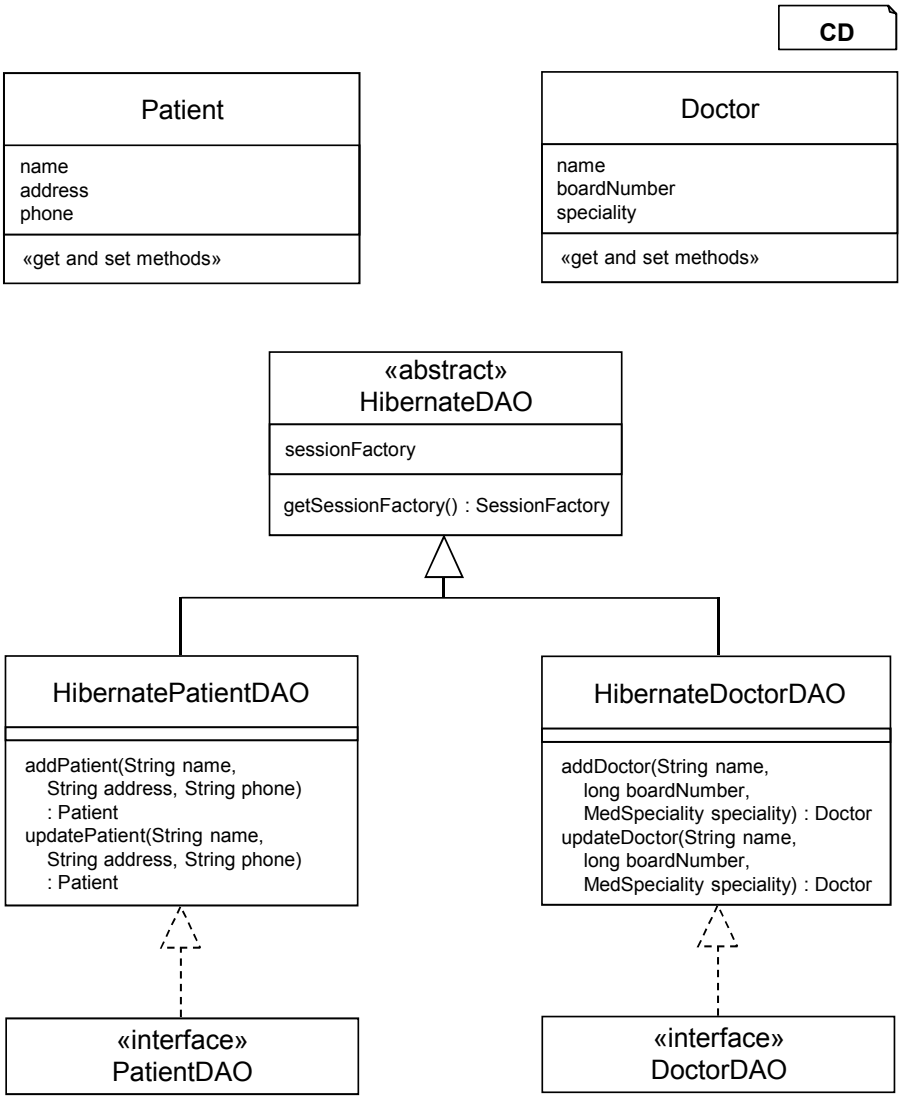


Figure 3.22: Example of a generatable structure of classes.

defines a couple of methods that take the fields of the type as parameters in order to register or update the entities. Additionally, the interface implementations inherit from a class `HibernateDAO` which includes behavior that is common to every Hibernate DAO implementation.

Overall, the structure depicted is a typical example of generatable code. The classes are good generation candidates because they heavily rely on the `Patient` and `Doctor` concepts and their attributes. Consequently, both the domain classes and the artifacts related to them can be generated partially or completely once the corresponding concepts have been defined in the model. Structural dependencies help us to recognize such a generatability potential. By analyzing the structural dependencies between the artifacts in the structure, we can check if the patterns apply and propose the structure as a more interesting generation candidate in that case.

Taking a look at the DAO classes, we see that they make use of the `Patient` and `Doctor` types extensively, since they need to create objects of such types and modify their state according to the received parameters. Consequently, a strong structural dependency exists between each DAO class and its corresponding domain class. According to Pattern 1, this reinforces the idea that they can be generated from the domain classes. If we consider all their structural dependencies, we can also compare the schemes of both DAO classes to check for Pattern 2:

- Both classes inherit from the same class: `HibernateDAO`.
- Both classes implement one single interface: `PatientDAO` or `DoctorDAO`.
- Both classes make use of one other single class in the system: `Patient` or `Doctor`[1].

As we can see, the two DAO classes have a structural dependency in common and share an overall similar structure of dependencies, which reinforces the idea that they can be generated together.

3.7.2 Description

Different notions of structural dependencies have been adopted in the literature. *Data dependencies* are sometimes considered. They exist when two compilation units refer to a same data element, such as a global variable. [WJRR08] and [SB91] are two examples where the identification of structural dependencies is performed by analyzing the data flow. However, the most widely adopted notion of structural dependency only considers when there is an explicit reference in a compilation unit to a syntactic element of another compilation unit, such as an invocation of one of its methods. This notion of structural dependency is used in [SJSJ05], whereas [OG11] uses a similar definition based on a metric called Message Passing Coupling (MPC) [LH93]. MPC measures the number of external operation calls of a class, that is, the number of calls from methods of a class to operations of other classes. Oliva et al. [OG11] define MPC* as a restricted version of MPC oriented to the identification of structural dependencies. MPC* measures the number of calls from methods of a class `P` to operations of another single class `Q`. We say that class `P` is

[1] We assume that no other class is used in the implementation of the *add* and *update* methods shown in the figure.

structurally dependent on class `Q` if $MPC * (P \Rightarrow Q) > 0$, i.e. if at least one operation of `Q` is called from `P`. A greater MPC* measure represents a stronger structural dependency between the classes. We propose the use of MPC* to measure the structural dependencies between artifacts. This helps us to detect Pattern 1, as identified in the motivation section.

For the identification of Pattern 2 we need to extend our definition of structural dependency. Class `P` is now structurally dependent on class `Q` if one of the following conditions is met:

- `P` inherits from `Q`.
- `Q` is an interface and `P` implements `Q`.
- $MPC * (P \Rightarrow Q) > 0$, which corresponds to our initial definition.

According to this extended definition, we calculate the dependencies for each artifact `P` and store them in three sets:

- A set `C` containing the superclasses of `P`.
- A set `I` containing the interfaces implemented by `P`.
- A set `U` containing the artifacts such that $MPC * (P \Rightarrow Q) > 0$, i.e. the artifacts used by `P`.

Then, if we want to bring into comparison the schemes of structural dependencies of two artifacts `P` and `Q`, we individually compare their three sets `C`, `I` and `U` taking into account two factors:

- **The sizes of the sets**. We calculate the relative difference between the two sizes `x` and `y` as $d_r(x, y) = \frac{|x-y|}{max(|x|,|y|)}$, where $0 <= d_r <= 1$.
- **The similarity between the sets**. We use the Jaccard index [LW71] for comparing sets, which defines the similarity between two sets S_1 and S_2 as $J(S_1, S_2) = \frac{|S_1 \bigcap S_2|}{|S_1 \bigcup S_2|}$, where $0 <= J(S_1, S_2) <= 1$.

Thus, we consider not only that the artifacts have dependencies in common but also that they share a similar number of them. Both factors are given the same importance, and the final measure for two sets S_1 and S_2 is calculated as

$$setSimilarity(S_1, S_2) = 1 - d_r(|S_1|, |S_2|) + J(S_1, S_2)$$

such that a smaller relative difference between the sizes of the sets and a greater Jaccard similarity results in a higher value. Finally, the values resulting from the three set comparisons are added to obtain a final measure of the structural dependencies similarity between the artifacts:

$$structuralSimilarity(P, Q) = setSimilarity(C_P, C_Q) + setSimilarity(I_P, I_Q) + setSimilarity(U_P, U_Q)$$

A static analysis of the code is required to apply both of the metrics presented here. The details of such an algorithm, however, heavily depend on the programming language. We consider them implementation-related, and therefore we do not include them in the description of the feature.

3.7.3 Application

As it was previously motivated, this feature is meant to be used as an additional factor in the evaluation of already identified generation candidates. Following this line of thought, the detection of structural dependencies can be integrated, for example, with the name similarity feature described in Section 3.2. In this feature, structures of related artifacts are identified along with their corresponding domain classes. The metrics described here can be applied to such structures in order to verify if the defined patterns can be found, acting as an additional factor for the filtering and/or categorization of the results.

3.8 Logical dependencies

3.8.1 Motivation

Logical dependencies, also known as change dependencies or evolutionary dependencies, are implicit dependencies between software artifacts that evolved together [GHJ98, BmKPS97]. A logical dependency is established between source code files that are changed together, that is, committed together in a version control system. Logical dependencies have been employed in a wide range of contexts. They can be useful to, for example, detect design issues [DLL09], predict changes in software artifacts [KM07, ZWDZ04] within the context of the impact analysis (IA) field [Leh11] or infer code decay [ESM+02]. For their identification, repository mining techniques are usually employed to parse and analyze the logs of the version control system [DGLP08, KCM07]. In our context, logical dependencies can be used for the identification of generation candidates. We have analyzed the logical dependencies that exist in groups of generatable classes. As a result of this analysis, we identified two logical dependency patterns that can help us to detect generation candidates in a system.

Pattern 1. *Artifacts that are logically dependent on a domain class can on occasion be generated from it.*

Domain classes were introduced in Section 3.1. If an artifact needs to be updated when a domain class is changed, this can mean that the code of the artifact depends somehow on the domain class. The strength of this dependency can be estimated from the frequency with which the artifacts are changed together. Although there is little more that we can say about the artifacts just from the analysis of their history, this pattern helps us to identify a relation between them that can be interesting for generation purposes. A more detailed analysis of the artifact is necessary to find out if the code is indeed tightly related to the domain class and can be generated from it.

Pattern 2. *Artifacts that are logically dependent on each other can on occasion be generated together.*

When two artifacts are regularly committed together, this can mean that they have a similar functionality or role in the system. Whenever code related to such functionality is changed, both artifacts must be updated, thus creating a logical dependency between them. In our context, such relation can indicate that a common generation is possible. The artifacts may share some similarities that derive from the common functionality or role they implement, and these similarities could be integrated into a generative architecture, thus making the artifacts interesting generation candidates.

Both patterns have been defined as hypothesis that result from a manual inspection of generatable structures of classes, and have not been formally or empirically tested. Additional patterns could also be identified, which we have not considered.

The identification of these two patterns can help to detect generation candidates in a system. However, using only logical dependencies for this purpose has important shortcomings. Artifacts can be logically dependent for many reasons, which we further analyze in Section 3.8.2. Even though our patterns identify situations which are particularly interesting in our context, multiple uninteresting generation candidates can be reported with this approach. For example, we may be able to generate only some of the artifacts that logically depend on a domain class, and find many artifacts that, despite being logically dependent, can not be generated together. For these reasons, the patterns are more useful when applied to structures of generation candidates that have already been identified. The presence or absence of the patterns in such structures can be used as an additional factor to measure their potential as generation candidates. Consequently, it is recommended to apply the logical dependencies feature in combination with other features.

There can be a connection between structural and logical dependencies. Both features also share a similar motivation and are meant to be applied in a similar way. Consequently, questions may arise about the interrelation between these two types of dependencies, and the interest of defining them separately as individual features. We refer to the work by Oliva et al. [OG11], a empirical study about the interplay between structural and logical dependencies. The analysis of the dependencies in 150,000 different commits of the Apache Software Foundation Subversion repository revealed that only a small intersection exists between the sets of structural and logical dependencies. For example, as much as 91% of logical dependencies were found to involve files that are not structurally related. This conclusion is supported by additional work such as [Han07] or [CMRH09]. This means that the identification of logical dependencies can yield additional results not detected by the mere analysis of structural dependencies. In short, the information obtained from the application of one of these features does not necessarily overlap, but complements that obtained by the other.

3.8.2 Description

Logical dependencies are commonly treated as data mining association rules. Oliva et al. [OG11] formally defines an association rule as an implication of the form $X_1 \Rightarrow X_2$, meaning that when X_1 occurs, X_2 also occurs. In this notation, X_1 and X_2 are two disjoint sets of items, and are called the antecedent and the consequent of the rule respectively. In the context of logical dependencies, a logical dependency from a file f_1 to another file f_2 is denoted by $F_1 \Rightarrow F_2$, i.e. an association rule in which the antecedent and consequent are both singleton sets containing f_1 and f_2 respectively.

Measures of interest and significance for association rules are also defined in [OG11], adapted from the concepts formalized by Zimmermann et al. [ZWDZ04]:

- **Frequency** of a set F in a set of commits C as $frq(C, F) = |\{c|c \in C, F \subseteq c\}|$.

  The frequency measure defines the number of commits containing all the files in F.

- **Support** of a rule $F_1 \Rightarrow F_2$ by a set of commits C as $supp(C, F_1 \Rightarrow F_2) = frq(C, F_1 \bigcup F_2)$.

  The support measure denotes the number of times the artifacts were changed together.

- **Confidence** of a rule $F_1 \Rightarrow F_2$ as $conf(C, F_1 \Rightarrow F_2) = frq(C, F_1 \bigcup F_2)/frq(C, F_1)$, i.e. the probability of finding the consequent of the rule in commits under the condition that these commits also contain the antecedent.

  The confidence measure defines the degree to which artifacts are logically connected, thus characterizing the strength of the relation.

In order to perform the extraction and evaluation of the association rules in a software repository, different mining tools can be used. [OG11] employs XFlow [SOdSG11], an interactive tool that works with SVN repositories. XFlow implements interesting features such as a custom sliding time window algorithm [ZW04]. In some occasions, developers commit code related to the same task in different, but close moments. A sliding time window algorithm handles this situation by grouping revisions from an author within a specified time window. An alternative is eRose [ZWDZ04], an Eclipse plugin that mines CVS repositories to identify logical dependencies and guide developers along related changes.

Once logical dependencies are identified, the results must be analyzed and filtered in order to detect the real interesting generation candidates. The empirical study by Oliva et al. [OSGdS11] investigates the origins of logical dependencies, and identifies several categories. Only two of these categories correspond to the patterns that we have identified. We present all the categories below, and analyze their interest in our context:

- **Refactoring elements that belong to the same semantic class.** Artifacts that change together due to refactoring actions make upon a **semantic class**. We denote by semantic class a group of software artifacts that intrinsically share the same basic functionality or architectural role [OSGdS11]. This type of logical dependency corresponds to Pattern 2, as defined in the motivation section, and is therefore interesting to identify generation candidates.

- **Structural dependencies on a changing semantic class.** This category is a special case of the previous one. The logical relations are characterized as a side-effect of refactoring actions made upon a semantic class. An illustrative example is given in Figure 3.23. Classes A, B and C belong to a semantic class that went through a change: changing `id` attribute type from `int` to `long`. Since class D structurally depends on class A, it also gets updated. This means that, even though D only depends structurally on A, a logical dependency also exists involving files D and B, as well as D and C, since these files are also changed together. Hence, structural dependencies on an element belonging to a semantic class can originate multiple logical dependencies. Nevertheless, these are indirect dependencies that do

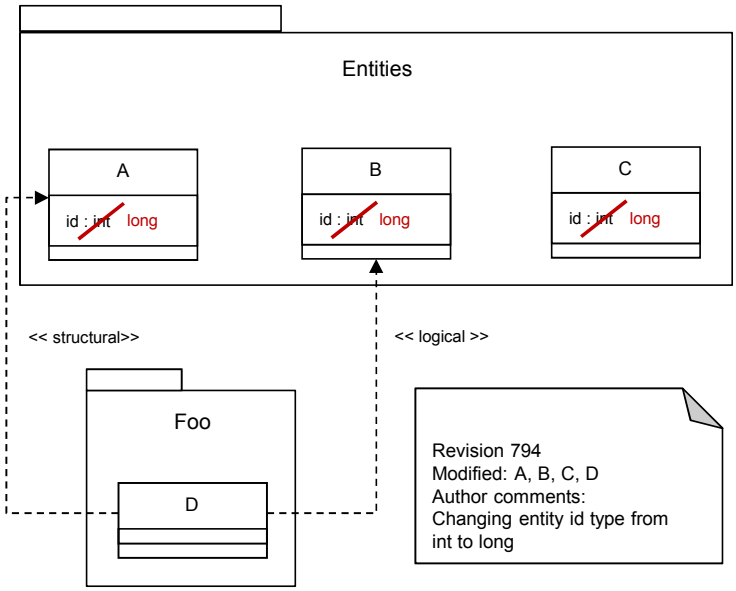


Figure 3.23: Example of a structural dependency on a changing semantic class, by Oliva et al.

not reflect an actual relation between the classes. Therefore, such logical dependencies do not help to identify situations that are interesting for generation purposes.

- **Cross-cutting concerns.** Cross-cutting concerns refer to non-core concerns that are spread among a significant amount of modules of a software system, such as logging, transaction management or concurrency control. These issues are too general to be included in generative architectures, since such code is dispersed across the system. Consequently, this type of logical dependency is not relevant for our purposes.

- **Overloaded revisions.** Pairs of files may change together simply by chance or convenience. For example, an author can modify different files for different reasons and commit them all together. No real, meaningful relation exists between such files, and so these dependencies do not provide information useful for the identification of generation candidates.

- **Repository operations.** Repository operations usually involve moving a large number of files across folders. As with overloaded revisions, these dependencies are not relevant for our purposes for obvious reasons.

- **Structural dependencies on specific elements.** Software artifacts may change together due to structural dependencies from clients to specific suppliers. This type of logical dependency corresponds to Pattern 1, as defined in the motivation section, and is therefore interesting to identify generation candidates.

  Since we are only analyzing the repository information, we are able to identify any kind of structural dependency regardless of the specific technologies related to the artifacts. This is particularly useful to detect dependencies between artifacts implemented in different languages, e.g. database scripts or GUI files that depend on a domain class.

- **Other reasons.** For example, classes can change together because they undergo code formatting. Furthermore, we include in this category some of the additional possible origins referenced by [OSGdS11], such as cascading function calls [CMRH09] or the use of dependency injection and reflection techniques [dRCFdS08]. No further origins relevant to our purposes have been identified.

We refer to [OSGdS11] for further details on these categories. The amount of identified categories proves the usefulness of a filtering process, since logical dependencies can have many different origins and only two of them are interesting for our purposes. In fact, in the case study reported by [OSGdS11] such types of dependencies only represent a 35,8% of the total. Unfortunately, logical dependency detection tools do not implement an automated categorization of the results, and therefore the origins must be manually identified. Furthermore, even within these categories the results must be filtered since not all the dependencies of such types are useful for our purposes, and so a very laborious analysis is required when many results are reported.

3.8.3 Application

As it was previously motivated, this feature is meant to be used as an additional factor in the evaluation of already identified generation candidates. Following this line of thought, the detection of logical dependencies can be integrated, for example, with the name similarity feature described in Section 3.2. In this feature, structures of related artifacts are identified along with their corresponding domain classes. The metrics described here can be applied to such structures in order to verify if the defined patterns can be found, acting as an additional factor for the filtering and/or categorization of the results.

Chapter 4

Tool overview

A tool has been developed in the course of this thesis to offer an implementation of some of the features that have been identified and described. In this way, we can see first-hand how the features perform. In this section, an overview of the structure of the tool is given, addressing general issues about its usage and implementation. For a more detailed explanation of how each feature has been implemented, see Section 5.

Java has been selected as the implementation language, and the Eclipse Plug-in Development Environment (PDE) as the development environment. The tool has been defined as an Eclipse plug-in. Eclipse [Ecl16b] is one of the most widely used Integrated Development Environments (IDE), particularly for the Java language. Eclipse's PDE subproject aims at the implementation of plug-ins to be used in Eclipse's IDE, and it offers a great variety of tools and libraries to develop plug-ins in order to add new functionality on top of the main IDE. This kind of mechanism is particularly useful to implement tools like ours, where code is the input of the system and we need to work with classes and projects and perform reverse-engineering actions. The integration of our tool with such a widespread programming IDE is therefore of great interest, since developers do not need to install new and independent tools. Instead, they can work within their own existing workspaces, making use of the generation-related functionality that we offer directly on their ongoing Java projects.

The usage of our tool comprises four steps, depicted respectively in Figures 4.1, 4.2, 4.3 and 4.4:

1. **File selection**. The features are applied on the file or files selected in the Project Explorer view in Eclipse.

2. **Feature selection**. In the context menu, the features can be launched from a menu called *Generation Candidates Detector*. The options shown may be different according to the selection, since not every feature can be applied on the same kind of selection.

3. **Configuration**. Once we have decided to apply a feature on a selection of files, a wizard containing one or more pages presents us with different configuration options. When we complete these steps, the feature is launched. Alternatively, the feature can be directly applied using its default configuration.

There is currently no generic approach for the combination of any two features. Instead, every desired combination must be defined by integrating the configuration steps of each feature and their results reports.

4. **Output inspection**. A HTML report with the results is created in the output path specified during the configuration steps. The user can navigate through the results and analyze them in order to extract the actual generation candidates.

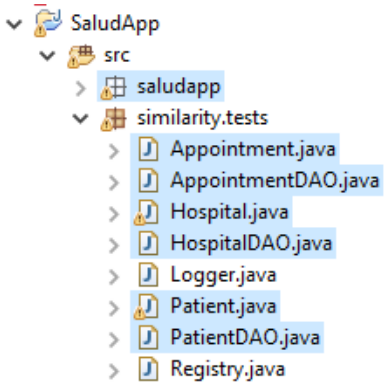


Figure 4.1: Usage of the tool. Step 1: File selection.

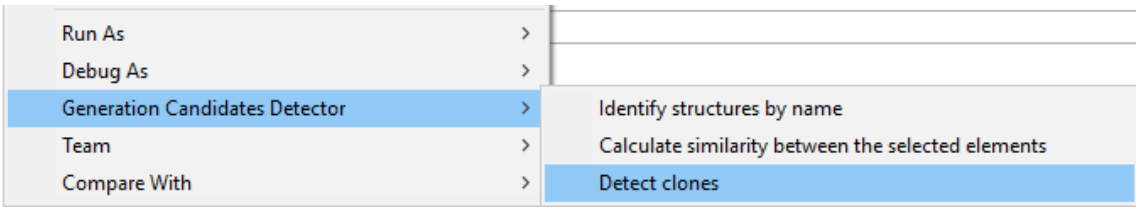


Figure 4.2: Usage of the tool. Step 2: Feature selection.

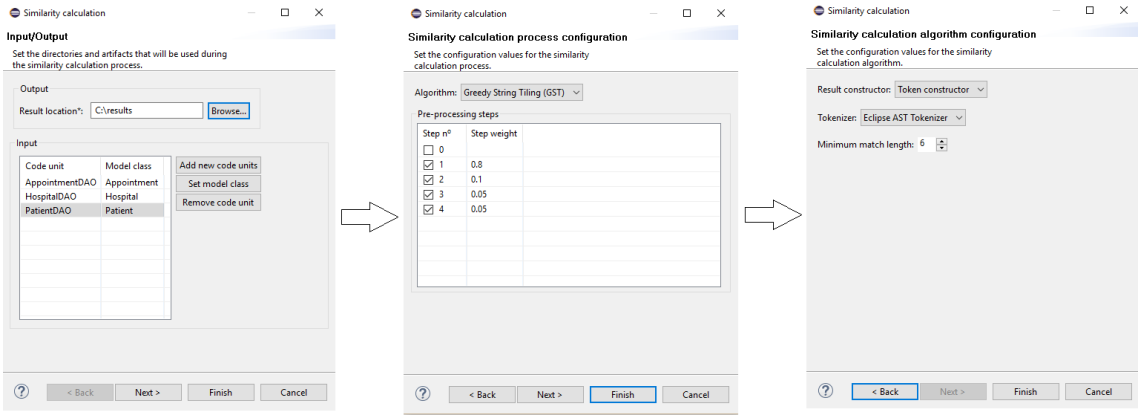


Figure 4.3: Usage of the tool. Step 3: Configuration.

Integrating a feature with our plugin comprises three different tasks:

- **Input processing**. We need to obtain the selected files and create the Eclipse wizard that allows the user to set the different configuration values.

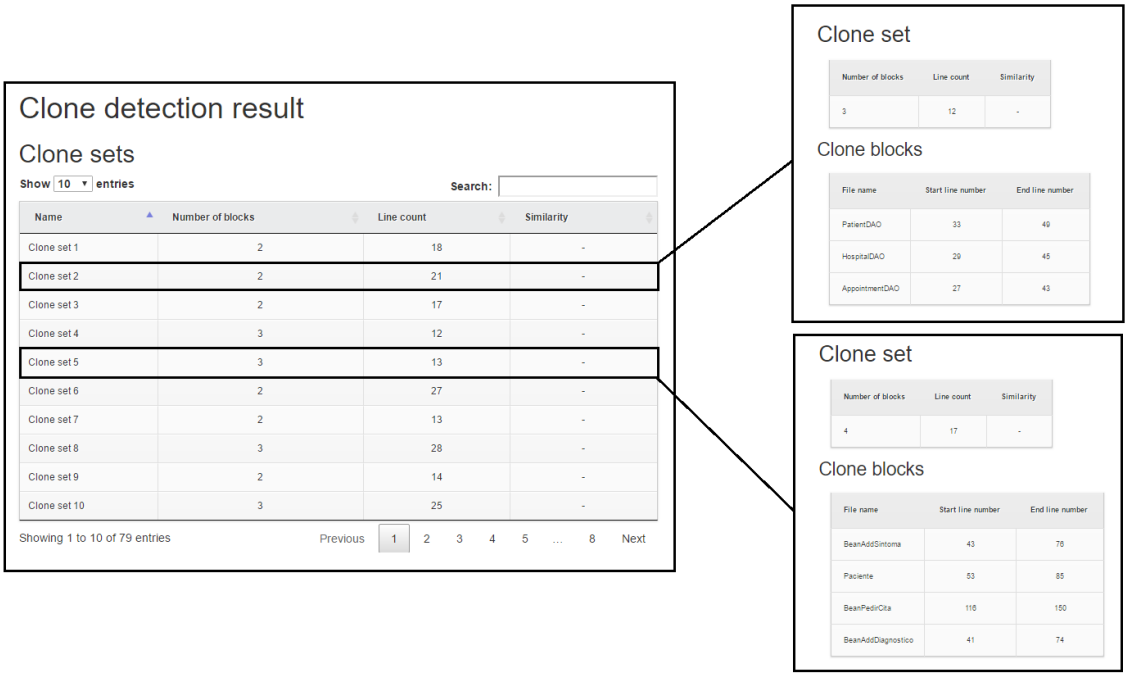


Figure 4.4: Usage of the tool. Step 4: Output inspection.

- **Core implementation**. A direct implementation of the feature as described in Section 3 must be provided.
- **Output processing**. We use FreeMarker [Fre16], a Java template engine, to generate HTML reports from the obtained results. A template must be defined to handle the results of each feature.

This separation not only allows for better comprehensibility of the code, but also fosters reusability. Core implementations of the features can be easily reused in different contexts, since they are independent of the way the input/output is handled. In Section 5 we go through every implemented feature in more detail, giving an overview of its core implementation and usage.

Chapter 5

Implementation of the features

Table 5.1 summarizes the features identified in Section 3, and marks the ones for which an implementation has been provided. The available implementations are presented and described in this section.

Name	**Implemented**
Identification of domain classes	X
Name similarity	✓
Clone detection	✓
Custom clone detection	✓
Clone evolution	X
Design patterns	✓
Structural dependencies	X
Logical dependencies	X

Table 5.1: List of implemented features.

5.1 Name similarity

5.1.1 Core implementation

The class diagram in Figure 5.1 depicts the structure of classes that has been developed to implement the name similarity feature described in Section 3.2. An interface has been defined, `INameSimilarityAlgorithm`, with the methods required to launch the application of the feature. The procedure can take either a single list of file names or two separate lists for the names being searched and the names in which the replacements are made, thus adopting one of the suggestions made in the *Application* section. An implementation of this interface is provided, `NameSimilarityAlgorithm`, which executes the name similarity detection process using some configuration values and returns the corresponding result.

An almost direct implementation of the name similarity algorithm as expressed in Section 3.2 has been developed. Only one minor aspect was added: a length restriction for the names used in the replacements, since practical execution of the method has probed

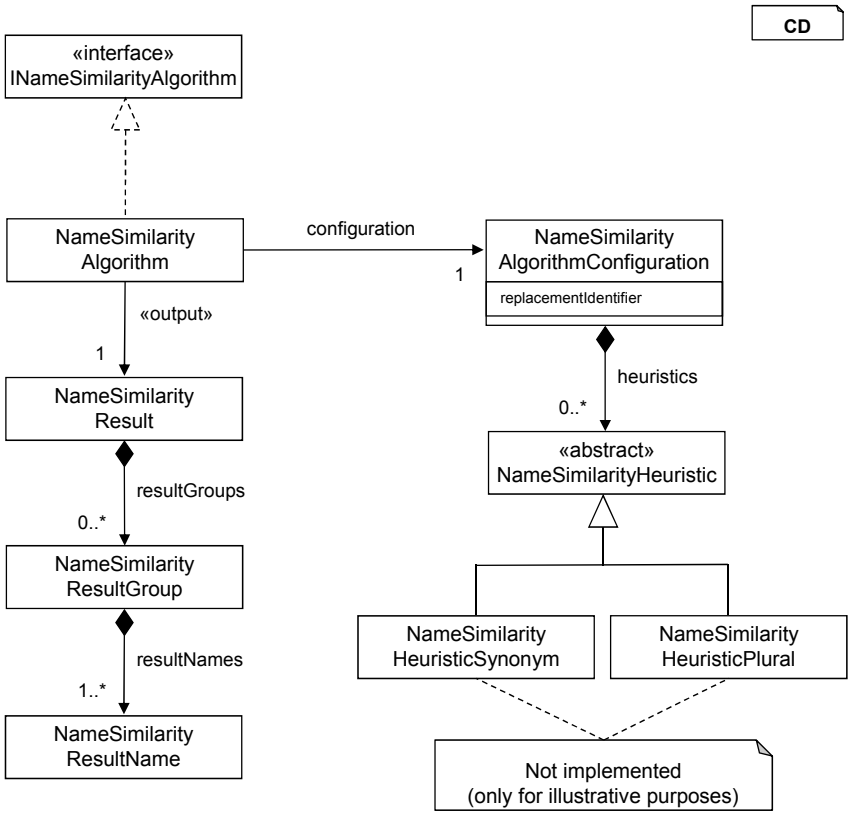


Figure 5.1: Class diagram for the implementation of the name similarity feature.

that short names usually yield bad results, causing multiple replacements that result in the identification of invalid groups of related files.

Heuristic support has been added to the tool, that is, its execution has been integrated into the overall process through the `NameSimilarityHeuristic` interface. However, no heuristic has actually been implemented yet, and therefore none are available in the current version of the tool. Thus, the classes `NameSimilarityHeuristicPlural` and `NameSimilarityHeuristicSynonym` are only shown in the figure for illustrative purposes.

Finally, a global result is created with the identified groups of related files, each one containing the file names found, together with the ones used in the corresponding replacements. There is a difference regarding the reported groups in relation to the theoretical description of the algorithm, as in our implementation there is always one group for every file. This means that groups containing only one file are also reported. Since these groups are irrelevant, we filter them out before delivering the results to the user.

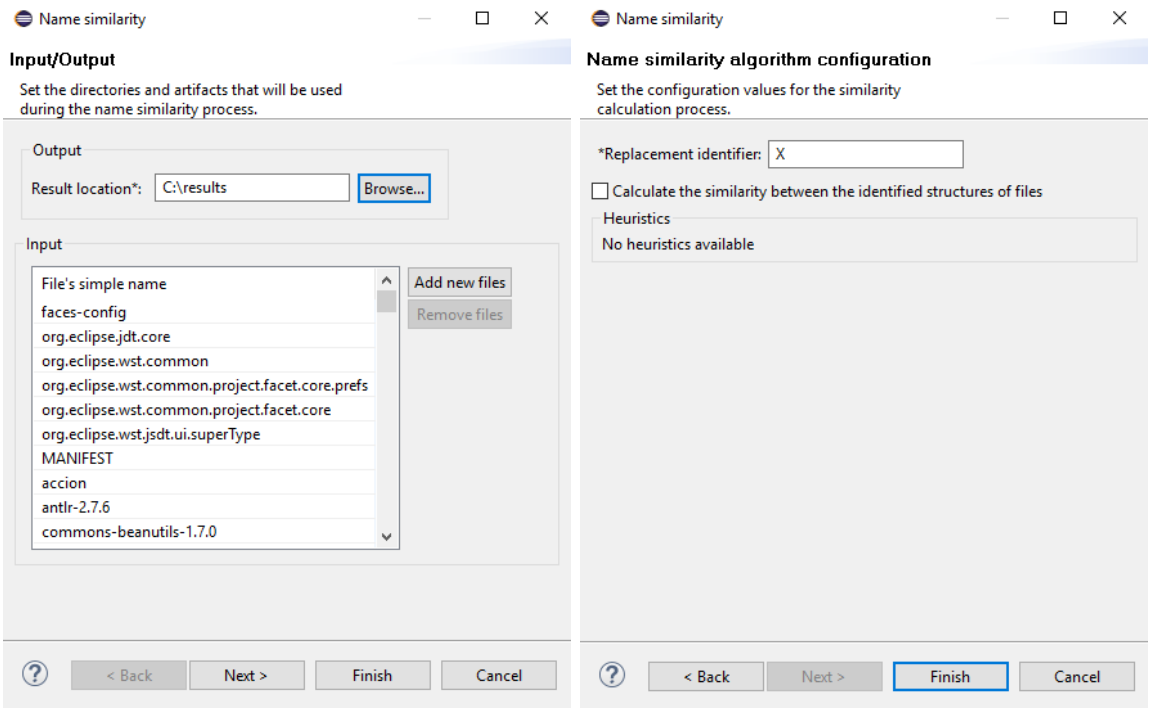
5.1.2 Usage

The name similarity feature can be applied on any group of files. If applied on containers, such as packages, folders or projects, each container is recursively searched and all the contained files are taken as input. Due to the simplicity of the process, not many configuration options are available. Figure 5.2 shows the pages forming the configuration wizard for the name similarity feature. It contains two pages:

- **Input/output**. The initial user's selection can be modified, and a path must be specified to place the results.

- **Algorithm configuration**. The following aspects can be customized:

  - The replacement identifier used during the search-and-replace process. This value will appear in the names of the identified groups. X was the string used during the theoretical description of the process in Section 3.2, and is set as the default value in this implementation.
  - The integration with the custom clone detection feature. If selected, this feature will be applied to every pair of files contained in each identified group. The configuration pages for this feature are in turn added to the wizard.
  - The set of heuristics to be applied. As already mentioned, although no heuristics are available in the current implementation of the tool, their support has been added.

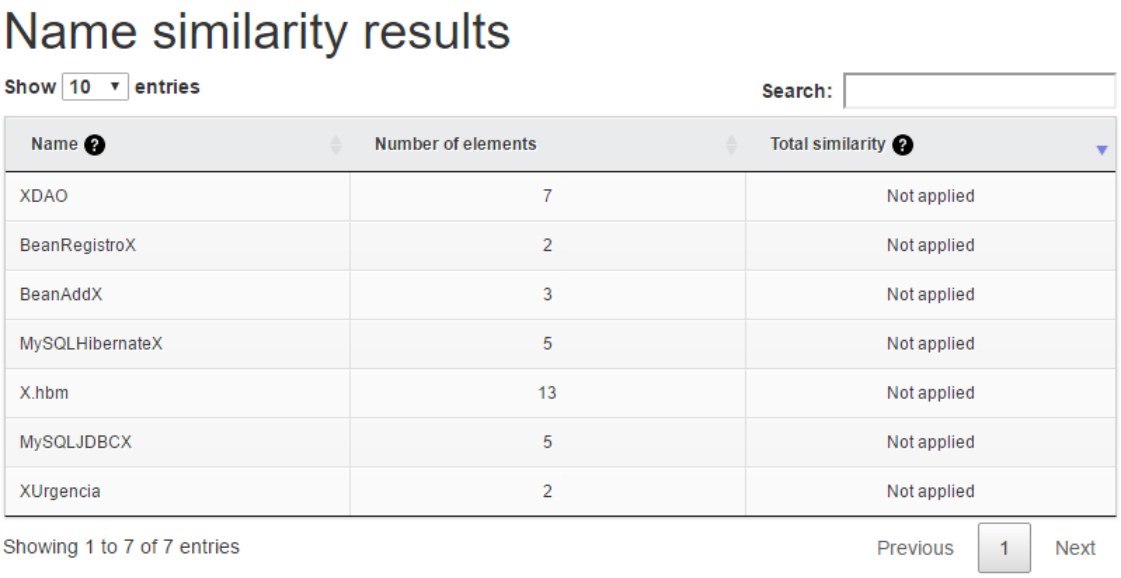


(a) Input and output

(b) Algorithm configuration

Figure 5.2: Configuration steps for the name similarity feature.

A group of HTML files is generated to show the results of the process. An index file is created as seen in Figure 5.3. It lists in a sortable table all the groups of related files that have been identified, along with their basic attributes. Each row in the table links to a different HTML file containing more detailed information about the corresponding group, as seen in Figure 5.4. If integrated with the custom clone detection feature, a link to these additional results is provided next to the total similarity measure of each group.



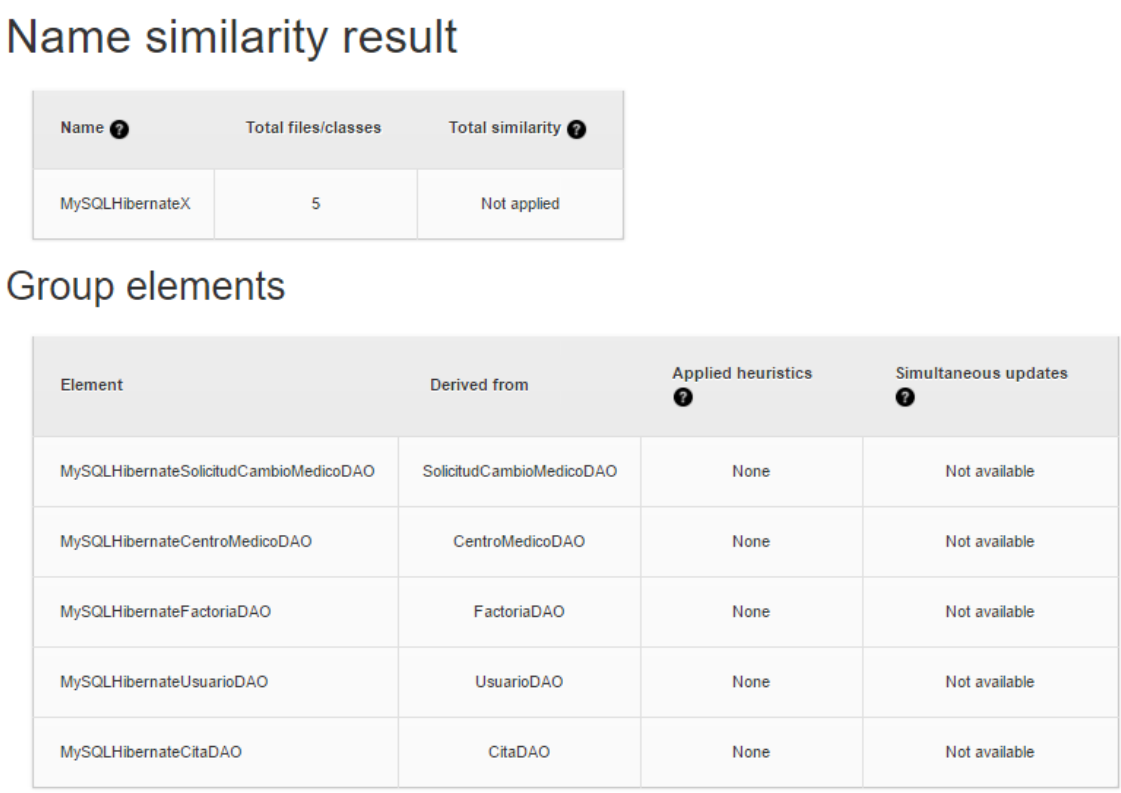
Name similarity results

Show 10 entries Search:

Name	Number of elements	Total similarity
XDAO	7	Not applied
BeanRegistroX	2	Not applied
BeanAddX	3	Not applied
MySQLHibernateX	5	Not applied
X.hbm	13	Not applied
MySQLJDBCX	5	Not applied
XUrgencia	2	Not applied

Showing 1 to 7 of 7 entries Previous 1 Next

Figure 5.3: HTML index file generated as part of the results report.



Name similarity result

Name	Total files/classes	Total similarity
MySQLHibernateX	5	Not applied

Group elements

Element	Derived from	Applied heuristics	Simultaneous updates
MySQLHibernateSolicitudCambioMedicoDAO	SolicitudCambioMedicoDAO	None	Not available
MySQLHibernateCentroMedicoDAO	CentroMedicoDAO	None	Not available
MySQLHibernateFactoriaDAO	FactoriaDAO	None	Not available
MySQLHibernateUsuarioDAO	UsuarioDAO	None	Not available
MySQLHibernateCitaDAO	CitaDAO	None	Not available

Figure 5.4: HTML file with the information about one identified group, generated as part of the results report.

5.2 Clone detection

5.2.1 Tool selection

Clone detection was motivated and identified as one of the features for detecting generation candidates in Section 3.3. We encourage the use of existing clone detection tools for the implementation of this feature, since there is a great variety of them already available. Three factors have guided our selection process:

- The clone detection tool must be open source or freely available.
- The clone detection tool must be able to process Java code. This language is the main focus of our feature implementations.
- The clone detection tool must be easy to integrate into our environment (Eclipse) or into our plugin. This way it can be easily accessed and/or combined with the rest of the features, without the need of external, independent tools or environments. Ideally, the clone detection tool must be easy to execute from Java, so that a new feature implementation is provided by our plugin which internally runs the tool. As an additional requirement, the output must be generated in a way that allows the extraction and manipulation of the results by our plugin. Alternatively, the clone detection tool can be itself an Eclipse plugin. In this way, even if an external dependency is added, it remains in the same environment and, thus, easily accessible to the user.

The comparison and evaluation of clone detection tools performed by Roy et al. [RCK09] is one of the main studies of this type and remains as a reference in the clone detection field. We have decided to use the results of this work in order to select a tool for our implementation. In this paper, Roy et al. define a scheme for describing the properties of clone detection tools, and use it to classify and compare them. Such properties are organized into **facets**, each of which may have different but not necessarily disjoint **attribute values**. Related facets are grouped into **categories**. In the following, we go through every category and every facet defined in the paper and identify the values that are interesting for us. This information helps us to filter the tools and select the most appropriate ones for our context. Every attribute value's declaration statement has been taken directly from the paper [RCK09]. We refer to it for further details as well as to consult the rest of attribute values that we have not considered.

Usage

- **Platform**. Ideally, the tool should be platform independent. However, it is sufficient for us if it can run on Windows. Consequently, one of the following attribute values is required (the attributes are mutually exclusive):
  - P.a - The tool is platform independent
  - P.c - The tool has been run on Windows
  - P.d - The tool has been run on both Windows and Linux/Unix

- **External dependencies**. Ideally, the tool should have no external dependencies. However, depending on the nature of such dependencies, the tool could still be easily integrated. In consequence, we do not require but only recommend the following attribute value:
  - D.a - Possibly the tool has no external dependencies
- **Availability**. This is one of the key factors. One of the following attribute values is required:
  - A.a - The tool is open source
  - A.b - The tool is freely available for research in binary form

Interaction

- **User interface**. The kind of user interface provided by the tool is not relevant to its integration into our plugin or environment.
- **Nature of output**. If we want to integrate the tool into our plugin, we need to parse the results, and therefore a textual output is required. However, this is not necessary for an integration into Eclipse. Consequently, the following attribute values are recommended but not required:
  - O.a - Emits results textually providing only the source coordinates of the cloned fragments (e.g., file name and begin-end line numbers of the cloned fragments)
  - O.c - Both textual source coordinates of the cloned fragments and original source in suitable format or abstracted visual representation
- **IDE support**. The only IDE into which a tool can be integrated or on which it can be dependent is Eclipse. One of the following attribute values is therefore required:
  - I.a - Is a plug-in for Eclipse
  - I.c - Others (often no dependent on any IDE)

Language

- **Language paradigm**. Since the tool must be able to work with Java, the support of the object-oriented paradigm is required and only the following attribute values are accepted:
  - LP.b - Applied to only object-oriented languages
  - LP.c - Applied to both procedural and object-oriented languages
- **Language support**. Java must be supported, and therefore one of the following attribute values is required:
  - LS.a - Is language independent
  - LS.e - Experimented with Java

Clone

- **Clone relation**. This facet concerns how clones are reported: as clone pairs, clone classes, or both. Clone classes are preferred, but not required:
  - R.d - Groups clone pairs in classes in post-processing
- **Clone granularity**. Indicates the granularity of the returned clones: free (i.e. no syntactic boundaries), fixed (i.e. within predefined syntactic boundaries such as method or block), or both. Both granularities have advantages and disadvantages. We think that no option is particularly better than the other, and therefore do not consider this facet relevant in our selection process.
- **Clone type**. As motivated in Section 3.3, Type-2 and specially Type-3 clones are particularly interesting for the identification of generation candidates. Therefore, both of the following attribute values are recommended:
  - CT.b - Type-2 or subset of Type-2
  - CT.c - Type-3 (near-miss) or subset of Type-3

Technical

- **Comparison algorithms**. As long as the rest of requirements are met, it is not relevant at all for us which algorithm is used by the tool to detect the clones.
- **Comparison granularity**. This is a characteristic of the clone detection algorithm, which is not relevant to us as long as the rest of requirements are met by the tool.
- **Worstcase computational complexity**. Although it is not a requirement, we certainly prefer efficient implementations. Consequently, a linear time is recommended over polynomial complexities:
  - CC.a - Linear

Adjustment

- **Pre-/post-processing**. The specific pre- or post-processing actions implemented are not relevant as long as the tool and its results satisfy the rest of requirements.
- **Heuristics/thresholds**. The use of threshold values to allow the user to partially customize the functioning of the tool is always recommended. In general, the more customizable parameters the better, although we particularly recommend one specific threshold value:
  - H.b - On code similarity

Processing

- **Basic transformation/normalization**. We take into account the clone types that the tool is able to identify. The normalization performed by the tool to reach such an identification is not relevant.

- **Code representation**. The internal code representation is another internal detail that is not relevant to our purposes.
- **Program analysis**. Whether the tool works directly on source code or requires, for example, a lexical analysis, is an internal characteristic not relevant to our purposes.

Evaluation

- **Empirical validation**. Although it is not of great importance, some type of validation of the tool is always recommended:
  - E.a - Yes, validated empirically in terms of precision, recall, memory and time and compared with other tools
  - E.b - Validated enough in support of the claim
  - E.c - Validated by other means or third party comparison study
- **Availability of empirical results**. We do not intend to replicate or formally compare the results, so their availability is not relevant at all to our purposes.
- **Subject systems**. The systems that have been used in the validation are not relevant to our purposes.

Table 5.2 on page 80 summarizes the required and recommended attribute values for each facet. Once the requirements are applied, most tools are discarded. Table 5.3 on page 81 shows the remaining clone detection tools, along with their attribute values for each facet. We have highlighted in green those values that meet the recommendations, and in red those which do not. All the tools satisfy the requirements, so those facets are not highlighted. We refer again to [RCK09] for the definition of each attribute value.

SDD, Simian and CPD are the most limited tools. They do not satisfy most of the recommendations. Furthermore, they only identify two types of clones. SDD misses type 2 and 4 clones, whereas Simian and CPD miss type 3 and 4 clones, which is an important shortcoming. Out of the other three tools, CCFinderX stands out as the most convenient one, meeting most of the recommended attribute values. CloneDigger and SimScan also perform good enough, satisfying some of the recommendations and identifying type 1, 2 and 3 clones.

However, there are other important factors to take into account in the selection process. As a next step, we have searched for the current version of each tool in order to: a) verify their availability, b) find differences in relation to the data collected in the paper and c) check the feasibility of their integration with our plugin or environment:

- **SDD** [LJ05]. The homepage of the project[1] is not reachable[2].
- **Simian** [Sim16a]. Both a .NET and a Java implementation are provided. As a standalone tool, it can generate an output in different formats, including XML. A Java API is also provided to launch the procedure and process the results from Java code, which makes its integration into our plugin considerably easier.

[1] `http://comedu.korea.ac.kr:8080/sddwiki/FrontPage`
[2] Accessed July 17, 2016

- **CCFinderX** [AIS16, KKI02]. A standalone tool is provided, with versions for Windows and Linux. The Windows version is only for 32-bit machines, but the installer did not work on our 32-bit Windows 10. Furthermore, the tool is implemented in C++, which hinders its integration with our Java Eclipse plugin.
- **CPD** [PMD16]. CPD is part of PMD, a source code analyzer. PMD is provided as a standalone tool as well as a plugin for multiple IDEs, including Eclipse. It is able to generate the results in various formats, including XML and CVS.
- **CloneDigger** [BM08]. A Python implementation is provided, as well as an Eclipse plugin. The integration of Python scripts in our Java plugin is not straightforward. Using the `Runtime` or `Process` classes from Java only works if an implementation of Python is available in the machine. To avoid such a dependency, a project like Jython [Fou16] can be used to embed the Python code in our Java code, but running a complex project like CloneDigger with it is not trivial. The Eclipse plugin requires Eclipse 3.4, according to the documentation. However, we obtain an error when installing both the 1.1.8 and 1.0.8 versions of the plugin, the ones available in the provided update site[3].
- **SimScan** [Sim16b]. The official website of the tool[4] is not reachable[5]. Only an Eclipse plugin has been found, CloneTracker [McG16], but it requires the SimScan libraries that are hosted on the official site.

Unfortunately, half of the tools turn out to either be unavailable at the time they were accessed, or not installable in our machine. As a result, only three tools have been eventually tested. CloneDigger is the one whose specification better suits our recommendations, as shown in Table 5.3. However, the Eclipse plugin does not work and the integration of its Python version is not straightforward. Both CPD and Simian, which share similar characteristics, can be used instead. We have decided to select Simian because it can be more easily integrated into our Java plugin. Integration into our tool is preferred over the use of an additional Eclipse plugin. If clone detection is integrated into our plugin, the access to all the features is uniform and all the results are reported in the same format, which makes our implementation more usable. Additionally, with this approach the results can be treated in another parts of our tool, allowing the combination of clone detection with other features.

5.2.2 Tool integration

The diagram in Figure 5.5 on page 82 depicts the structure of classes that has been developed to implement the clone detection feature described in Section 3.3. A `CloneDetector` interface has been defined, with a method to detect clones in a list of files. The method returns a `CloneDetectionResult` object with the sets of clones that have been detected. Each `CloneSet` contains a list of blocks with the specific portions of code in each file that have been detected as clones. Every clone detector must implement this interface. To launch the process, a `CloneDetectionProcess` object is required, which is configured with a specific clone detector. We have provided an implementation for the Simian tool.

[3] `http://clonedigger.svn.sourceforge.net/svnroot/clonedigger/trunk/org.clonedigger.feature/`

[4] `http://www.blue-edge.bg/simscan/`

[5] Accessed July 17, 2016

Category	**Facet**	**Attribute value**	**Type**
Usage	Platform	a / c / d	Required
		a	Recommended
	External dependencies	a	Recommended
	Availability	a / b	Required
Interaction	User interface	-	-
	Nature of output	a / c	Recommended
	IDE support	a / c	Required
Language	Language paradigm	b / c	Required
	Language support	a / e	Required
Clone	Clone relation	d	Recommended
	Clone granularity	-	-
	Clone type	b & c	Recommended
Technical	Comparison algorithm	-	-
	Comparison granularity	-	-
	Worstcase computational complexity	a	Recommended
Adjustment	Pre-/post-processing	-	-
	Heuristics/thresholds	b	Recommended
Processing	Basic transformation/normalization	-	-
	Code representation	-	-
	Program analysis	-	-
Evaluation	Empirical validation	a / b / c	Recommended
	Availability of empirical results	-	-
	Subject systems	-	-

Table 5.2: List of required and recommended attribute values for each facet.

Facet/Tool	SDD	Simian	CCFinderX	CPD	CloneDigger	SimScan
Platform	a	d*	d	a	a	d
External dependencies	a	a	a	b	b	b
Availability	a	b	b	a	a	b
Interaction	b	a	c	c	a	c
Nature of output	b	a	c	c	c	c
IDE support	a	c	c	c	a	ab
Language paradigm	c	cd	c	c	b	b
Language support	abe	a	bcef	bce	eg	e
Clone relation	a	b	ad	a	a	ad
Clone granularity	a	a	a	a	a	a
Clone type	ac	ab	abc	ab	abc	abc
Comparison algorithm	o	q	a	q	a	g
Comparison granularity	b	a	d	d	f	f
Worstcase computational complexity	a	d	a	d	b	d
Pre-/post-processing	a	c	b	c	c	c
Heuristics/thresholds	ac	e	abd	c	b	b
Basic transformation/normalization	a	e	eh	g	i	i
Code representation	b	f	f	p	l	j
Program analysis	c	a	dg	d	e	e
Empirical validation	d	d	bc	d	d	d
Availability of empirical results	b	b	b	b	a	b
Subject systems	be	b	abf	b	g	b

*Simian is actually platform independent in its current version.

Table 5.3: List of tools that fulfill the requirements.

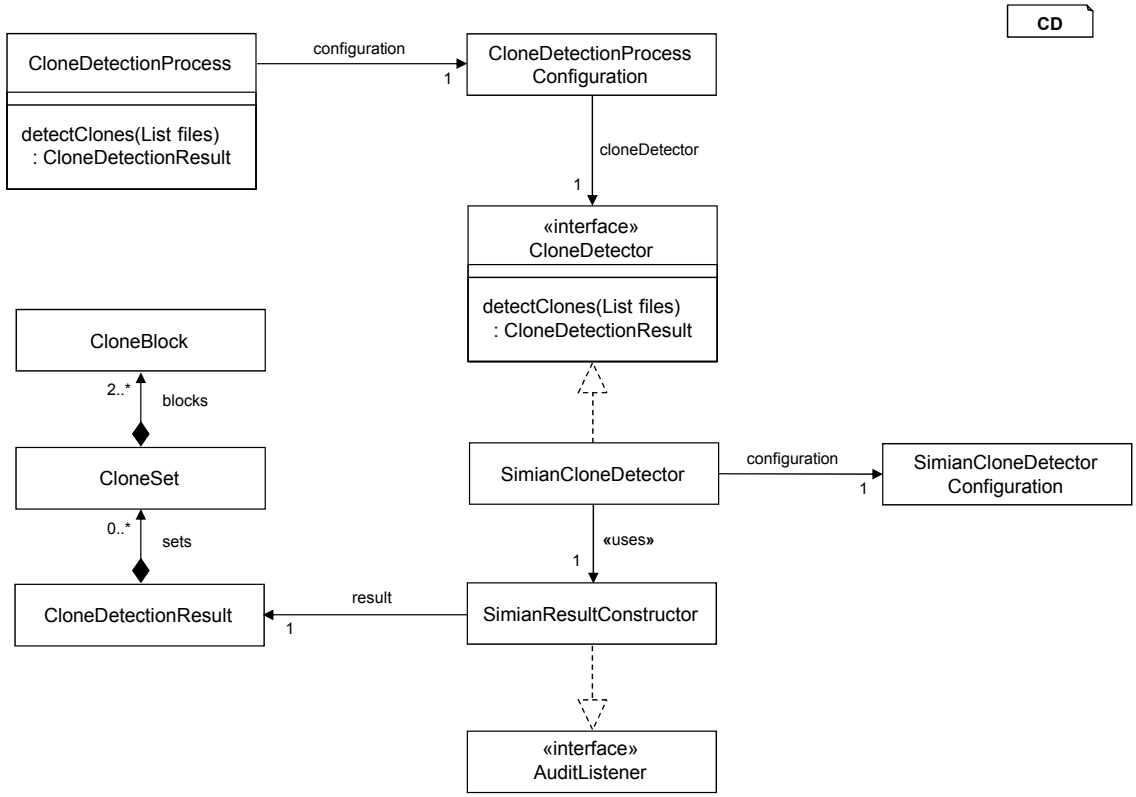


Figure 5.5: Class diagram for the implementation of the clone detection feature.

Our Simian clone detector makes use of the Simian Java API. Listing 5.1 shows how to use the API to launch the clone detection process. `Checker` is the class that manages the whole process. A class `Options` is provided to handle the configuration values. We wrap up these options in our own `SimianCloneDetectorConfiguration` class, which offers only those configuration values that are interesting for us, and internally manages the corresponding `Options` object. To load the files, we use the `StreamLoader` and `FileLoader` classes in the API. Finally, an interface `AuditListener` is provided by the API to process the results. This interface defines methods that are called during the clone detection process every time that, for example, a clone set or a clone block is detected. Our implementation of the interface, `SimianResultConstructor`, uses these methods to incrementally build a `CloneDetectionResult` object. Once everything has been set up, the process is launched by invoking the `check` method on our `Checker` object. The result with the information about the detected clones is finally obtained from the constructor and returned.

```
1 @Override
2  public CloneDetectionResult detectClones(List<String> files) {
3   SimianResultConstructor resultConstructor = new
        SimianResultConstructor();
4   Checker checker = new Checker(resultConstructor, configuration.
        getOptions());
5   StreamLoader streamLoader = new StreamLoader(checker);
6   FileLoader fileLoader = new FileLoader(streamLoader);
7   for (String file : files) {
8    try {
```

```
9       fileLoader.load(file);
10     } catch (IOException e) {
11      e.printStackTrace();
12     }
13    }
14    checker.check();
15
16    return resultConstructor.getResult();
17   }
```

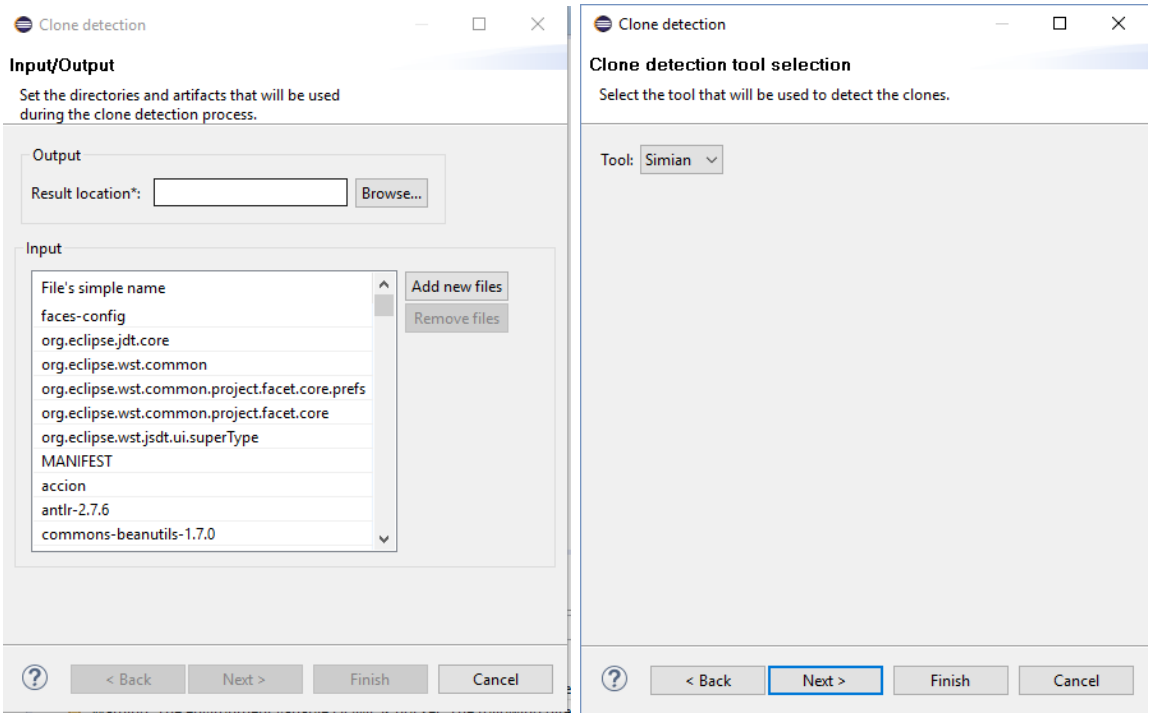
Listing 5.1: Usage of the Simian API to launch a clone detection process.

5.2.3 Usage

The clone detection feature can be applied on any group of files. If applied on containers, such as packages, folders or projects, each container is recursively searched and all the contained files are taken as input. Figure 5.6 shows the pages forming the configuration wizard for the feature. It contains three pages:

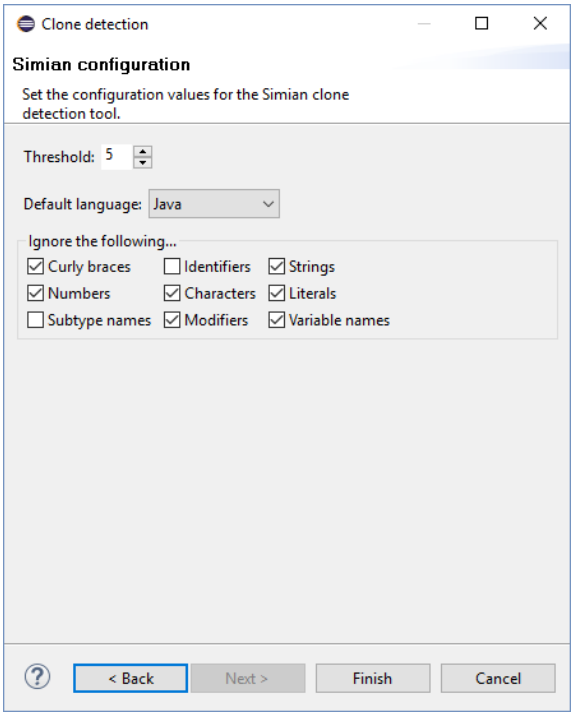
- **Input/output**. The initial user's selection can be modified, and a path must be specified to place the results.
- **Tool selection**. A tool must be selected from the available implementations to perform the actual clone detection process. Only the Simian implementation is currently provided.
- **Tool configuration**. This page is specific to the tool selected in the previous step. For Simian, different configuration options are provided which are a subset of the total set of options available in the tool. We have selected the ones that we consider most relevant, including:
  - The threshold value, i.e. the minimum number of lines that must be found equal between two files to report them as clones.
  - The language.
  - The possibility to ignore elements such as modifiers, identifiers or literals, among others.

A group of HTML files is generated to show the results of the process. An index file is created as seen in Figure 5.7 on page 85. It lists in a sortable table all the clone sets that have been identified, along with their basic attributes. Each row in the table links to a different HTML file containing more detailed information about the corresponding clone set, as seen in Figure 5.8 on page 86. This information includes the clone blocks contained in the set as well as the actual portion of code that is shared between the blocks.



(a) Input and output

(b) Tool selection



(c) Tool configuration

Figure 5.6: Configuration steps for the clone detection feature.

Clone detection result

Clone sets

Show 10 entries

Search:

Name	Number of blocks	Line count	Similarity
Clone set 1	2	18	-
Clone set 2	2	21	-
Clone set 3	2	17	-
Clone set 4	3	12	-
Clone set 5	3	13	-
Clone set 6	2	27	-
Clone set 7	2	13	-
Clone set 8	3	28	-
Clone set 9	2	14	-
Clone set 10	3	25	-

Showing 1 to 10 of 79 entries

Previous 1 2 3 4 5 ... 8 Next

Figure 5.7: HTML index file generated as part of the results report.

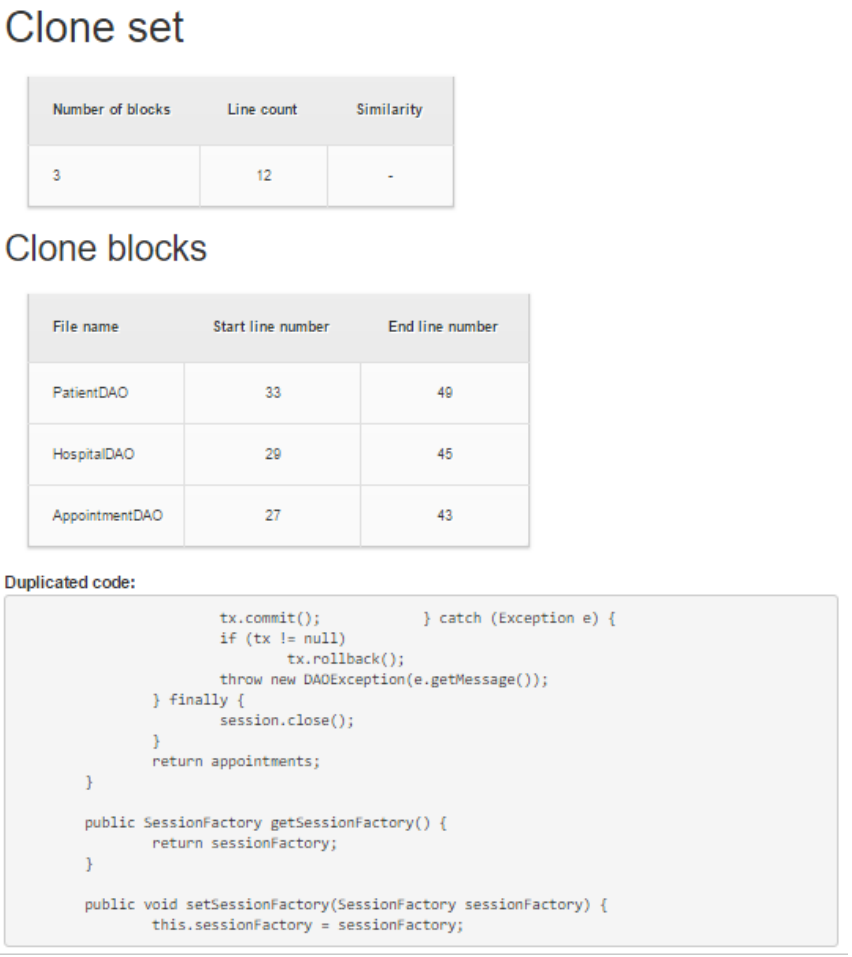
5.3 Custom clone detection

5.3.1 Core implementation

The diagram in Figure 5.9 on page 87 depicts the structure of classes that has been developed to implement the custom clone detection feature described in Section 3.4. An important distinction is made between the global clone detection process and the similarity calculation algorithm. The process refers to the application of the feature on two or more code units, and includes the execution of the four pre-processing steps and the calculation of the final result. The algorithm, however, only deals with the similarity calculation between two code units, and is used by the process to obtain a similarity measure between two specific code units for a specific pre-processing step. The interfaces `ICustomCloneDetectionProcess` and `ISimilarityCalculationAlgorithm` are defined respectively to represent these concepts, and an implementation of both interfaces is provided.

Custom clone detection process

An implementation of the process interface is provided, with methods to calculate the global similarity between two or more code units. The code unit concept has been implemented to represent the pieces of code that are compared, and the Java `IType` class has been selected to represent the optional domain classes. The custom clone detection process takes each pair of input code units and launches the algorithm for each specified step. Finally, a global result is built containing every individual step result. Additionally,



Clone set

Number of blocks	Line count	Similarity
3	12	-

Clone blocks

File name	Start line number	End line number
PatientDAO	33	49
HospitalDAO	29	45
AppointmentDAO	27	43

Duplicated code:

```
                    tx.commit();            } catch (Exception e) {
                    if (tx != null)
                            tx.rollback();
                    throw new DAOException(e.getMessage());
            } finally {
                    session.close();
            }
            return appointments;
    }

    public SessionFactory getSessionFactory() {
            return sessionFactory;
    }

    public void setSessionFactory(SessionFactory sessionFactory) {
            this.sessionFactory = sessionFactory;
```

Figure 5.8: HTML file with the information about one clone set, generated as part of the results report.

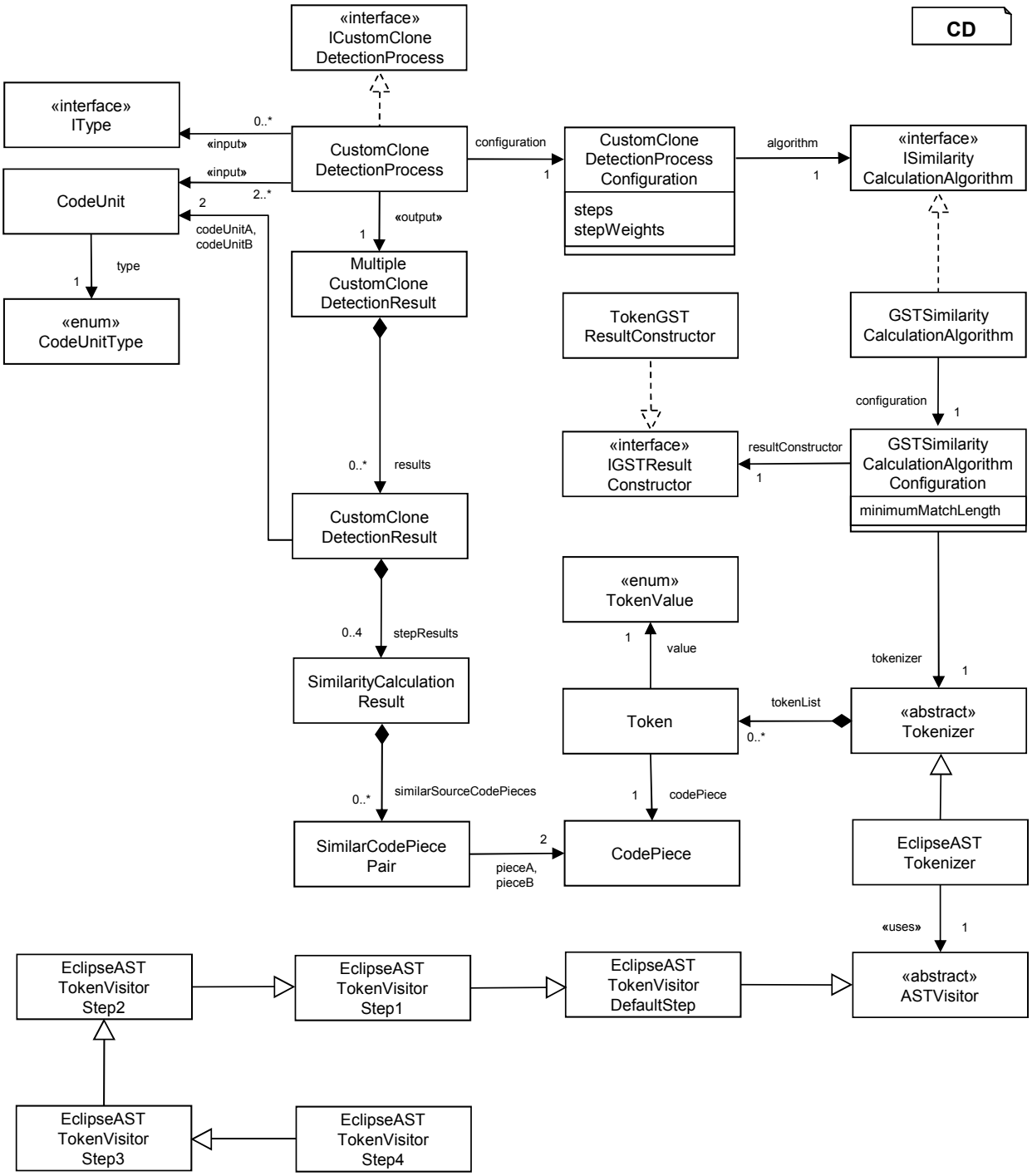


Figure 5.9: Class diagram for the implementation of the custom code similarity feature.

a final weighted similarity measure is calculated from the individual results, according to the weights configured for each step.

Similarity calculation algorithm

In order to perform the string comparison we have selected the Greedy String Tiling (GST) [Wis93] algorithm. This algorithm tries to find sequences of equal elements in two strings represented as lists of tokens. Some definitions are necessary in order to understand how the algorithm works:

- A **maximal-match** is when a subsequence of a list of tokens matches, element by element, a subsequence of the other list of tokens. The match is assumed to be as long as possible. Maximal-matches are temporary and possibly not unique associations, i.e. a subsequence involved in one maximal-match may form part of several other maximal-matches.
- A **tile** is a permanent and unique association of a subsequence of one token list with a matching subsequence of the other list. In the process of forming a tile from a maximal-match, tokens of the two lists are **marked**, and thereby become unavailable for further matches.
- A **minimum match length** is defined such that maximal-matches (and hence tiles) below this length are ignored. The minimum match length can be as low as 2, but in general will be an integer greater than that. A higher value means that only big subsequences are considered. This increases the efficiency of the algorithm, but also misses some matches that can be significant for the final result. Therefore, a compromise needs to be reached. We decided to use a default minimum match length of 6.

The algorithm involves multiple passes, each pass having two phases. In the first phase, all maximal-matches are collected. Both token lists are iterated, and the unmarked tokens are compared one by one in order to find as big as possible equal subsequences. In the second phase, maximal-matches are taken, one at a time starting with the longest, and tested to see whether they have been occluded by a sibling tile (i.e. part of the maximal-match is already marked). If not, a tile is created by marking the tokens forming the two subsequences. When all the maximal-matches have been dealt with, the algorithm proceeds with the following pass. Maximal-matches are searched again among the unmarked tokens, until no match is found greater than the minimum match length. Once the algorithm is finished, we get a collection of tiles with the subsequences that are equal in both token lists. The number and length of these tiles determine the degree of similarity between the two lists, and therefore between the two strings that the lists represent. We refer to [Wis93] for a more detailed description of the GST algorithm which includes the corresponding pseudo-code.

In order to apply the GST algorithm to a couple of code units, we make use of a tokenizer to divide each code unit into tokens, and find tiles of equal tokens between both token lists. An abstract tokenizer is defined and different implementations can be developed and integrated into the tool. Our tokenization process uses the Eclipse's Abstract Syntax Tree (AST) libraries. A list of tokens is defined with all the symbols used in the Java language, and a special token to represent a custom string. The construction of the token list is

performed by a structure of AST visitors, which invoke a `visit` method for each element in the AST.

A basic visitor is implemented to obtain an initial list of tokens from the code, that is a direct tokenization of it. No pre-processing actions are performed, but instead the code is simply divided into its forming syntactical elements, by creating and adding to the list the corresponding tokens in each `visit` method. Listing 5.2 shows the basic `visit` method for a method invocation. Using the formal definition of this expression in Java [Ecl16a] as a reference, we recursively visit the contained elements and generate the necessary tokens for the syntactic elements.

```
1  /**
2   * MethodInvocation: [ Expression . ] [ < Type { , Type } > ]
3   *   Identifier ( [ * Expression { , Expression } ] )
4   */
5  @Override
6  public boolean visit(MethodInvocation node) {
7   if (node.getExpression() != null) {
8    node.getExpression().accept(this);
9    addToken(TokenValue.DOT);
10  }
11  if (!node.typeArguments().isEmpty()) {
12   addToken(TokenValue.LESS_THAN);
13   visitList(node.typeArguments(), TokenValue.COMMA);
14   addToken(TokenValue.GREATER_THAN);
15  }
16  node.getName().accept(this);
17  addToken(TokenValue.OPENING_PARENTHESIS);
18  visitList(node.arguments(), TokenValue.COMMA);
19  addToken(TokenValue.CLOSING_PARENTHESIS);
20
21
22  return false;
23 }
```

Listing 5.2: `visit` method for a `MethodInvocation` in the basic visitor.

Additional visitors are then defined for each step, including the pre-processing actions directly in the tokenization process. The method invocation expression, for example, is processed in different ways depending on the step. Listing 5.3 shows the corresponding `visit` method overridden for the first pre-processing step. Since one of the pre-processing actions in this step is the removal of method arguments, the behavior in the basic visitor is kept and only the part where the arguments are visited is eliminated (line 19).

```
1  /**
2   * Remove method arguments.
3   */
4  @Override
5  public boolean visit(MethodInvocation node) {
6   // Implement the default behavior but without pre-processing the
7   // arguments.
8   if (node.getExpression() != null) {
9    node.getExpression().accept(this);
10   addToken(TokenValue.DOT);
11  }
```

```
12  if (!node.typeArguments().isEmpty()) {
13   addToken(TokenValue.LESS_THAN);
14   visitList(node.typeArguments(), TokenValue.COMMA);
15   addToken(TokenValue.GREATER_THAN);
16  }
17  node.getName().accept(this);
18  addToken(TokenValue.OPENING_PARENTHESIS);
19  // visitList(node.arguments(), ",");
20  addToken(TokenValue.CLOSING_PARENTHESIS);
21
22  return false;
23 }
```

Listing 5.3: `visit` method for a `MethodInvocation` in the step 1 visitor.

Finally, listing 5.4 shows the implementation of the same method for the step 4 visitor. In this step, a more aggressive pre-processing action must be implemented to replace all method invocations with a unique identifier. Defining this behavior using the visitor approach is as simple as overriding the corresponding `visit` method and replacing the previous actions with the insertion of a token. All the pre-processing actions presented in the theoretical description of this feature have been implemented in a similar way.

```
1 /**
2  * Replace method invocations with a unique identifier.
3  */
4 @Override
5 public boolean visit(MethodInvocation node) {
6  addToken(node, TokenValue.METHOD_INVOCATION);
7  return false;
8 }
```

Listing 5.4: Visit method for a MethodInvocation in the step 4 visitor.

The visitor classes are defined in an inheritance structure, so that every visitor inherits from the previous one. In this way, a visitor corresponding to one specific step will only need to override the `visit` methods necessary to implement its pre-processing actions, and ignore the other methods, which will be handled by the previous step. This goes on until reaching the initial visitor, where the code is left unmodified and the original tokens are added to the list. This inheritance structure simulates the scheme shown in Figure 3.6, where the steps are represented such that the input of each one is the output of the previous one. It is, however, not the same behavior, since all our visitors work with the original AST and not with the tokens generated by the previous visitor. This structure simplifies the implementation, since all the steps can be implemented as AST visitors and make use of the AST entities to implement the actions, instead of dealing with a list of custom tokens as input. Nevertheless, the inheritance approach also makes the process redundant, since for the execution of a given step, methods of the previous steps may have to be executed once again.

However, in practice the main bottleneck of the process is the GST algorithm, and not the tokenization part. The definition of four different pre-processing steps together with the fine-grained tokenization performed by the initial visitor can lead to efficiency problems when dealing with large code units. Big token lists are generated, and the execution time grows exponentially with the code size, notably slowing down if two large code units

are provided. This is specially problematic for the first steps, because less substitutions and simplifications are made on the code, and the generated lists of tokens are bigger. One solution could be to implement more aggressive pre-processing actions in the first steps. Alternatively, a more efficient version of the GST algorithm can be implemented, like the Running Karp-Rabin Matching version [Wis93]. Nevertheless, the identification of generation candidates is not a process meant to be executed on a regular basis, and therefore efficiency is not a priority as long as time and resource costs do not exceed a practical threshold.

Once the corresponding visitor, depending on the step, has been executed for each code unit, two lists of tokens are obtained and the GST algorithm runs on them, returning a set of tiles with the identical sequences of tokens in both lists. A `SimilarityCalculationResult` is created with the total similarity measure of the step, the corresponding percentages in relation to the total size of both code units and, additionally, the length of the biggest tile. Information about the position of each tile in the source code unit can also be provided through the `SimilarCodePiecePair` entity. However, with our AST tokenization approach it is not trivial to extract such information for some kinds of tokens. Therefore, this behavior is currently only partially supported and should not be relied on.

Since the algorithm works with tokens, we can use them as our unit of measure when reporting the results. Both the similarity measure and the percentages refer to the number of tokens in the tiles and in the token lists. However, this approach is not the only option. Since not all tokens represent the same number of characters in the source code, it could be argued that using them as the unit of measure is not the most accurate method. An alternative could be to use characters as the unit of measure and take into account the character length of each token when calculating the similarity measure. Because of the reasonable interest in defining different ways of calculating the result, the `IGSTResultConstructor` interface is provided, which can be implemented with alternative ways of building the similarity result.

5.3.2 Usage

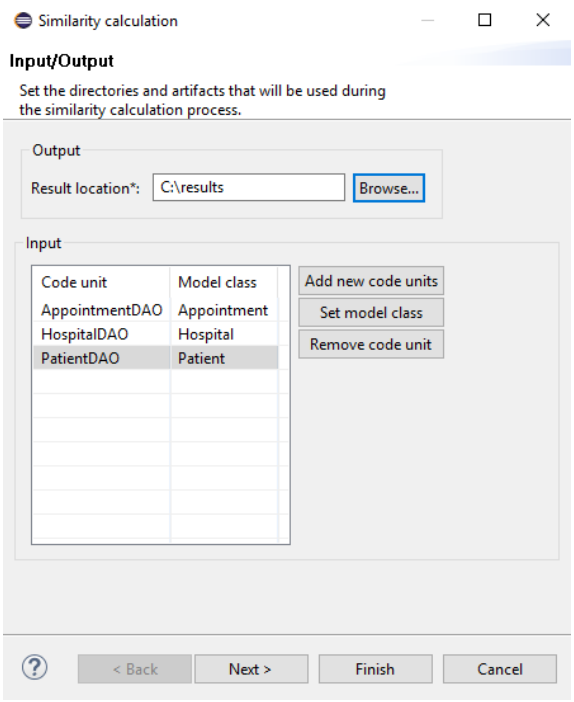
Our custom clone detection feature can be applied in two different ways:

- On a set of two or more Java classes. The classes are paired and the process is performed on each possible pair.
- On a single Java class, to be compared with a set of other classes. The selected class is paired to each of the rest.

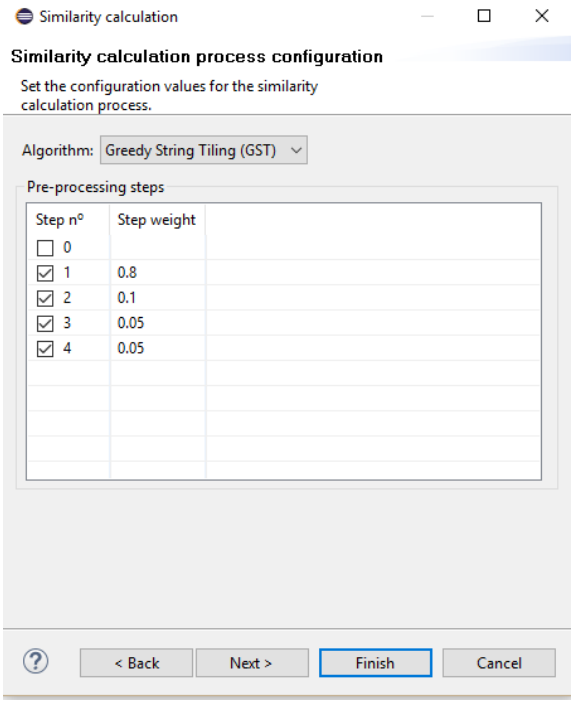
Similarity calculation within a class, which was an additional application possibility described in Section 3.4, is not currently supported by the tool.

Figure 5.10 shows the configuration wizard for the custom clone detection feature. It contains three pages:

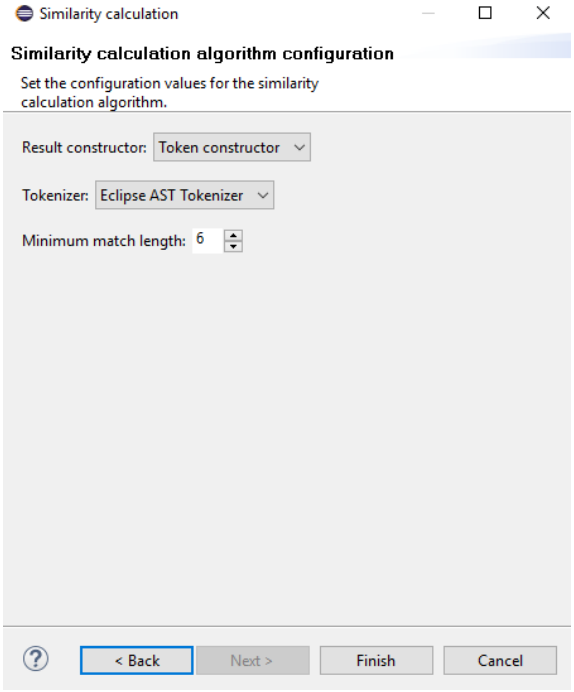
- **Input/output**. The set of classes to be compared can be modified, and the related domain classes specified. The page shown in the wizard corresponds to the application of the feature to a set of classes, but applying it to a single class results in a very similar input/output wizard page. Finally, the output path where the results report are generated must also be specified.



(a) Input and output

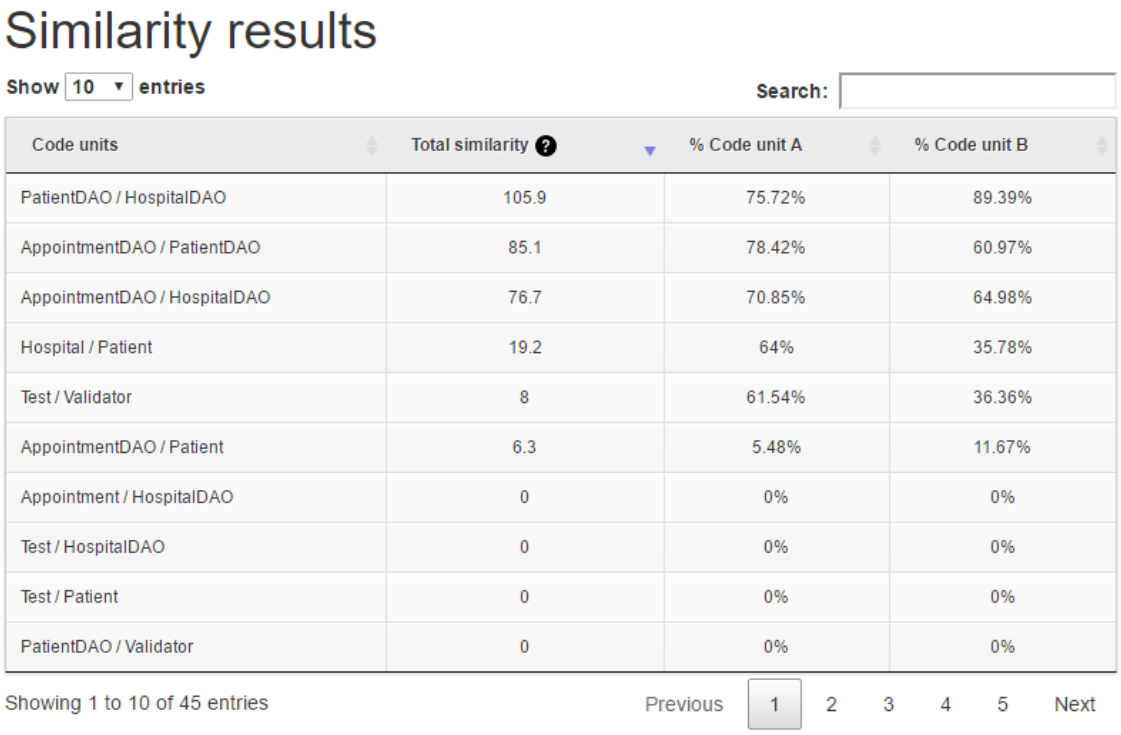


(b) Process configuration



(c) Algorithm configuration

Figure 5.10: Configuration steps for the custom code similarity feature.



Similarity results

Show 10 entries

Search:

Code units	Total similarity	% Code unit A	% Code unit B
PatientDAO / HospitalDAO	105.9	75.72%	89.39%
AppointmentDAO / PatientDAO	85.1	78.42%	60.97%
AppointmentDAO / HospitalDAO	76.7	70.85%	64.98%
Hospital / Patient	19.2	64%	35.78%
Test / Validator	8	61.54%	36.36%
AppointmentDAO / Patient	6.3	5.48%	11.67%
Appointment / HospitalDAO	0	0%	0%
Test / HospitalDAO	0	0%	0%
Test / Patient	0	0%	0%
PatientDAO / Validator	0	0%	0%

Showing 1 to 10 of 45 entries

Previous 1 2 3 4 5 Next

Figure 5.11: HTML index file generated as part of the results report.

- **Process configuration**. This page deals with the information regarding the overall clone detection process. It allows to specify the algorithm that we want to use and the information regarding the pre-processing steps. The user can select which steps will be executed and assign them a specific weight, that is used to calculate the final similarity measure.

  A step 0 is included, which corresponds to the initial state where both code units are kept unmodified. If using the GST algorithm with the AST Tokenizer, in this step the initial lists of tokens of both code units are compared. Similarity calculation at this step can be useful to have a basic measure that we can use to compare the results with those of later steps. However, since the original code is used in the comparison, the process can be particularly costly, and in most occasions its execution should not be considered.

- **Algorithm configuration**. Once the algorithm has been selected, an additional configuration page is shown with the different customization options for that algorithm. In the figure, we can see the wizard page for the GST algorithm. Both the result constructor and the tokenizer can be specified here. Currently, only the token constructor and the Eclipse AST tokenizer are available, but new alternatives could be implemented such as a tokenizer that can handle different languages or files so that similarity calculation is not only applicable to Java classes. Additionally, an integer for the minimum match length can be specified, which is one of the parameters that the algorithm works with.

A group of HTML files is generated to show the results of the process. An index file is created as seen in Figure 5.11. It lists in a sortable table all the pairs of files that

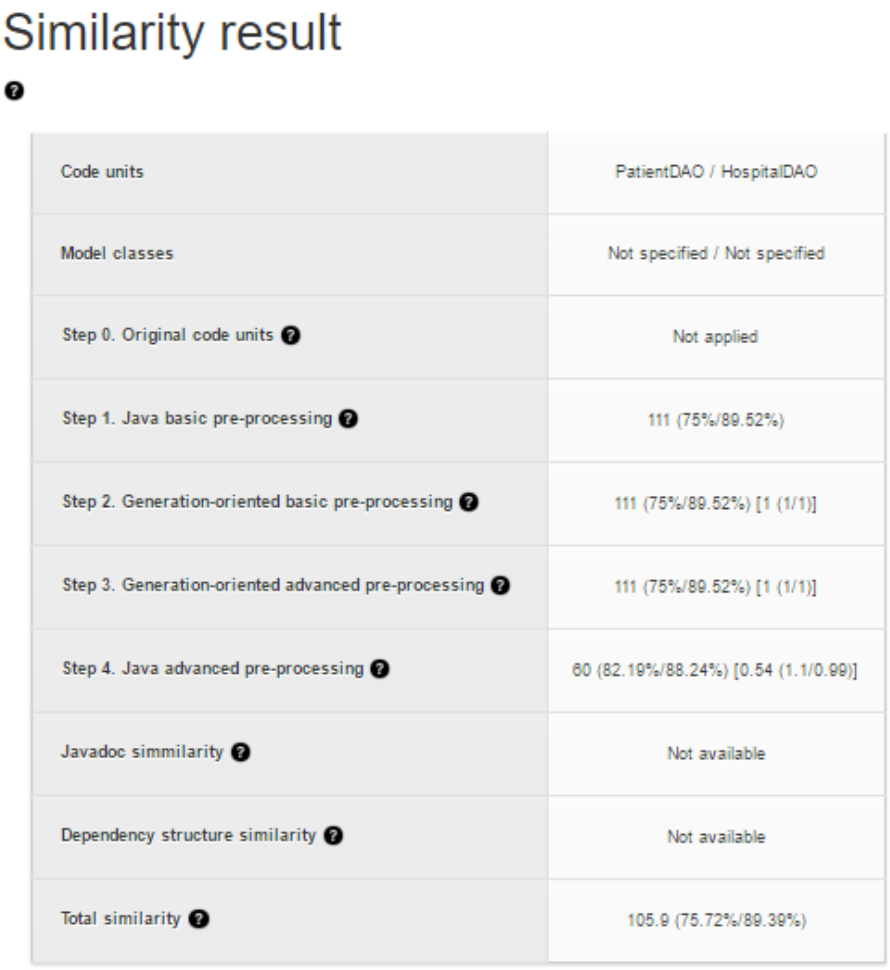


Figure 5.12: HTML file with the information about one specific comparison, generated as part of the results report.

have been compared, along with the total and percentage similarity results. Each row in the table links to a different HTML file containing more detailed information about the corresponding comparison, as seen in Figure 5.12 on page 94. This information includes the individual similarity measure after each pre-processing step, which is crucial for a meaningful interpretation of the results.

5.4 Design patterns

5.4.1 Tool selection

The identification of design patterns was motivated and identified as one of the features for detecting generation candidates in Section 3.6. Since there are different pattern detection tools already available, we decided to use one of them for the implementation of this feature. We have carried out an informal tool selection process, guided by three factors:

- The tool must be open source or freely available.
- The tool must be able to process Java code. This language is the main focus of our feature implementations.
- The tool must be easy to integrate into our environment (Eclipse) or into our plugin.

We have focused on four design pattern detection tools found in the literature:

- PINOT [SO06]. PINOT (Pattern INference recOvery Tool) is a fully automated pattern detection tool. It recognizes all the GoF patterns in the structure and behavior-driven categories. PINOT is built from Jikes, an open source Java compiler written in C++. It is therefore implemented in C++, but analyzes Java code.
- WebOfPatterns (WOP) [DE05]. The WebOfPatterns (WOP) is a toolkit that facilitates the sharing of knowledge about software design. WOP consists of two parts: a language-neutral format to describe design patterns based on the W3C standards RDF [LS99] and OWL [SWM04], and a set of Eclipse plugins which use this format. WOP includes a pattern scanner that can be used to find pattern instances in Eclipse Java projects.
- Reclipse [vDMT10]. Reclipse is a reverse engineering tool suite based on Fujaba[6]. It provides static and dynamic design pattern detection in combination with a pattern rating that is used to evaluate the quality of the results. It is provided as an Eclipse plugin and works with Java code.
- Design Pattern detection using Similarity Scoring [TCSH06] (in the following referred to as DPSS). Tsantalis et al. offer a simple Java application to detect design patterns using a technique called similarity scoring. The tool also allows the definition of new patterns.

[6] `http://www.fujaba.de/`

The four tools are freely available and can process Java code. Both PINOT and DPSS are independent tools, and therefore do not satisfy our third requirement. Neither WOP nor Reclipse can be easily integrated into our tool either. However, since they are provided as Eclipse plugins they can be integrated into our environment. Nevertheless, we had problems with their installation and execution. We were not able to successfully install Reclipse in our Eclipse environment. This plugin uses Eclipse's modeling tools and requires the installation of the plugins MoDisco SDK Incubation, Palladio and SoMoX. Nevertheless, even after adding these dependencies the installation of the plugin failed. We were not able to solve the problem, partly because of the lack of documentation. As for WOP, the plugin was successfully installed and tested on some simple projects in our Eclipse workspace. However, it failed when applied on the bigger Java projects that are used for our case study (see Section 6).

We decide to select DPSS to implement our feature. Despite being an independent tool, it is really easy to use. It does not require any installation and needs only a Java Virtual Machine to be executed. Furthermore, it could be successfully run on every Java project that we tried, including those defined as part of our case study. This was the main reason behind the choice of DPSS as our design pattern detection tool.

The selection process has been kept intentionally simple, since we just need a tool that can act as a proof of concept of the ideas presented in the description of the feature. However, a more complete and formal selection process could be carried out, taking into account a wider variety of tools and evaluating additional factors such as the design patterns supported or the accuracy of the detection. Finally, multiple tools could be used instead of one. This way, we could support the detection of a wider range of design patterns.

5.4.2 Usage

DPSS is a very simple tool. The design pattern detection process can be executed in three simple steps:

1. Selection of the project. A *Open Directory* option is available in the *File* menu to select the root of the project. Only class files (bytecode files) are necessary for the project analysis.
2. Selection of the patterns. Through the *Patterns* menu, we can individually look for a specific pattern or search for every supported pattern at once.
3. Export of the results. Once executed, the results are shown in the tool. However, we can also export the results as an XML file through the *Export as XML* option in the *File* menu.

No more functionality is provided. However, the tool offers everything necessary, is very simple to use and works with every Java project.

The following design patterns are supported by the tool by default:

- Creational patterns
  - Factory Method

  - Prototype
  - Singleton
- Structural patterns
  - Adapter
  - Bridge
  - Composite
  - Decorator
  - Proxy
  - Proxy2[7]
- Behavioral patterns
  - Observer
  - State
  - Strategy
  - Command
  - Template Method
  - Chain of Responsibility
  - Visitor

This set includes 16 out of the 23 GoF patterns. Custom design pattern definitions can also be added[8].

Since we have not implemented our own approach but used an independent tool instead, there is no support for the measures that we defined in Section 3.6. The generatability value of each design pattern is not considered when reporting the identified instances. Therefore, it must be incorporated in the manual analysis of the results. For example, instances corresponding to patterns with a high generatability value can be given a higher priority. This can be particularly useful when dealing with a large code base. If many instances are detected, we can use the generatability values to focus first on the most interesting design patterns.

[7]The design pattern reported by the tool as Proxy2 is a variation of the Proxy pattern. This variation is actually structurally more similar to the Decorator pattern than to the Proxy pattern. For more details, see `http://users.encs.concordia.ca/~nikolaos/pattern_detection.html`.

[8]`http://users.encs.concordia.ca/~nikolaos/files/pattern_detection/DPD_extension.pdf`

Chapter 6

Case study

In this chapter, we evaluate the ideas described in the thesis by applying the implemented features on a real-world system. Section 6.1 describes the subject system of this case study. The research questions are presented in Section 6.2. Section 6.3 explains how the features are applied in order to answer the research questions, while Section 6.4 discusses the obtained results. We end by examining the threats to validity in Section 6.5.

6.1 Subject system

The case study has been applied to software systems of the company DSA[1]. DSA is one of the leading manufacturers of hardware and software systems for electrical and electronic testing of vehicle components. The software systems in DSA use Eclipse as the development environment. Each system is composed of different Java projects, which define Eclipse plugins with the desired functionality.

The DSA systems already define a generative architecture. A domain model and a set of templates have been developed to generate some of the artifacts in the systems. In order to integrate the generated and handwritten code, an adapted version of the extended generation gap mechanism [GHK+15] is used. Figure 6.1 illustrates this adapted mechanism. For every class `Foo` in the domain model, two artifacts `FooBase` and `FooBaseImpl` are automatically created. `FooBase` contains the functionality that is expected from the class, while `FooBaseImpl` provides an implementation of this functionality. In this way, we are programming to interfaces, allowing for the modification or exchange of the implementations while minimizing the consequences in the rest of the system. `FooBase` and `FooBaseImpl` are both automatically generated. In order to allow for the extension of the generated code, `Foo` and `FooImpl` are manually implemented, as inheriting from `FooBase` and `FooBaseImpl` respectively. This way, the generated implementation can be extended by adding new handwritten code to the inheriting artifacts. By keeping this separation between generated and handwritten code, we can modify the generative architecture and generate the artifacts again without losing the manually added implementation.

[1] http://www.dsa.de/en/

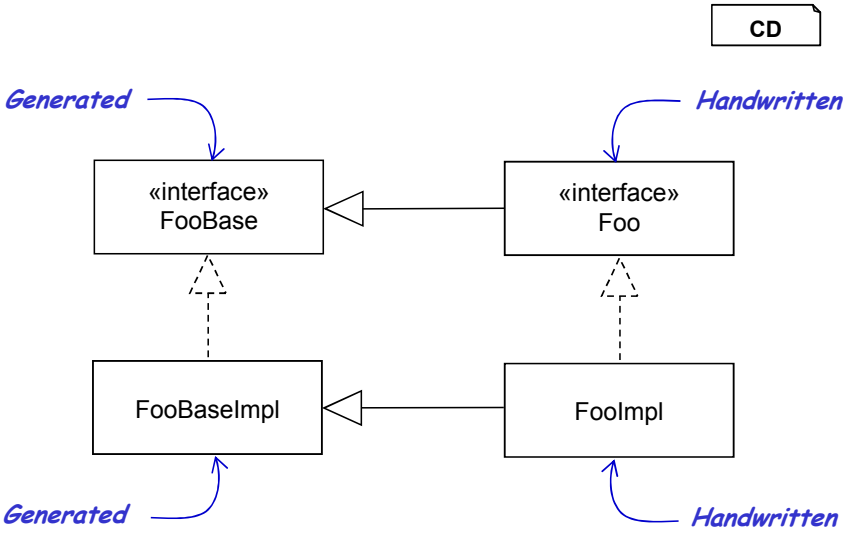


Figure 6.1: Adapted version of the extended generation gap mechanism for the integration of generated and handwritten code.

We believe that the use of the DSA systems is particularly interesting in our context for multiple reasons:

- It meets the technological restrictions of our tool, since a) the implementation language is Java and b) Eclipse is used as the development environment.
- It provides a large code base.
- A generative architecture is already defined. This offers interesting possibilities for the validation process, as we will study in the next sections.
- We count on the collaboration of the software engineers in DSA. Their knowledge about both MDE techniques and the DSA system can help in the interpretation of the results.

Each software system in DSA typically consists of hundreds of plugin projects, while the total number of artifacts in a system is in the order of thousands. Dealing with such a large code base hinders both the application of the features and the interpretation of the results. Consequently, in order to have a more manageable subject system for this case study we decided to select only a subset of the projects. A total of 176 projects have been selected for this case study. The selected set includes a subset of projects which are common to all the systems and define the artifacts that are directly related to the domain. These projects contain all the generated code, but also additional parts that have been manually implemented. The rest of the selected projects only contain handwritten code.

6.2 Research questions

The research questions define the issues that we want to investigate and eventually guide the whole validation process. We have identified four research questions, considering the characteristics of our tool and the subject system.

RQ1. How many generation candidates are reported by each feature?

With this research question we analyze the quantity of results. We analyze the total amount of generation candidates reported by each feature and its proportion to the total size of the projects. This provides a rough measure of its utility. However, other aspects need to be considered, like the quality of the reported candidates and the combination of the results from different features. Following research questions deal with such factors and allow drawing more significant conclusions from the results.

RQ2. Are the candidates reported by each feature actually generatable?

With this research question we analyze the quality of the results. A great number of reported candidates is not significant if most of them can not be generated. Therefore, it is important to consider not only the total amount of generation candidates, but also analyze how many of them are actually generatable.

RQ3. What percentage of the already generated code is reported by the features as generation candidates?

With this research question we study if the results are complete. In order to do that, we analyze the results and measure how much of the already generated code gets reported as generation candidates. Ideally, all the generated code should be reported. If, on the contrary, a feature identifies only part of the generated code as candidates, then we can assume that some candidates are also missed when applying the feature on non-generated code.

RQ4. What percentage of the total code can be newly generated based on the combined results of the features?

With this research question we analyze the overall utility of the features. Previous research questions give measures on the total amount of identified and generatable candidates. We analyze here the combination of these results and give a final measure on the total amount of unique candidates identified by all the features, i.e. the set of candidates reported by at least one of the features.

6.3 Validation procedure

In order to answer the research questions, we use the four features for which an implementation has been provided. The features are applied on the subset of projects selected in Section 6.1. We describe below the way each feature has been configured for our case study.

Name similarity

The name similarity feature is applied on the whole code of the projects. The following configuration values are used:

- Replacement identifier: *X*.
- Calculate the similarity between the identified structures of files: *true*.

Custom clone detection

The custom clone detection feature is applied in combination with the name similarity feature. This means that the code similarity is only calculated on the groups of artifacts identified by the name similarity feature, and not on the whole code of the projects. The custom clone detection process is launched with the following configuration values:

- Algorithm: *Greedy String Tiling (GST).*
- Pre-processing steps:
  - Step nº0: *false.*
  - Step nº1: *true.* Step weight: *0,30.*
  - Step nº2: *true.* Step weight: *0,40.*
  - Step nº3: *true.* Step weight: *0,15.*
  - Step nº4: *true.* Step weight: *0,15.*
- Result constructor: *Token constructor.*
- Tokenizer: *Eclipse AST Tokenizer.*
- Minimum match length: *6.*

However, the name similarity feature can sometimes identify groups that contain hundreds of related artifacts. In such cases, the number of total combinations skyrockets: for a group containing 100 related artifacts, 4950 comparisons are performed, while for 500 related artifacts there are 124750 different combinations. With the configuration above, such a high number of comparisons has tremendous implications on the performance of the feature. In order to solve this problem, we decided to set a threshold. Only groups containing 50 or less artifacts are processed by this feature. This limitation hinders the study of the feature and the interpretation of its results. However, it was a necessary measure due to time and resource constraints.

Clone detection

The clone detection feature is applied on the whole code of the projects. The default configuration is used, which defines the following values:

- Tool: *Simian.*
- Threshold: *5.*
- Default language: *Java.*
- Ignore the following:
  - Curly braces: *true.*
  - Identifiers: *false.*
  - Strings: *true.*
  - Numbers: *true.*
  - Characters: *true.*

- Literals: *true*.
- Subtype names: *false*.
- Modifiers: *true*.
- Variable names: *true*.

Design patterns

The design patterns feature is applied on the whole code of the projects. The selected tool does not offer any configuration option. The root folder of every project is selected and all the available patterns are searched in the code. When analyzing the results, the instances identified for the `Proxy` and `Proxy2` patterns are counted together as instances of the `Proxy` pattern.

Each research question is then answered in the way we describe below.

RQ1. How many generation candidates are reported by the features?

The artifacts that are already generated are filtered out of the set of generation candidates. We then obtain the total number of candidates identified by each feature, and the total number of handwritten artifacts in the projects. Depending on the feature, additional measures are shown for a more detailed quantitative analysis of the results.

RQ2. Are the candidates reported by the features actually generatable?

The artifacts that are already generated are filtered out of the set of generation candidates. For each feature, a subset of the generation candidates is manually analyzed, to see if they can actually be generated or not. From the analysis of this subset we estimate the proportion of candidates in the total results that is generatable.

RQ3. What percentage of the already generated code is reported by the features as generation candidates?

Only the artifacts that are already generated are considered, filtering out the rest of candidates. The total number of candidates reported by each feature is shown. Then, we analyze the combined results from all the features. A final measure is given with the total number of candidates reported by all the features in proportion to the total amount of generated artifacts in the projects.

RQ4. What percentage of the total code can be newly generated based on the combined results of the features?

The artifacts that are already generated are filtered out of the set of generation candidates. We analyze the combination of the results reported by all the features. A final measure is obtained with the total amount of unique generation candidates identified by the tool. This result is compared to the total size of the analyzed projects. Finally, the percentages obtained in RQ2 are used to obtain an estimation of the generatable candidates reported by the features.

6.4 Discussion of the results

In this section, we answer each research question by presenting and analyzing the obtained results.

6.4.1 RQ1. How many generation candidates are reported by the features?

There is a total of 9959 handwritten artifacts in the selected set of projects. We analyze the generation candidates reported by each feature.

Name similarity

Table 6.1 shows the number of identified generation candidates through two measures:

- The total number of identified groups of related artifacts. For this purpose, only groups containing at least one handwritten artifact are considered.
- The total number of artifacts belonging to the identified groups. For this purpose, only handwritten artifacts are considered.

Total number of groups	460
Total number of artifacts	3543

Table 6.1: Total number of handwritten generation candidates identified by the name similarity feature.

The name similarity feature has detected a great amount of groups of related artifacts. Up to 35,58% of the handwritten artifacts in the projects are contained in one of these groups. This result indicates how often programmers follow the usual conventions when naming the artifacts in a system.

Each group contains on average 7,7 artifacts. However, this does not give an accurate idea about the actual size of the identified groups. Figure 6.2 shows the distribution of the groups' sizes. We decided to use non-uniform intervals to better represent the frequency of the groups' sizes. Only groups with at least 2 elements are reported by the feature. As depicted in the chart, most of the identified groups of related artifacts contain only 2 artifacts. There is also an important number of small groups containing between 3 and 10 artifacts, while bigger groups are much less common. The feature has also identified very large groups containing hundreds of artifacts. In general, big groups are more interesting. If all the artifacts in a group can be generated together, then large groups can potentially automate the generation of more code than smaller ones.

Custom clone detection

The artifacts contained in each group, as identified by the name similarity feature, are compared pair-wise. All the results from these comparisons are reported by the feature. Therefore, if we want to measure the amount of generation candidates reported by this feature we need to set a threshold to classify the results. We define two different conditions. If the comparison result between two artifacts satisfies at least one of these conditions, then the artifacts are considered generation candidates:

- The similarity measure is greater than 30.
- The percentage of similar code is greater than 30% in any of the two artifacts.

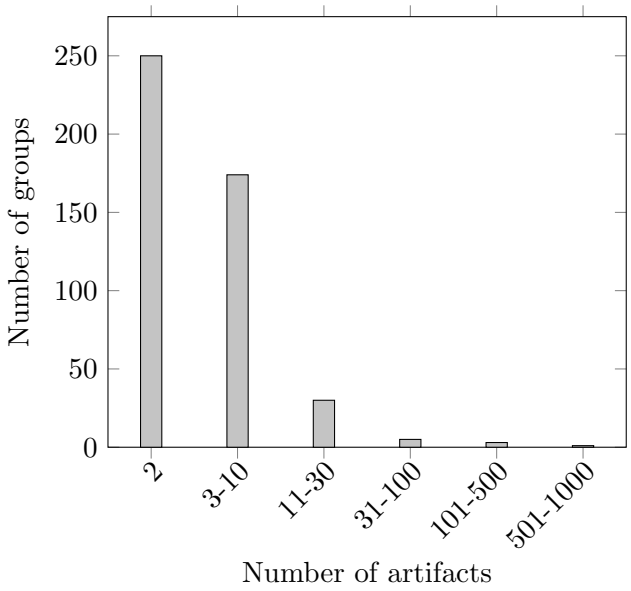


Figure 6.2: Distribution of the size of the identified groups.

We decided to set low threshold values to minimize the number of missed generation candidates. This decision will be evaluated in the context of RQ2 after studying the quality of the obtained results. In order to obtain the total number of generation candidates, the results have been filtered in the following way:

- Only handwritten artifacts are considered.
- Only artifacts are considered for which there is at least one comparison result above any of the threshold values.

A total amount of 640 generation candidates has been identified. This number of identified generation candidates represents only a 6,43% of the total amount of handwritten artifacts in the projects. However, we should not consider this total number, since the custom clone detection feature has not been applied on the whole code of the projects. Instead, it was only applied on the artifacts reported by the name similarity feature. Moreover, due to performance concerns, not every group of related artifacts reported by the name similarity feature was processed. The custom clone detection feature was configured to ignore groups containing more than 50 artifacts, as it was described in Section 6.3. In our case study, up to 11 groups with more than 50 artifacts were reported by the name similarity feature, containing 1797 handwritten artifacts. Therefore, only 1746 out of the 3543 artifacts reported by the name similarity feature were processed by our custom clone detection method. In consequence, the final number of obtained generation candidates represents a 36,66% of the analyzed artifacts.

Clone detection

Table 6.2 shows the number of identified generation candidates through three measures:

- The total number of identified sets of clones. For this purpose, only sets containing at least one handwritten artifact are considered.

- The total number of different artifacts in the identified sets. For this purpose, only handwritten artifacts are considered.
- The total number of LOC in all the clones. For this purpose, the number of cloned LOC in each set is multiplied by the number of clones in that set that satisfy the condition above.

Total number of sets	24018
Total number of artifacts	4826
Total number of LOC	14966710

Table 6.2: Total number of handwritten generation candidates identified by the clone detection feature.

The clone detection feature has detected a great amount of clones in the projects. Up to 48,46% of the handwritten artifacts are contained in at least one clone set, and almost a million and a half cloned LOC have been identified. Figure 6.3 shows the distribution of the sets' sizes in terms of number of contained clones. We decided to use non-uniform intervals to better represent the frequency of the set sizes. A logarithmic scale has been selected for the number of clone sets. It works better than the usual linear scale in this case, where big differences exist between the frequencies of each interval. It is important to notice that Simian, our selected clone detection tool, also reports clones contained in the same artifact. Therefore, the number of clones does not necessary equal the number of artifacts contained in the set.

Only sets with at least 2 clones are reported by the feature. There is a great variability in the sizes of the sets, ranging from 2 to thousands of contained clones. As it is depicted in the chart, a great amount of clone sets contain only 2 clones. There is also an important number of small sets containing between 3 and 10 clones. The frequency decreases with the size of the sets, with a few hundreds ranging from 11 to 500 and hardly a hundred clone sets containing more than 500 clones.

When analyzing the size of the sets it is not only important to measure the number of contained clones, but also the amount of cloned LOC that have been detected in those clones. Figure 6.4 shows the distribution of the sets' sizes in terms of cloned LOC. Once again, we decided to use non-uniform intervals for the sizes as well as a logarithmic scale for the number of sets. According to our configuration, only sets containing at least 5 cloned LOC are reported by the feature. As it is depicted in the chart, most clone sets contain only between 5 and 10 cloned LOC. Bigger sets are also common, with up to 500 cloned LOC identified. The feature has also detected some sets with thousands of LOC. However, sets containing more than 500 LOC are rare.

In general, small sets are less interesting from a generative point of view. In order to achieve the automatic generation of the identified candidates, they must be integrated into the generative architecture. This requires a certain effort that is only justified if it automates the generation of a significant amount of code. In this sense, clone sets containing only two clones do not offer in most occasions enough generation opportunities. The same applies to clone sets which only identify 5-10 cloned LOC. However, these candidates can sometimes be interesting if, after a manual analysis of the clones, we are able to identify further parts of the artifacts which can be generated. Nevertheless, in general a significant percentage of the reported results are not particularly interesting for generative purposes, due to the

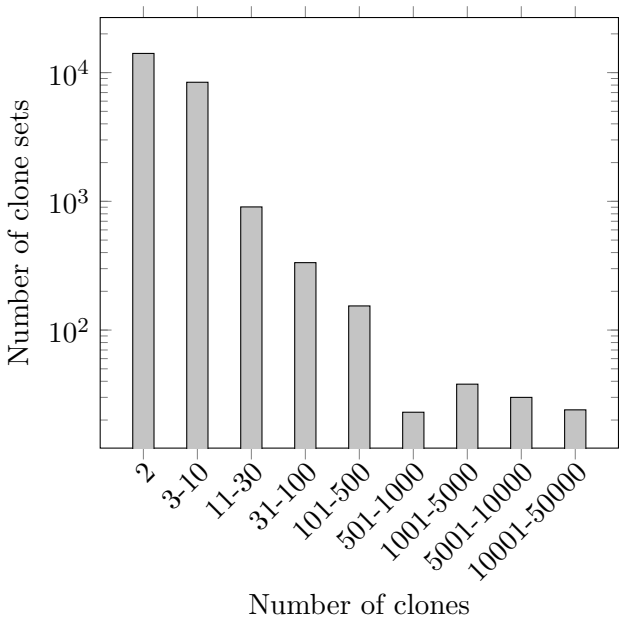


Figure 6.3: Distribution of the size of the identified clone sets, in terms of contained clones.

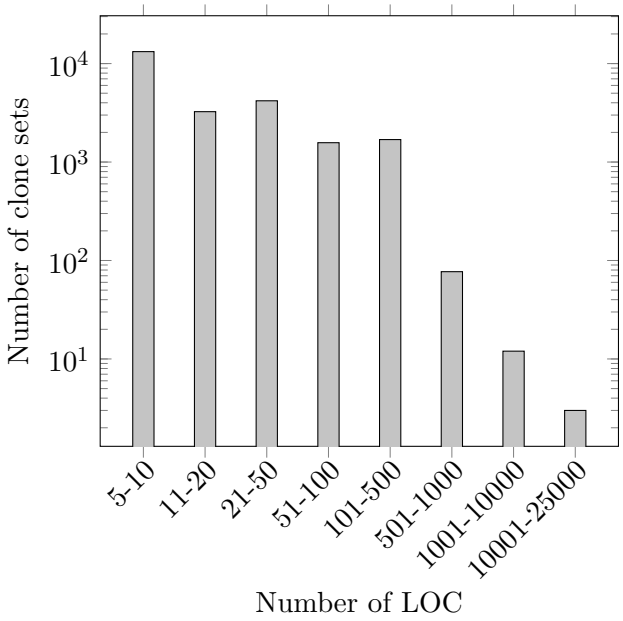


Figure 6.4: Distribution of the size of the identified clone sets, in terms of cloned LOC.

small size of the clone sets. Big clone sets are usually more relevant, since a great amount of clones and/or cloned LOC potentially allows the generation of more code. However, we have found that clone sets that are too large, both in terms of contained clones and LOC, are usually erroneously identified or just not interesting for generation purposes. Sets containing thousands of clones usually just detect a few non-significant cloned LOC, often contained multiple times in the same artifact. Similarly, sets that identify a great amount of LOC usually contain non-Java code that is not handled well by the Simian clone detection tool. Nevertheless, this is related to the quality of the results reported by our specific clone detection tool, Simian. We will study the quality of the results in more detail in the context of RQ2.

Design patterns

Table 6.3 shows the number of identified generation candidates through two measures:

- The total number of design pattern instances. For this purpose, only instances containing at least one handwritten artifact are considered.
- The total number of artifacts implementing the identified instances. For this purpose, only handwritten artifacts are considered.

Total number of instances	547
Total number of artifacts	570

Table 6.3: Total number of handwritten generation candidates identified by the design pattern feature.

Figure 6.5 shows the distribution of the number of instances according to the design pattern.

Out of the 15 design patterns supported by the tool, at least one instance has been identified for 13 of them, for a total of more than 500 instances. The *Singleton* pattern is by far the most common, followed by the *Template Method* pattern. Multiple instances have also been identified for the *Factory Method*, *Adapter*, *State* and *Visitor* patterns. The remaining detected patterns are not very significant, with only a few reported instances.

However, not every artifact involved in the implementation of a design pattern is generatable. This depends on the specific pattern and the role that the artifact plays in its implementation, as discussed in the description of this feature. Therefore, these numbers are only illustrative. A more detailed analysis of the instances of each pattern is required to obtain an actual measure of the amount of code that can be generated. Nevertheless, there is one additional measure that we can calculate in order to estimate the generative potential of the identified patterns. To obtain this measure, we take into account the generatability values of the design patterns, as presented in Section 3.6.2. We normalize generatability values to a [0, 1] range, and multiply them by the number of instances identified for the corresponding pattern. The results are depicted in Figure 6.6. The so-called generative potential measure is only meant to be used in the comparison of the different design patterns. It has, however, no direct interpretation regarding the amount of code that can be generated from them.

Calculating the generative potential allows us to derive more significant conclusions about the results. The *Singleton* pattern, for example, is not the most interesting from a generative point of view, despite being by far the most common in the projects. This is due to its

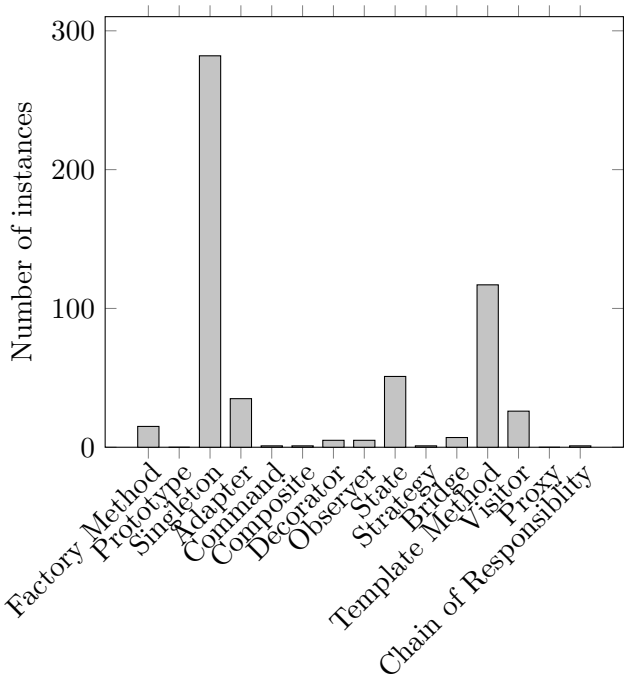


Figure 6.5: Instances identified for every design pattern.

simplicity, which only allows the generation of a small portion of code. The *State* pattern, on the contrary, has less identified instances but with greater generation possibilities, and therefore arises as the most interesting pattern in our test case. Finally, other patterns like *Template Method* or *Adapter*, despite being identified multiple times, are of no interest for us because no code artifacts can be automatically generated from them.

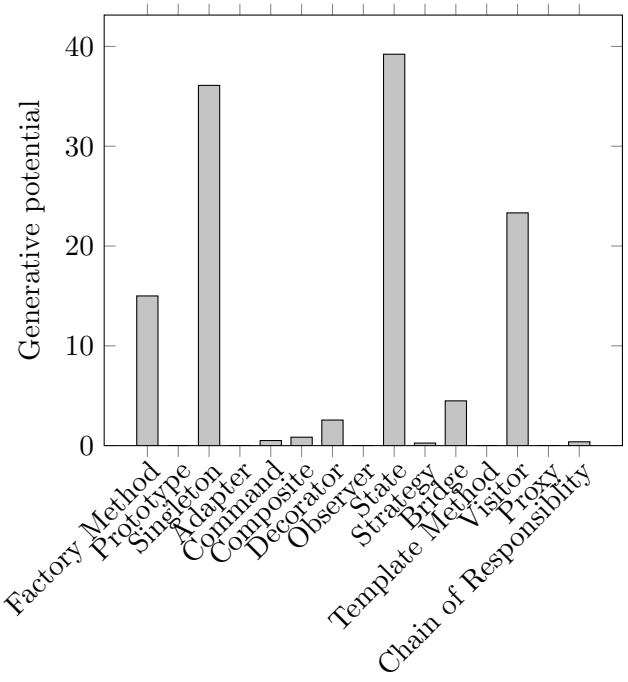


Figure 6.6: Generative potential of every design pattern in our case study.

6.4.2 RQ2. Are the candidates reported by the features actually generatable?

We analyze the results reported by each feature.

Name similarity

We have analyzed a random set of 20 generation candidates. The inner code of the artifacts forming each group has been manually studied to see how many of the candidates can be generated together. Figure 6.7 shows the candidates found generatable and non-generatable after the analysis.

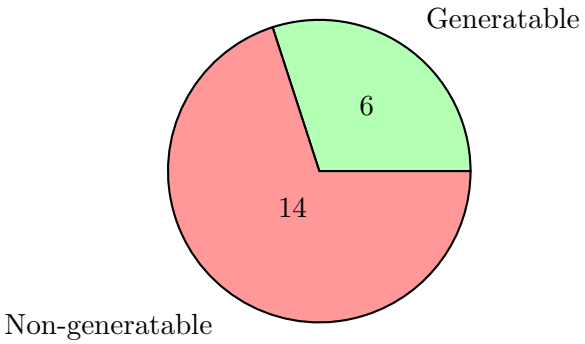


Figure 6.7: Results of the manual analysis of a set of candidates for the name similarity feature.

Up to 70% of the candidates have been found to be non-generatable. There were groups of related non-code artifacts such as images. In other occasions, the artifacts were incorrectly identified as related due to the existence of files in the project with short names. Sometimes these names can be easily contained in other artifacts even though they are not related whatsoever. Finally, for multiple candidates there was an actual relation between the names, but the code was just not found likewise related.

Only a 30% of the analyzed candidates are identified as generatable. However, the generatability of these candidates is not equal. Some of them were perfect candidates, with artifacts whose code could be almost completely generated together. In other occasions, the relation was weaker and only a small part of the code was generatable. Finally, in some groups only a portion of the artifacts were related, with additional artifacts being incorrectly included in the group.

In general, the percentage of generatable candidates is not very high. However, we are only taking into account the names of the artifacts for the application of this feature. Considering its simplicity, the method reveals itself as a useful utility for the identification of generation candidates.

If we extrapolate the obtained results to the total number of candidates reported by the feature, we obtain a total of 138 groups of generatable artifacts in the projects. Certainly, such an extrapolation is risky, since the analyzed sample is not very big. Nevertheless, it helps us to better estimate the real usefulness of the feature in our case study.

The combined application of the name similarity and custom clone detection features provides an alternative way to estimate the number of generatable candidates. Custom clone detection obtains a total similarity measure for every identified group of candidates.

It is calculated as the average of the similarities of every pair of artifacts in the group. We can use this total measure in order to classify the groups of artifacts. Figure 6.8 shows the distribution of the groups' similarities. We decided to use non-uniform intervals to better represent the frequency of the similarity values.

We have only considered groups for which the custom clone detection feature could be applied. This means that groups of non-Java artifacts are ignored, as well as groups containing more than 50 artifacts. Out of the 314 remaining groups of candidates, 58 groups were found completely different, with a total similarity of 0. Most of the groups have a total similarity value between 1 and 100, with more similar groups being much less common. From these results, we can estimate the total amount of generatable candidates. We reuse the similarity threshold of 30, defined in the context of RQ1. Every group of related artifacts with a total similarity above this threshold is considered generatable. Of course, the validity of this assumption depends on the reliability of the custom clone detection feature, which we study below.

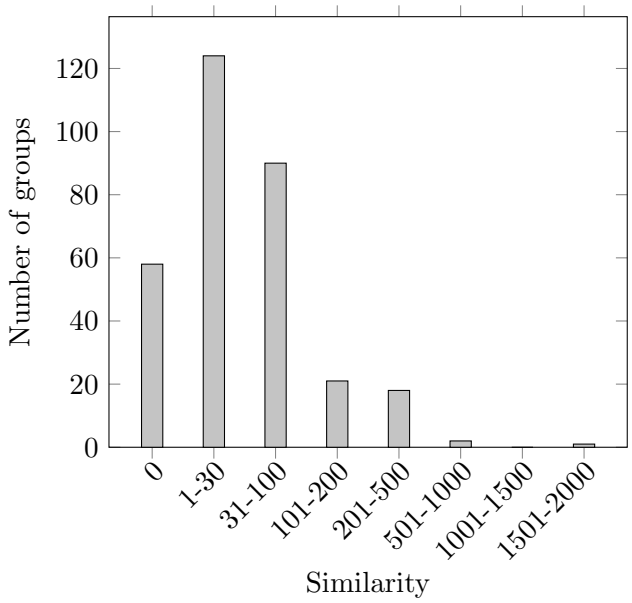


Figure 6.8: Distribution of the similarity of the identified groups.

Figure 6.9 shows the total number of candidates classified according to the similarity threshold. Only 132 groups reported by the name similarity feature have a total similarity value above the threshold. They represent a 42,04% of the groups for which a similarity value was obtained, i.e. excluding groups containing non-Java artifacts and/or more than 50 artifacts. However, if we consider all the 640 groups reported by this feature, they amount to only a 29% of them. Interestingly enough, this percentage is pretty similar to the one that we obtained from the manual analysis of a set of candidates.

Custom clone detection

We have analyzed a random set of 20 similarity comparisons that satisfy the conditions presented in RQ1. For every comparison, the inner code of both artifacts has been manually checked to see if they can be generated together. Figure 6.10 shows the candidates found generatable and non-generatable after the analysis.

Up to 40% of the candidates have been found to be non-generatable. In most cases they are artifacts that have a similar functionality, but their implementations are just too different

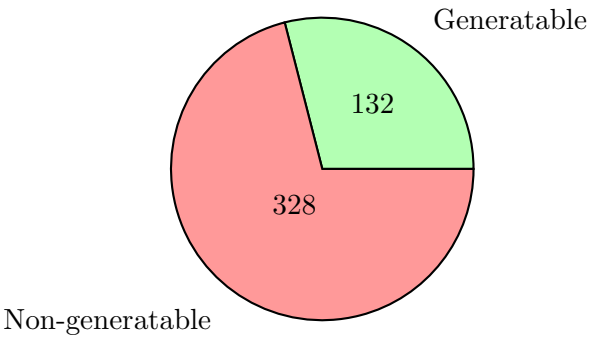


Figure 6.9: Classification of the groups according to their total similarity.

and no common generation is possible. Up to 60% of the analyzed candidates are identified as generatable. For approximately half of them a complete or almost complete generation is possible, while for the rest of candidates the code can only be partially generated. In general, the performance of this feature was really good. The ratio of generatable candidates is considerably higher than in the other features. This may be in part due to the fact that the custom clone detection process is only applied on the results of a previous feature, i.e. name similarity. Some artifacts have already been filtered out and the input of the custom clone detection feature is, a priori, more generatable. Therefore, the amount of uninteresting candidates is lower.

If we extrapolate the obtained results to the total number of candidates reported by the feature, we obtain a total of 384 generatable artifacts in the projects. Again, such an extrapolation is risky, since the analyzed sample is not very big. Nevertheless, it helps us to better estimate the real usefulness of the feature in our case study.

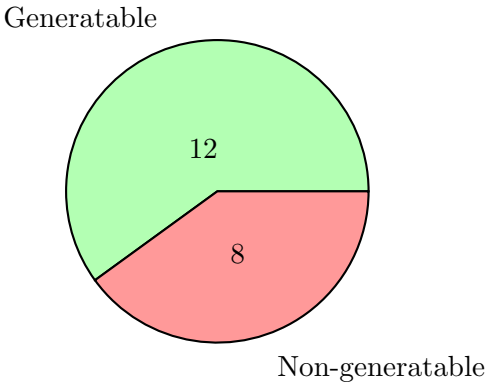


Figure 6.10: Results of the manual analysis of a set of candidates for the custom clone detection feature.

Two compared artifacts are reported as candidates if their similarity result is above the threshold. However, there can be great differences between the similarity results of the candidates' comparisons. Therefore it is also interesting to take this into account when analyzing the results. Figure 6.11 shows the final similarity values of every analyzed comparison, i.e. the value obtained after the combination of each step's result. A distinction is made between the comparisons found generatable and non-generatable. In the same way, Figure 6.12 depicts the percentages of similar code in each of the two compared code units for every generatable and non-generatable comparison. In terms of total similarity, we

do not see significant differences between the results for generatable and non-generatable candidates. A low similarity does not necessarily mean that the candidates are probably non-generatable. In fact, many of the generatable artifacts that we have identified are short classes, which can be almost completely generated despite having only a few similar lines. In the same way, non-generatable candidates with good similarity values have also been found. On the other hand, significant differences exist between the percentages of similar code found in generatable and non-generatable candidates. Figure 6.12 shows how, in general, generatable candidates have higher percentages of similar code. In most cases, more than 30% of similar code is found in at least one of the two candidates compared, while the highest percentage is usually not lower than 50-60%. Non-generatable candidates, on the contrary, rarely exceed 50% of similar code even if we take the highest percentage of the two artifacts being compared. Therefore, while the total similarity does not seem to be a deciding factor, the percentage of similar code can be a very good indicator of the generatability of the artifacts being compared.

For future executions, we can use these results in order to modify the threshold values. In this sense, it can be interesting to set a stricter threshold for the percentage of similar code. For example, we could only report as candidates those comparisons where a minimum of 50% of the code has been found similar in at least one of the two compared artifacts. Some generatable candidates would be missed, but the general quality of the results could be considerably higher.

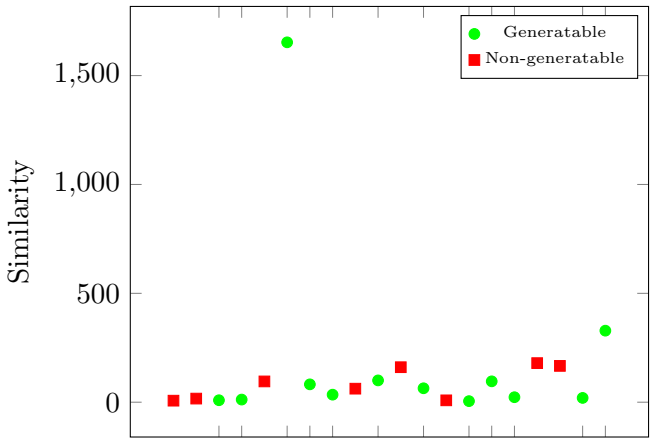


Figure 6.11: Similarity values of the analyzed candidates.

The separation of the similarity calculation process in four steps is one of the main contributions of our approach. Each step implements different pre-processing actions, with two steps implementing generation-specific actions. It is therefore particularly interesting to analyze the similarity results obtained after each step. We study the similarity increment/decrement after each pre-processing step for the candidates identified as generatable and non-generatable independently. The idea is to find differences between the evolutions of the similarity across the different steps. With our current implementation, similarity is measured in terms of tokens. However, some pre-processing actions modify the total number of tokens. Therefore, the total similarity value after each step is not a reliable measure to compare their performance. Instead, we take the percentages of similar tokens in both artifacts and obtain their average. This gives us a better idea of how similar both artifacts are found after each pre-processing step. Each measure is divided by the one obtained in the previous step, in such a way that:

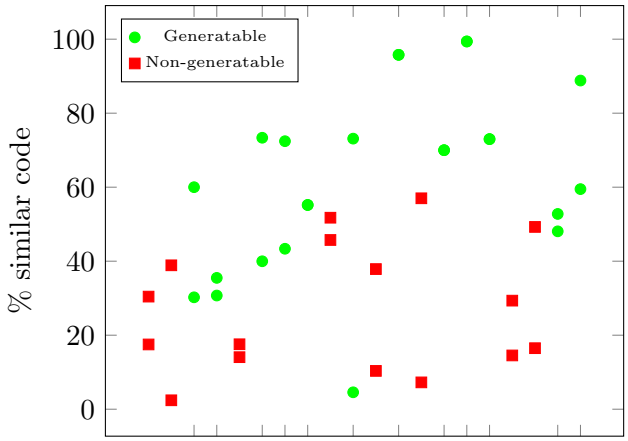


Figure 6.12: Similarity percentages of the analyzed candidates.

- A value of 1 means that the percentage of similar code have remained the same after the application of the step.
- A value greater than 1 means that the percentage of similar code have increased after the application of the step.
- A value less than 1 means that the percentage of similar code have decreased after the application of the step.

Figures 6.13 and 6.14 show the results obtained. We have added some jitter to avoid the overlap of the points. In order to correctly interpret the results, we must take into account that, in general, no great differences are found between the measures obtained in each step. Therefore, even slight increases/decreases can be significant. In this sense, there is a certain difference between the results obtained after step 2 for generatable and non-generatable candidates. In general, there are bigger increments after this step in generatable candidates. However, similarity has also increased after step 2 in some non-generatable candidates. Therefore, such a result does not guarantee the generatability of the artifacts. Step 3 had practically no effect. Only a couple of non-generatable candidates have been affected by the pre-processing actions in this step, but no conclusions can be extracted from such limited results. After the application of step 4, similarity increases in most cases, regardless of the generatability of the candidates. In fact, it seems to increase even more when the artifacts are non-generatable.

In the end, only the application of step 2 seems to provide significant information about the generatability of the candidates, and just to a limited extent. Nevertheless, the multiple-step approach remains useful, since it allows us to study the results of each set of pre-processing actions independently. From the results obtained, we can take some decisions to improve the current structure of steps. It would be important, for example, to try to include further generation-specific pre-processing actions in step 3. Alternatively, we could unify steps 2 and 3, since the latter does not have much influence in most cases anyway. Regarding step 4, the pre-processing actions should probably be less general. It would be important to try to make the comparison more accurate so that better results are obtained for generatable candidates. In any case, a more detailed analysis is necessary to draw significant conclusions. A greater amount of generatable and non-generatable

candidates should be studied. In the same vein, it could be interesting to study the differences in the results obtained after each step for the candidates identified among the generated and handwritten code.

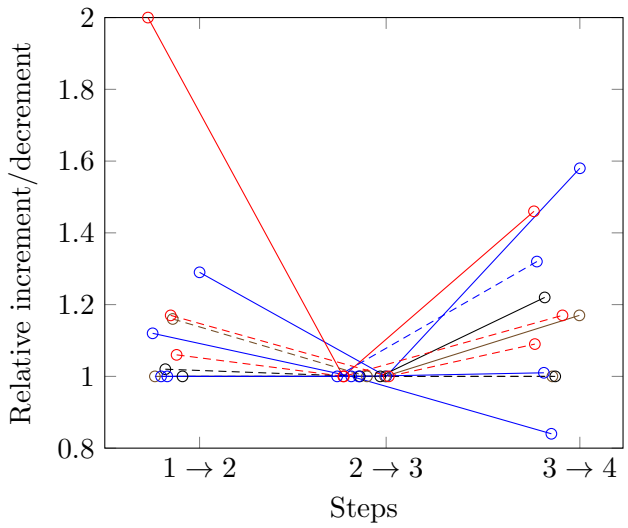


Figure 6.13: Evolution of the similarity across the pre-processing steps for the generatable comparisons.

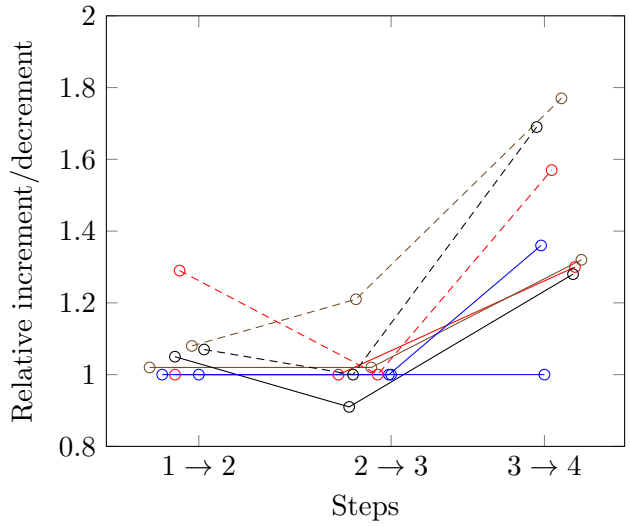


Figure 6.14: Evolution of the similarity across the pre-processing steps for the non-generatable comparisons.

There is one last issue that must be considered, and that explains to some extent why the results of steps 2 and 3 are not better. We studied the naming strategy adopted in DSA for the generated code, depicted in Figure 6.1 on page 100. For every model class `Foo`, different artifacts are created. In particular, the implementation of the model class is distributed among two interfaces and two classes. In practice, most of the implementation is contained in the generated class `FooBaseImpl`. Depending on the case, a big part of the implementation can also be handwritten in the `FooImpl` class. However, when classes related to `Foo` are identified, the handwritten interface `Foo` is taken as the domain

class, since it is the artifact named after the concept. Such an interface does not define any attributes and, in fact, in many occasions it is empty because all the important functionality was automatically generated in the `FooBase` interface. This has important consequences in the application of the generation-specific steps. For example, one of the pre-processing actions in step 2 is the replacement of all the occurrences of the domain class' attributes. The identification of the handwritten interface as the domain class makes this action useless, since no attributes are available. In order to solve this problem, the name similarity feature needs to be adapted to consider the specific naming system in DSA. However, we preferred not to include project-specific information for this case study, so that the obtained results were as unbiased as possible.

Clone detection

We have analyzed a random set of 20 generation candidates. The inner code of the artifacts forming each clone set has been manually checked to see how many of the candidates can be generated together. Figure 6.15 shows the candidates found generatable and non-generatable after the analysis.

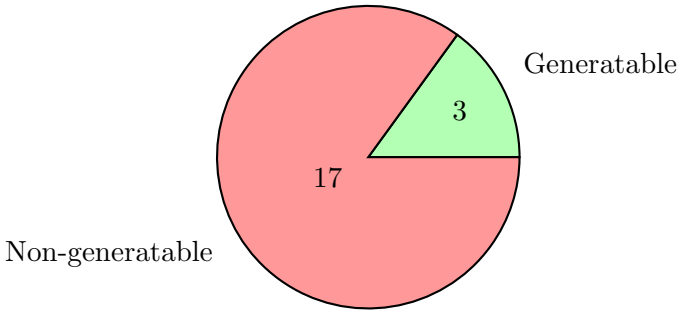


Figure 6.15: Results of the manual analysis of a set of candidates for the clone detection feature.

Only a 15% of the analyzed candidates were identified as generatable. Therefore, an 85% of them have been found to be non-generatable. Some of these non-generatable clone sets are composed of non-Java artifacts like XML and properties files. Occasionally, clones are detected in such files even if the corresponding lines are not very similar. This is probably caused by the Simian clone detection tool not handling well files with certain extensions. Most of the other uninteresting candidates correspond to large sections of code containing field declarations and/or the definition of simple `get` and `set` methods. Up to 8 of the candidates identified as non-generatable fit into this category. This reinforces the importance of eliminating simple `get` and `set` methods as part of the pre-processing of the code, something that we include in our custom clone detection method.

In general, the percentage of generatable candidates is not very high. Furthermore, since the feature reports a great amount of clone sets, the manual analysis of the results is very laborious. Therefore, it can be more convenient to apply the clone detection feature in combination with other features.

If we extrapolate the obtained results to the total number of candidates reported by the feature, we obtain a total of 360 generatable clone sets in the projects. Again, such an extrapolation is risky, since the analyzed sample is not very big. Nevertheless, it helps us to better estimate the real usefulness of the feature in our case study.

Design patterns

We have analyzed a set of 23 design pattern instances, considering at least one instance of every type of pattern that has been identified. For every instance, we have analyzed the inner code of the artifacts and manually set a fit value for the implementation of the pattern, as defined in Section 3.6. We have followed the following criteria:

- A fit value of 0 is assigned when the design pattern has been erroneously identified and is not actually implemented.
- A fit value of 1 is assigned when a standard implementation of the pattern is provided.
- A fit value between 0 and 1 is assigned when the implementation of the pattern is incomplete or deviates to some extent from the standard one.

Figure 6.16 shows the fit values assigned for all the different analyzed instances. The values are evenly distributed, and in fact the mean fit value is 0.5. This means that the selected tool identifies standard pattern implementations roughly as often as it reports incorrect results. Figure 6.17 shows the candidates identified as generatable and non-generatable after the manual analysis, according to the following criteria:

- An instance is non-generatable when $fitvalue = 0$.
- An instance is generatable when $0 < fitvalue <= 1$.

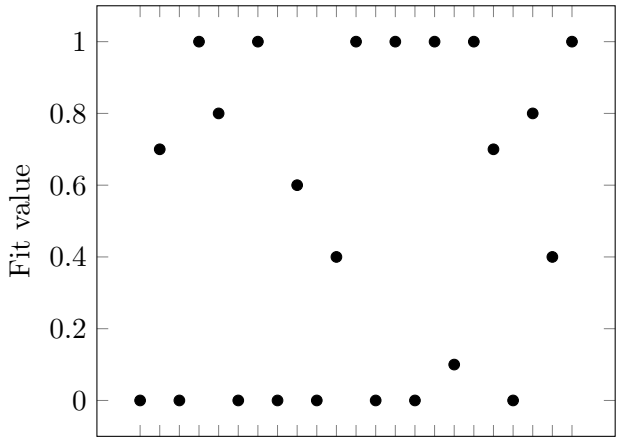


Figure 6.16: Fit values assigned to the analyzed candidates.

A 35% of the candidates have been identified as non-generatable. They are instances erroneously reported by the tool which do not implement the corresponding design pattern. The remaining 65% of the candidates have been identified as generatable. Nevertheless, the generative potential of each candidate depends ultimately on its fit value and the generatability of the implemented design pattern. In general, the obtained results are not particularly good. We have adopted a lax criteria, which defines every correctly identified pattern as generatable. Considering this, the percentage of generatable candidates should be higher. However, the tool has reported too many false positives, i.e. instances that do not implement the corresponding pattern. Furthermore, not all the generatable candidates

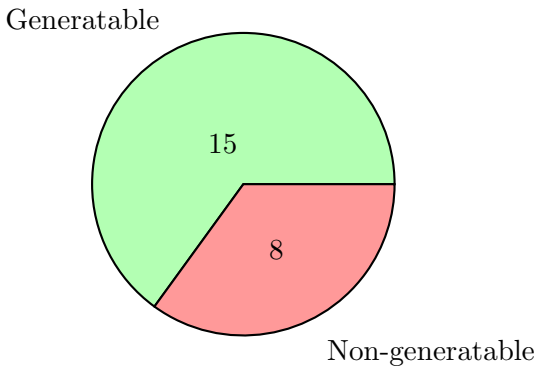


Figure 6.17: Results of the manual analysis of a set of candidates for the design patterns feature.

are standard implementations of the design patterns, and the mean fit value is lower than desired.

If we extrapolate the obtained results to the total number of candidates reported by the feature, we obtain a total of 360 generatable candidates in the projects. Again, such an extrapolation is risky, since the analyzed sample is not very big. Nevertheless, it helps us to better estimate the real usefulness of the feature in our case study.

6.4.3 RQ3. What percentage of the already generated code is reported by the features as generation candidates?

There is a total of 1985 generated artifacts in the selected set of projects. We analyze the candidates reported by each feature, and then study the combined results.

Name similarity

Table 6.4 shows the number of identified generation candidates through two measures:

- The total number of identified groups of related artifacts. For this purpose, only groups containing at least one generated artifact are considered.
- The total number of artifacts belonging to the identified groups. For this purpose, only generated artifacts are considered.

Total number of identified groups	24
Total number of identified artifacts	1856

Table 6.4: Total number of generated candidates identified by the name similarity feature.

Only 24 groups of related artifacts have been identified. However, they contain 93,5% of the generated artifacts in the projects. Therefore, the name similarity feature has detected almost all the generated code. This is mainly due to the naming strategy adopted in DSA for the generated code, which was introduced in Section 6.1. The name similarity feature identifies every `FooBase` artifact as derived from the corresponding `Foo` artifact. A big group is therefore reported which contains a great percentage of the generated artifacts.

However, not all the artifacts comply with this naming strategy, and the feature has been able to detect additional groups among them.

Custom clone detection

In order to calculate the total amount of generatable candidates, we have used the same thresholds defined for RQ1. The results have been filtered in the following way:

- Only generated artifacts are considered.
- Only artifacts are considered for which there is at least one comparison result that satisfies any of the conditions defined in RQ1.

A total amount of 38 generation candidates has been identified. This number of identified generation candidates represents only a 1,91% of the total amount of generated artifacts in the projects. However, we should not consider this total number, since the custom clone detection feature has not been applied on the whole code of the projects. Instead, it was only applied on the artifacts reported by the name similarity feature. More importantly, due to performance concerns, not every group of related artifacts reported by the name similarity feature was processed. The custom clone detection feature was configured to ignore groups containing more than 50 artifacts, as it was described in Section 6.3. Since the generative architecture produces big sets of artifacts adopting the same naming strategies, most of the generated artifacts are included in large groups, and therefore not processed by the feature. In our case study, up to 11 groups with more than 50 artifacts were reported by the name similarity feature, containing 1690 generated artifacts. Therefore, only 166 out of the 1856 artifacts reported by the name similarity feature were processed by our custom clone detection method. The final number of obtained generation candidates represents, in consequence, a 22,89% of the analyzed artifacts. The percentage of generation candidates reported by this feature is very low. In fact, it was higher when the feature was applied to the handwritten code. However, considering that only a 8,94% of the artifacts reported by the name similarity feature were processed, the results are not very significant.

Clone detection

Table 6.5 shows the number of identified generation candidates through two measures:

- The total number of identified clone sets. For this purpose, only sets containing at least one generated artifact are considered.
- The total number of clones belonging to the identified sets. For this purpose, only generated artifacts are considered.

Total number of clone sets	3003
Total number of artifacts	1776

Table 6.5: Total number of generated candidates identified by the clone detection feature.

A great number of clone sets has been identified, containing up to a 89,47% of the generated artifacts in the projects. The feature has therefore performed really good, reporting a great percentage of the generated artifacts as candidates.

Design patterns

Table 6.6 shows the number of identified generation candidates through two measures:

- The total number of design pattern instances. For this purpose, only instances containing at least one generated artifact are considered.
- The total number of artifacts implementing the identified instances. For this purpose, only generated artifacts are considered.

Total number of instances	51
Total number of artifacts	51

Table 6.6: Total number of generated candidates identified by the design patterns feature.

A few design pattern instances have been identified by the feature, amounting to only a 2,57% of the generated artifacts in the projects. Furthermore, only instances of the `Singleton` pattern have been detected. Therefore, both the amount and variety of patterns is much smaller than in the handwritten code. There is one main factor that can explain this difference. In general, the code that is generated in the projects is simpler than the manually written code. It contains base classes which are often simple POJOs, and structures that are easy to model and generate. Design patterns, on the contrary, often define complex structures of artifacts with different roles. These structures are used to solve specific problems that do not appear that often during the initial modeling and design of the system. In consequence, the use of design patterns is more common in the later manually written code than in the initially generated part of the system.

Combination of the features

We have analyzed the generation candidates reported by each feature. However, it is also necessary to study the combined results of the features. Figure 6.18 shows the existing overlap between the reported sets of candidates. There is a strong overlap between name similarity and clone detection, the two features that report by far the greatest amount of candidates. However, some candidates have also been detected only by each of those features. All the candidates identified by the custom clone detection feature are also detected by the name similarity feature. This is evident, since the former feature is only applied on the results of the latter. Moreover, they are all also detected by the clone detection feature. This means that when our custom method detects a high similarity, the traditional clone detection approach also does. Unfortunately, the limited applicability of the custom clone detection feature within the generated code hinders a more meaningful interpretation of the results. Since this feature was only applied to a small number of artifacts, we can not compare its results under the same conditions to those obtained by clone detection. Finally, it is interesting to notice that every candidate identified by the design patterns feature has not been detected by any other feature.

A total amount of 1932 different generation candidates have been reported by all the features. This represents 99,33% of the generated artifacts in the projects. These are indeed great results, since almost all the generated code has been correctly reported as candidates. Such a high percentage indicates that the ideas behind our features definitely point in the right direction when it comes to identifying generatable code.

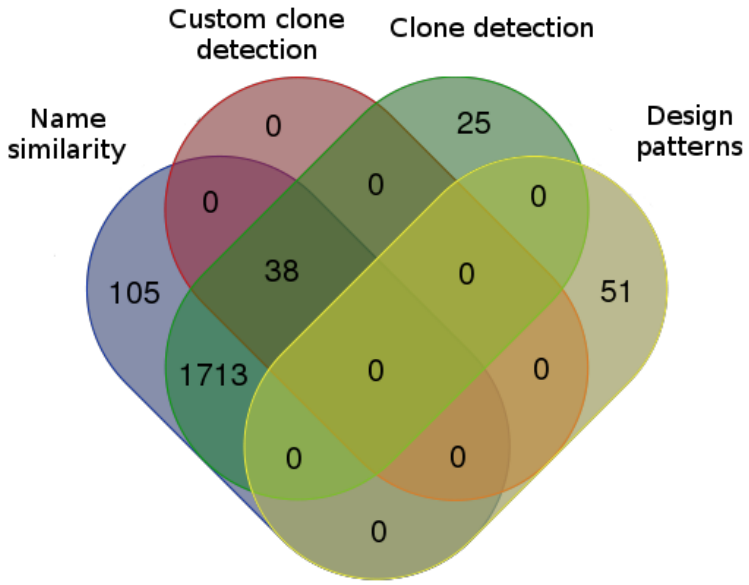


Figure 6.18: Existing overlap between the candidates reported by the features within the generated code.

6.4.4 RQ4. What percentage of the total code can be newly generated based on the generation candidates reported by the features?

Figure 6.19 shows the existing overlap between the generation candidates reported by all the features. We have used the results from RQ1, thus filtering out the artifacts that are already generated. As we have already studied in the context of RQ1, there are some important differences in the total amount of candidates identified by each feature, with name similarity and particularly clone detection as the most prolific features. Overlap exists between the results of all the features, but at the same time each feature reports some unique candidates. Custom clone detection is a special case. Since this feature is only applied on the results of name similarity, there is a full overlap between those results. A priori, candidates reported by more than one feature are more interesting, since it is more probable that they are generatable. In this sense, it would be particularly convenient to analyze the 80 candidates that have been reported by all the features.

A total amount of 6283 different generation candidates have been reported by all the features. This represents 63,09% of the non-generated artifacts in the projects. It is a considerably high percentage. However, we have studied the quality of the results in the context of RQ2, and found that many of the candidates reported by some features are eventually non-generatable. This is specially relevant for the name similarity and clone detection features, which report a great amount of generation candidates.

In the context of RQ2, a small set of candidates was analyzed for each feature to check if the candidates were actually generatable. As part of the results, we estimated the probability for a candidate reported by each feature to be generatable. We can use these probabilities to estimate the total amount of generatable candidates in the analyzed projects. Overlap needs to be considered when doing the calculations. The probabilities for an artifact to be generetable when identified by each feature are independent and non-mutually exclusive events. For the addition of non-mutually exclusive events, we apply the inclusion-exclusion

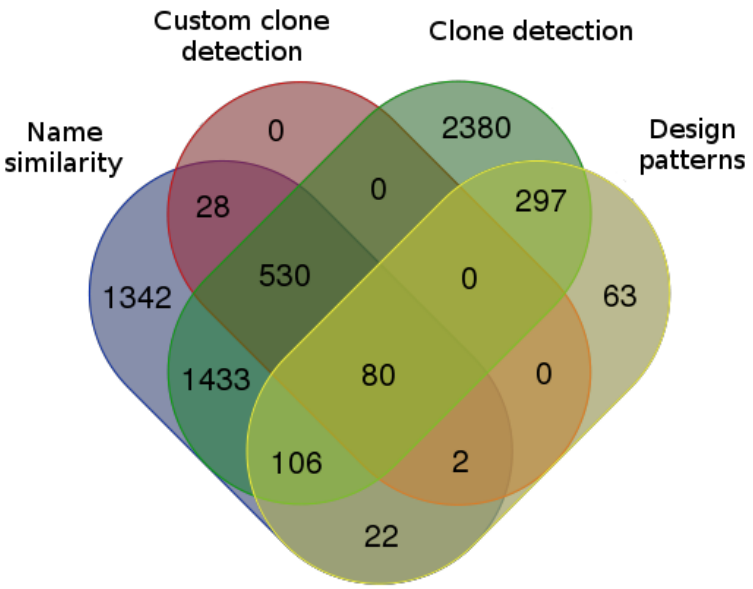


Figure 6.19: Existing overlap between the candidates reported by the features within the handwritten code.

principle [AS10]. In combinatorics, the inclusion-exclusion principle is used to calculate the number of elements in the union of finite, overlapping sets. In its general form, the principle is enunciated as follows:

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_{\emptyset \neq J \subseteq \{1,2,\ldots,n\}} (-1)^{|J|-1} \left|\bigcap_{j \in J} A_j\right|$$

Adapted to probabilities, the principle has the following form when applied to the probability of two events `a` and `b`:

$$P(a \cup b) = P(A) + P(B) - P(a \cap b)$$

where the combined probability of two independent probabilities `a` and `b` is defined as

$$P(a \cap b) = P(A) * P(B)$$

Similar formulas can be derived from the inclusion-exclusion principle to add the probabilities of three and four events.

We multiply each subset of identified candidates by its corresponding added probability of being generatable. As a result, we estimate that a total of 2195 candidates are generatable. This represents a 22,04% of the non-generated artifacts in the projects. Certainly, this estimation is risky, since the probabilities that we used were calculated from a small sample of analyzed candidates. Nevertheless, it helps us to better estimate the real usefulness of the combined features in our case study.

In general, the features performed well. A great number of generation candidates was detected. From the results obtained in RQ3, we estimate that most of the existing candidates in the projects were reported. However, the analysis performed in RQ2 demonstrated that

only some of the reported candidates are actually generatable. Therefore, a manual study of the results is always necessary. In this sense, the combination of the results from the different features offers some interesting possibilities to help in the detection of the best candidates. In the end, we estimate that the final amount of generatable candidates is considerably high in relation to the total size of the projects. As a conclusion, the application of the features in our case study demonstrates how the ideas described throughout this thesis can help in the process of identifying generation candidates.

6.5 Threats to validity

For our case study, the tool has been applied to the systems developed in DSA. In this application, not all the projects in the systems have been considered. Instead, we selected a subset of the projects to apply the features to. The selection of the systems and projects has an evident importance in the results obtained. In this sense, the conclusions that we derive from the application of the features on our case study may not hold when applied on a different software system or subset of projects. For example, the systems in DSA have one important peculiarity: part of the code is already generated. If there is already a generative architecture, we can expect that the code that is most prone to an automatic generation is already included in such an architecture. Therefore, identifying new generation candidates in the remaining handwritten code can be more difficult. In this sense, the results may be worse than those obtained from the application of the tool to a totally handwritten software system.

With the definition of this case study we do not only evaluate the usefulness of the developed tool. By extension, we also test the utility of the ideas presented and described for each feature. However, there is a certain distance between the conception of the feature and its implementation. On occasions, the implementations differ a little from the theoretical description of the method, or even contain errors. This is particularly relevant when external tools are selected to implement a feature. In this sense, the results from the case study provide a direct feedback about the performance of the implementations. However, they need to be considered more carefully when deriving conclusions about the corresponding features. Moreover, the case study only analyzes the features for which an implementation has been provided. Our tool currently supports only 4 out of the 8 defined features. In this sense, the case study provides only a partial vision of the overall utility of the ideas described in this thesis. The configuration of the features must be also considered as an additional and significant factor. The selected values have an important influence on the performance of certain features, and different results could be obtained if we used alternative configurations.

Finally, due to time constraints the analysis of the results was not complete. The custom clone detection feature could only be applied to a small subset of the artifacts. Furthermore, only a small portion of the candidates could be manually studied. A more detailed analysis would allow us to do more accurate estimations and derive more significant conclusions about the utility of the tested features.

Chapter 7

Conclusions and future work

In a MDE re-engineering context, one crucial activity is the identification of handwritten artifacts that are suitable for code generation. We denote by generation candidates those artifacts in the system that can be generated, and the process that leads to their detection identification of generation candidates. In this thesis, we have presented an approach for the semi-automatic identification of generation candidates. The goal is to provide support during this process, reducing the manual effort required. In order to do that, we have identified and described 8 generatability features. When applied to existing code, these features report which artifacts or parts of artifacts are suitable for an automatic generation. As part of this thesis, we have developed an Eclipse plugin and provided an implementation for 4 of the defined features. The performance of these features has been evaluated by their application to a real-world system.

We have studied different areas in the software research field and reused some of their ideas for the definition of our generatability features:

- **Identification of domain classes**. A categorization approach was described as our first feature, with the goal of identifying the domain classes of a system. The domain classes are the classes that implement the important concepts of the system's domain. They are generation candidates and their identification can be useful in additional parts of the process. The feature, however, was not implemented and in consequence it could not be evaluated.
- **Name similarity**. The systematic way in which artifacts in a system are usually named allows us to easily identify relations between artifacts. In this feature, a method was described which reports groups of related artifacts from the analysis of their names. An implementation of the method was provided and evaluated in our case study. The name similarity feature performed well, identifying a great number of generation candidates including almost all the already generated code in the system. The rate of generatable candidates was relatively low. However, the results were good given the simplicity of the method. The feature can be particularly useful as a first step, filtering the artifacts and applying further features to the results.
- **Clone detection**. Code similarity was identified as one of the main indicative factors for generatable code. In this feature, clone detection was proposed to take advantage of this, by reporting code clones as generation candidates. A clone detection tool was integrated into our plugin, and the feature was evaluated as part

of our case study. Clone detection was able to identify a huge number of generation candidates, including most of the already generated code in the system. However, the rate of generatable candidates was low. In order to improve the results, stricter thresholds must be set. Alternatively, a different clone detection tool could be selected, since the used tool has been found to report low-quality results. One last possibility is to use clone detection as a first step, applying further features on its results to filter out uninteresting candidates.

- **Custom clone detection**. A custom clone detection method was defined in order to respond to some deficiencies that were identified in traditional clone detection approaches. This customized approach is based on four pre-processing steps and includes actions specific to our context. An implementation of our custom clone detection method was provided as part of our Eclipse plugin and evaluated in our case study. The feature was applied in combination with name similarity. However, due to efficiency issues it could only be tested on an even smaller subset of the artifacts. This problem hindered the interpretation of the results and the comparison with other features. Nevertheless, the reported results are promising. The rate of generatable candidates was good, and a decent number of interesting generation candidates could be identified from the results. The four-step approach allowed a more meaningful interpretation of the similarity results. From their analysis, we were able to identify one particular step where small similarity increases were associated to a higher generatability of the candidates. However, other steps did not provide significant results. A more detailed analysis of the pre-processing actions defined in each step is necessary.

- **Clone evolution**. This feature further explores the usefulness of code similarity in our context, by not only identifying clones in the current version of the system but also analyzing its whole history. Control version repositories are mined in order to detect clones in previous versions and evaluate their change history. This provides additional information which can be useful for the identification of generation candidates. No implementation was provided for this feature.

- **Design patterns**. Design patterns are structured solutions to common design problems. On occasions, these structures are generatable, and therefore artifacts implementing design patterns are proposed as generation candidates. As part of this feature, we: a) studied the generatability of the main design patterns and b) defined a procedure to evaluate the identified instances from a generative perspective. An external design pattern detection tool was selected in order to evaluate this feature in the context of our case study. The use of different design patterns was detected in the system, mostly in the handwritten code. Unfortunately, the rate of correctly identified instances was not particularly high. In this sense, it would be interesting to try using a different tool for the detection of design patterns. In general, the feature did not identify a great amount of generatable code. Nevertheless, a more detailed analysis of the results is necessary in order to estimate more precisely the generation opportunities of this feature in the system.

- **Structural dependencies**. Two compilation units are structurally dependent if there is a dependency between them at compilation or linkage time. After analyzing structures of generatable classes, we identified in them two structural dependency patterns. The detection of these patterns is useful as an additional factor for the

identification of generation candidates. The structural dependencies feature was not implemented, and therefore it could not be evaluated.

- **Logical dependencies**. Two artifacts are logically dependent if they were changed together, i.e. committed together in a version control system. For this feature, we analyzed structures of generatable classes and identified in them two logical dependency patterns. The detection of these patterns is used as an additional factor in the process of identifying generation candidates. No implementation was provided for this feature.

We believe that the ideas presented in the features represent a good first step towards the automation of the identification of generation candidates process. In general, the 4 evaluated features performed well in our case study and reported a great amount of generation candidates. In the end, only a subset of the reported candidates was generatable, and a manual analysis of the results is necessary. Nevertheless, the overall manual effort required decreases, since the developer only needs to analyze the reported candidates and not the whole code base.

There are many different contributions that can be made to the work developed in this thesis. The 8 generatability features identified are just a first approach to this complex problem. Additional factors need to be taken into account, and there is room for the identification of more features. Furthermore, the features that have been described as part of this thesis can also be extended. Some of these possibilities have been commented as optimizations, but additional extensions can be defined for any of the presented features.

A tool was developed as a proof of concept for some of the ideas described in the thesis. The extension of this tool is an interesting line of future work. In this sense, it is important to provide an implementation of the remaining features, as well as a better support for the combination of the features and their results. A more advanced version of the tool will help us to draw more significant conclusions from its application to real-world systems. In this respect, the results obtained from our case study, while useful, are just a first step towards a more detailed interpretation of the ideas described in the thesis. A more thorough and complete analysis will provide a better validation of the approach and guide future work.

As a more ambitious possibility, automation efforts could go beyond the identification of generation candidates. Additional support can be provided within the MDE re-engineering process by not only helping in the detection of generatable code, but also supporting the development of the generative architecture. Such a tool would propose not only what to generate but also how, by automatically defining the corresponding models and templates from the generatable handwritten artifacts. This is undoubtedly a much more complex process, but we believe it is a promising field where some interesting progress can be achieved.

Bibliography

[Acc16] Acceleo. Acceleo, accessed February 15, 2016.

[AIS16] AIST. Ccfinderx, accessed July 17, 2016.

[ALSU06] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools (2Nd Edition)*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 2006.

[AMC+03] Deepak Alur, Dan Malks, John Crupi, Grady Booch, and Martin Fowler. *Core J2EE Patterns (Core Design Series): Best Practices and Design Strategies*. Sun Microsystems, Inc., Mountain View, CA, USA, 2 edition, 2003.

[AS10] R. B.J.T. Allenby and Alan Slomson. *How to Count: An Introduction to Combinatorics, Second Edition*. Chapman & Hall/CRC, 2nd edition, 2010.

[ASSV10] Simon Allier, Houari A. Sahraoui, Salah Sadou, and Stéphane Vaucher. *Component-Based Software Engineering: 13th International Symposium, CBSE 2010, Prague, Czech Republic, June 23-25, 2010. Proceedings*, chapter Restructuring Object-Oriented Applications into Component-Oriented Applications by Using Consistency with Execution Traces, pages 216–231. Springer Berlin Heidelberg, Berlin, Heidelberg, 2010.

[Bak95] B. S. Baker. On finding duplication and near-duplication in large software systems. In *Reverse Engineering, 1995., Proceedings of 2nd Working Conference on*, pages 86–95, Jul 1995.

[BBI+13] A. Bergmayr, H. Brunelière, J. L. C. Izquierdo, J. Gorroñogoitia, G. Kousiouris, D. Kyriazis, P. Langer, A. Menychtas, L. Orue-Echevarria, C. Pezuela, and M. Wimmer. Migrating legacy software to the cloud with artist. In *Software Maintenance and Reengineering (CSMR), 2013 17th European Conference on*, pages 465–468, March 2013.

[BM08] Peter Bulychev and Marius Minea. Duplicate code detection using anti-unification. In *Spring Young Researchers Colloquium on Software Engineering*, 2008.

[BmKPS97] Thomas Ball, Jung min Kim, Adam A. Porter, and Harvey P. Siy. If your version control system could talk..., 1997.

[BMR+96] Frank Buschmann, Regine Meunier, Hans Rohnert, Peter Sommerlad, and Michael Stal. *Pattern-Oriented Software Architecture - Volume 1: A System of Patterns*. Wiley Publishing, 1996.

[BWDP00] Lionel C. Briand, Jürgen Wüst, John W. Daly, and D. Victor Porter. Exploring the relationships between design measures and software quality in object-oriented systems. *Journal of Systems and Software*, 51(3):245 – 273, 2000.

[CIM09] Javier Luis Cánovas Izquierdo and Jesús García Molina. *A Domain Specific Language for Extracting Models in Software Modernization*, pages 82–97. Springer Berlin Heidelberg, Berlin, Heidelberg, 2009.

[CMRH09] Marcelo Cataldo, Audris Mockus, Jeffrey A. Roberts, and James D. Herbsleb. Software dependencies, work dependencies, and their impact on failures. *IEEE Trans. Softw. Eng.*, 35(6):864–878, November 2009.

[CYC+01] Andy Chou, Junfeng Yang, Benjamin Chelf, Seth Hallem, and Dawson Engler. An empirical study of operating systems errors. In *Proceedings of the Eighteenth ACM Symposium on Operating Systems Principles*, SOSP '01, pages 73–88, New York, NY, USA, 2001. ACM.

[DE05] Jens Dietrich and Chris Elgar. A formal description of design patterns using owl. In *Proceedings of the 2005 Australian Conference on Software Engineering*, ASWEC '05, pages 243–250, Washington, DC, USA, 2005. IEEE Computer Society.

[DGLP08] Marco D'Ambros, Harald Gall, Michele Lanza, and Martin Pinzger. *Analysing Software Repositories to Understand Software Evolution*, pages 37–67. Springer Berlin Heidelberg, Berlin, Heidelberg, 2008.

[DLL09] Marco D'Ambros, Michele Lanza, and Mircea Lungu. Visualizing co-change information with the evolution radar. *IEEE Transactions on Software Engineering*, 35(5):720–735, 2009.

[dRCFdS08] J. M. d. R. Costa, R. M. Feitosa, and C. R. B. d. Souza. Raisaware: Uma ferramenta de auxilio a engenharia de software colaborativa baseada em analises de dependencias. In *2008 Simposio Brasileiro de Sistemas Colaborativos*, pages 254–264, Oct 2008.

[Ecl16a] Eclipse. Eclipse documentation - methodinvocation, accessed June 29, 2016.

[Ecl16b] Eclipse. Eclipse, accessed May 16, 2016.

[ESM+02] Stephen G. Eick, Paul Schuster, Audris Mockus, Todd L. Graves, and Alan F. Karr. Visualizing software changes. *INTERACTIONS*, 17:29–31, 2002.

[FB99] M. Fowler and K. Beck. *Refactoring: Improving the Design of Existing Code*. Component software series. Addison-Wesley, 1999.

[FMM94] Marc Frappier, Stan Matwin, and Ali Mili. Software metrics study: Technical memorandum 2, 1994.

[Fou16] Python Software Foundation. The jython project, accessed July 17, 2016.

[Fow02] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 2002.

[Fow10] Martin Fowler. *Domain Specific Languages*. Addison-Wesley Professional, 1st edition, 2010.

[FR07] Robert France and Bernhard Rumpe. Model-driven development of complex software: A research roadmap. In *2007 Future of Software Engineering*, FOSE '07, pages 37–54, Washington, DC, USA, 2007. IEEE Computer Society.

[Fre16] FreeMarker. Freemarker, accessed May 23, 2016.

[GHJ98] Harald Gall, Karin Hajek, and Mehdi Jazayeri. Detection of logical coupling based on product release history. In *Proceedings of the International Conference on Software Maintenance*, ICSM '98, pages 190–, Washington, DC, USA, 1998. IEEE Computer Society.

[GHJV95] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-oriented Software*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 1995.

[GHK+15] Timo Greifenberg, Katrin Hölldobler, Carsten Kolassa, Markus Look, Pedram Mir Seyed Nazari, Klaus Müller, Antonio Navarro Pérez, Dimitri Plotnikov, Dirk Reiss, Alexander Roth, Bernhard Rumpe, Martin Schindler, and Andreas Wortmann. A Comparison of Mechanisms for Integrating Handwritten and Generated Code for Object-Oriented Programming Languages. pages 74–85, Feb 2015.

[GK10] Nils Göde and Rainer Koschke. Studying clone evolution using incremental clone detection. *Journal of Software Maintenance and Evolution: Research and Practice*, 25(2):165–192, 2010.

[Han07] N. Hanakawa. Visualization for software evolution based on logical coupling and module coupling. In *14th Asia-Pacific Software Engineering Conference (APSEC'07)*, pages 214–221, Dec 2007.

[HC01] George T. Heineman and William T. Councill, editors. *Component-based Software Engineering: Putting the Pieces Together*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 2001.

[HG11] Jan Harder and Nils Göde. Efficiently handling clone data: Rcf and cyclone. In *Proceedings of the 5th International Workshop on Software Clones*, IWSC '11, pages 81–82, New York, NY, USA, 2011. ACM.

[ID10] Nitin Indurkhya and Fred J. Damerau. *Handbook of Natural Language Processing*. Chapman & Hall/CRC, 2nd edition, 2010.

[JJ94] Manfred A. Jeusfeld and Uwe A. Johnen. *An executable meta model for reengineering of database schemas*, pages 533–547. Springer Berlin Heidelberg, Berlin, Heidelberg, 1994.

[JL06] Stan Jarzabek and Shubiao Li. Unifying clones with a generative programming technique: A case study: Practice articles. *J. Softw. Maint. Evol.*, 18(4):267–292, July 2006.

[Joh93] J. Howard Johnson. Identifying redundancy in source code using fingerprints. In *Proceedings of the 1993 Conference of the Centre for Advanced Studies on Collaborative Research: Software Engineering - Volume 1*, CASCON '93, pages 171–183. IBM Press, 1993.

[KCM07] Huzefa Kagdi, Michael L. Collard, and Jonathan I. Maletic. A survey and taxonomy of approaches for mining software repositories in the context of software evolution. *J. Softw. Maint. Evol.*, 19(2):77–131, March 2007.

[KGH06] O. Kaczor, Y. G. Gueheneuc, and S. Hamel. Efficient identification of design patterns with bit-vector algorithm. In *Conference on Software Maintenance and Reengineering (CSMR'06)*, pages 10 pp.–184, March 2006.

[KGMI06] Shinji Kawaguchi, Pankaj K. Garg, Makoto Matsushita, and Katsuro Inoue. Mudablue: An automatic categorization system for open source repositories. *Journal of Systems and Software*, 79(7):939 – 953, 2006. Selected papers from the 11th Asia Pacific Software Engineering Conference (APSEC2004).

[KKI02] T. Kamiya, S. Kusumoto, and K. Inoue. Ccfinder: a multilinguistic token-based code clone detection system for large scale source code. *Software Engineering, IEEE Transactions on*, 28(7):654–670, Jul 2002.

[KKLK05] KyoChul Kang, Moonzoo Kim, Jaejoon Lee, and Byungkil Kim. Feature-oriented re-engineering of legacy systems into product line assets – a case study. In Henk Obbink and Klaus Pohl, editors, *Software Product Lines*, volume 3714 of *Lecture Notes in Computer Science*, pages 45–56. Springer Berlin Heidelberg, 2005.

[KM07] H. Kagdi and J. I. Maletic. Combining single-version and evolutionary dependencies for software-change prediction. In *Fourth International Workshop on Mining Software Repositories (MSR'07:ICSE Workshops 2007)*, pages 17–17, May 2007.

[KN05] Miryung Kim and David Notkin. Using a clone genealogy extractor for understanding and supporting evolution of code clones. In *Proceedings of the 2005 International Workshop on Mining Software Repositories*, MSR '05, pages 1–5, New York, NY, USA, 2005. ACM.

[KSCC12] S. Kebir, A. D. Seriai, S. Chardigny, and A. Chaoui. Quality-centric approach for software component identification from object-oriented code. In *Software Architecture (WICSA) and European Conference on Software Architecture (ECSA), 2012 Joint Working IEEE/IFIP Conference on*, pages 181–190, Aug 2012.

[KSNM05] Miryung Kim, Vibha Sazawal, David Notkin, and Gail Murphy. An empirical study of code clone genealogies. *SIGSOFT Softw. Eng. Notes*, 30(5):187–196, September 2005.

[KT08] S. Kelly and J.P. Tolvanen. *Domain-Specific Modeling: Enabling Full Code Generation*. Wiley, 2008.

[Lak96] John Lakos. *Large-scale C++ Software Design*. Addison Wesley Longman Publishing Co., Inc., Redwood City, CA, USA, 1996.

[Leh11] Steffen Lehnert. A taxonomy for software change impact analysis. In *Proceedings of the 12th International Workshop on Principles of Software Evolution and the 7th Annual ERCIM Workshop on Software Evolution*, IWPSE-EVOL '11, pages 41–50, New York, NY, USA, 2011. ACM.

[LELP02] Welf Löwe, Morgan Ericsson, Jonas Lundberg, and Thomas Panas. Software comprehension - integrating program analysis and software visualization. In *in 2nd Conf. on Software Engineering Research and Practise in Sweden*, 2002.

[LH93] W. Li and S. Henry. Maintenance metrics for the object oriented paradigm. In *Software Metrics Symposium, 1993. Proceedings., First International*, pages 52–60, May 1993.

[LJ05] Seunghak Lee and Iryoung Jeong. Sdd: High performance code clone detection system for large scale source code. In *Companion to the 20th Annual ACM SIGPLAN Conference on Object-oriented Programming, Systems, Languages, and Applications*, OOPSLA '05, pages 140–141, New York, NY, USA, 2005. ACM.

[LLMZ06] Z. Li, S. Lu, S. Myagmar, and Y. Zhou. Cp-miner: finding copy-paste and related bugs in large-scale software code. *IEEE Transactions on Software Engineering*, 32(3):176–192, March 2006.

[LM93] J. P. Loyall and S. A. Mathisen. Using dependence analysis to support the software maintenance process. In *Software Maintenance ,1993. CSM-93, Proceedings., Conference on*, pages 282–291, Sep 1993.

[LS99] Ora Lassila and Ralph R. Swick. Resource Description Framework (RDF) Model and Syntax Specification. W3c recommendation, W3C, February 1999.

[LW71] Michael Levandowsky and David Winter. Distance between sets. *Nature*, 234(5):34–35, 1971.

[Mad13] Mirko Mades. Identizierung von merkmalen fur generierbaren code anhand von entwurfsmuster-analyse. Master's thesis, RWTH Aachen University, 2013.

[McG16] McGill. Clonetracker, accessed July 17, 2016.

[MT12] Jabier Martinez and Anil Kumar Thurimella. Collaboration and source code driven bottom-up product line engineering. In *Proceedings of the 16th International Software Product Line Conference - Volume 2*, SPLC '12, pages 196–200, New York, NY, USA, 2012. ACM.

[NNP+08] T. T. Nguyen, H. A. Nguyen, N. H. Pham, J. M. Al-Kofahi, and T. N. Nguyen. Cleman: Comprehensive clone group evolution management. In *Automated Software Engineering, 2008. ASE 2008. 23rd IEEE/ACM International Conference on*, pages 451–454, Sept 2008.

[NR15] Pedram Mir Seyed Nazari and Bernhard Rumpe. Using software categories for the development of generative software. *CoRR*, abs/1509.02293, 2015.

[OG11] G. A. Oliva and M. A. Gerosa. On the interplay between structural and logical dependencies in open-source software. In *Software Engineering (SBES), 2011 25th Brazilian Symposium on*, pages 144–153, Sept 2011.

[OSGdS11] Gustavo A. Oliva, Francisco W.S. Santana, Marco A. Gerosa, and Cleidson R.B. de Souza. Towards a classification of logical dependencies origins: A case study. In *Proceedings of the 12th International Workshop on Principles of Software Evolution and the 7th Annual ERCIM Workshop on Software Evolution*, IWPSE-EVOL '11, pages 31–40, New York, NY, USA, 2011. ACM.

[PMD16] PMD. Pmd - cpd, accessed July 17, 2016.

[Pri16] Princeton. Princeton university - wordnet, accessed May 16, 2016.

[PTK11] Jeremy Pate, Robert Tairas, and Nicholas Kraft. Clone Evolution: A Systematic Review. *Journal of Software Maintenance and Evolution*, September 2011.

[QYCX03] Bing Qiao, H. Yang, W.C. Chu, and Baowen Xu. Bridging legacy systems to model driven architecture. In *Computer Software and Applications Conference, 2003. COMPSAC 2003. Proceedings. 27th Annual International*, pages 304–309, Nov 2003.

[Raj97] Vaclav Rajlich. A model for change propagation based on graph rewriting. In *Proceedings of the International Conference on Software Maintenance*, ICSM '97, pages 84–91, Washington, DC, USA, 1997. IEEE Computer Society.

[RB10] Iris Reinhartz-Berger. Towards automatization of domain modeling. *Data Knowl. Eng.*, 69(5):491–515, May 2010.

[RC07] Chanchal Kumar Roy and James R. Cordy. A survey on software clone detection research. *SCHOOL OF COMPUTING TR 2007-541, QUEEN'S UNIVERSITY*, 115, 2007.

[RC08] C. K. Roy and J. R. Cordy. Nicad: Accurate detection of near-miss intentional clones using flexible pretty-printing and code normalization. In *Program Comprehension, 2008. ICPC 2008. The 16th IEEE International Conference on*, pages 172–181, June 2008.

[RCK09] Chanchal K. Roy, James R. Cordy, and Rainer Koschke. Comparison and evaluation of code clone detection techniques and tools: A qualitative approach. *Sci. Comput. Program.*, 74(7):470–495, May 2009.

[RGvD06] Thijs Reus, Hans Geers, and Arie van Deursen. Harvesting software systems for mda-based reengineering. In Arend Rensink and Jos Warmer, editors, *Model Driven Architecture – Foundations and Applications*, volume 4066 of *Lecture Notes in Computer Science*, pages 213–225. Springer Berlin Heidelberg, 2006.

[RPP+14] Davide Di Ruscio, Richard F. Paige, Alfonso Pierantonio, Jesús Sánchez Cuadrado, Javier Luis Cánovas Izquierdo, and Jesús García Molina. Special issue on success stories in model driven engineering applying model-driven

engineering in small software enterprises. *Science of Computer Programming*, 89:176 – 198, 2014.

[Rum12] Bernhard Rumpe. *Agile Modellierung mit UML*. Xpert.press. Springer Berlin, 2nd edition edition, March 2012.

[SAZ+10] R. K. Saha, M. Asaduzzaman, M. F. Zibran, C. K. Roy, and K. A. Schneider. Evaluating code clone genealogies at release level: An empirical study. In *Source Code Analysis and Manipulation (SCAM), 2010 10th IEEE Working Conference on*, pages 87–96, Sept 2010.

[SB91] Richard W. Selby and Victor R. Basili. Analyzing error-prone system structure. *IEEE Trans. Softw. Eng.*, 17(2):141–152, February 1991.

[Sel12] Bran Selic. What Will it Take? A View on Adoption of Model-Based Methods in Practice. *Software and Systems Modeling*, 11(4):513–526, 2012.

[Sim15] Devon M. Simmonds. Metrics for migrating distributed applications. In *Proceedings of the International Conference on Software Engineering Research and Practice (SERP)*, pages 217–222, 2015.

[Sim16a] Simian. Simian, accessed July 17, 2016.

[Sim16b] SimScan. Simscan, accessed July 17, 2016.

[SJSJ05] Neeraj Sangal, Ev Jordan, Vineet Sinha, and Daniel Jackson. Using dependency models to manage complex software architecture. In *Proceedings of the 20th Annual ACM SIGPLAN Conference on Object-oriented Programming, Systems, Languages, and Applications*, OOPSLA '05, pages 167–176, New York, NY, USA, 2005. ACM.

[SL16] Thomas Schmorleiz and Ralf Lämmel. Similarity management of 'cloned and owned' variants. In *Proceedings of the 31st Annual ACM Symposium on Applied Computing, Pisa, Italy, April 4-8, 2016*, pages 1466–1471, 2016.

[SO06] N. Shi and R. A. Olsson. Reverse engineering of design patterns from java source code. In *21st IEEE/ACM International Conference on Automated Software Engineering (ASE'06)*, pages 123–134, Sept 2006.

[SOdSG11] F. Santana, G. Oliva, C. R. B. de Souza, and M. Gerosa. Xflow: An extensible tool for empirical analysis of software systems evolution. In *Proceedings of the VIII Experimental Software Engineering Latin American Workshop*, ESELAW 2011, 2011.

[SRS11] R. K. Saha, C. K. Roy, and K. A. Schneider. An automatic framework for extracting and classifying near-miss clone genealogies. In *Software Maintenance (ICSM), 2011 27th IEEE International Conference on*, pages 293–302, Sept 2011.

[SSRB00] Douglas C. Schmidt, Michael Stal, Hans Rohnert, and Frank Buschmann. *Pattern-Oriented Software Architecture: Patterns for Concurrent and Networked Objects*. John Wiley & Sons, Inc., New York, NY, USA, 2nd edition, 2000.

[SSS07] Parvinder Singh Sandhu, Janpreet Singh, and Hardeep Singh. Approaches for categorization of reusable software components, 2007.

[Sta16a] Stanford. Nlp stanford - stemming and lemmatization, accessed May 16, 2016.

[Sta16b] Stanford. Stanford corenlp, accessed May 16, 2016.

[SVC06] Thomas Stahl, Markus Voelter, and Krzysztof Czarnecki. *Model-Driven Software Development: Technology, Engineering, Management*. John Wiley & Sons, 2006.

[SWM04] Michael K. Smith, Chris Welty, and Deborah L. McGuinness. Owl web ontology language guide. Technical report, W3C, February 2004.

[TC09] N. Tsantalis and A. Chatzigeorgiou. Identification of move method refactoring opportunities. *Software Engineering, IEEE Transactions on*, 35(3):347–367, May 2009.

[TCSH06] N. Tsantalis, A. Chatzigeorgiou, G. Stephanides, and S. T. Halkidis. Design pattern detection using similarity scoring. *IEEE Transactions on Software Engineering*, 32(11):896–909, Nov 2006.

[TRP09] K. Tian, M. Revelle, and D. Poshyvanyk. Using latent dirichlet allocation for automatic categorization of software. In *2009 6th IEEE International Working Conference on Mining Software Repositories*, pages 163–166, May 2009.

[Uni16] Carnegie Mellon University. Carnegie mellon university - software product lines, accessed May 31, 2016.

[vDMT10] M. von Detten, M. Meyer, and D. Travkin. Reverse engineering with the reclipse tool suite. In *2010 ACM/IEEE 32nd International Conference on Software Engineering*, volume 2, pages 299–300, May 2010.

[WHR14] Jon Whittle, John Hutchinson, and Mark Rouncefield. The State of Practice in Model-Driven Engineering. *IEEE Software*, 31(3):79–85, 2014.

[Wis93] Michael J. Wise. String similarity via greedy string tiling and running karp-rabin matching, Dec 1993.

[WJRR08] Lee White, Khaled Jaber, Brian Robinson, and Václav Rajlich. Extended firewall for regression testing: An experience report. *J. Softw. Maint. Evol.*, 20(6):419–433, November 2008.

[Xpa16] Xpand. Xpand, accessed February 15, 2016.

[YCM93] S. S. Yau, J. S. Collofello, and T. M. MacGregor. In Martin Shepperd, editor, *Software Engineering Metrics I*, chapter Ripple Effect Analysis of Software Maintenance, pages 71–82. McGraw-Hill, Inc., New York, NY, USA, 1993.

[YGSZ04] Gueheneuc Y-G, H. Sahraoui, and F. Zaidi. Fingerprinting design patterns. In *Reverse Engineering, 2004. Proceedings. 11th Working Conference on*, pages 172–181, Nov 2004.

[ZW04] Thomas Zimmermann and Peter Weißgerber. Preprocessing CVS data for fine-grained analysis. pages 2–6, Los Alamitos CA, 2004. IEEE Press.

[ZWDZ04] Thomas Zimmermann, Peter Weisgerber, Stephan Diehl, and Andreas Zeller. Mining version histories to guide software changes. In *Proceedings of the 26th International Conference on Software Engineering*, ICSE '04, pages 563–572, Washington, DC, USA, 2004. IEEE Computer Society.